

Cover

01	First	3
E	Source	5
Pages: 103		

First

api/orders/IdempotencyKeys.java

looking up a key

```

/** The order already created with this key, if any. */
Optional<String> existingOrder(String scope, String key, String fingerprint) {
    Instant now = clock.instant();
    entries.values().removeIf(entry -> entry.createdAt().plus(RETENTION).isBefore(now));
    Entry entry = entries.get(scope + "|" + key);
    if (entry == null) {
        return Optional.empty();
    }
    if (!entry.fingerprint().equals(fingerprint)) {
        throw new ApiException(ProblemType.IDEMPOTENCY_KEY_REUSED,
            "Idempotency-Key " + key + " was already used for a different order request");
    }
    return Optional.of(entry.orderId());
}

```

HTTP

```

POST /accounts/ACC-1001/orders HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI5ZjlkYzAw...
API-Version: 2
Idempotency-Key: 7d0e4c9a-3b1f-4e6d-8a52-9c0f1e2b3a4d
Content-Type: application/json

{
  "instrumentId": "EQ-MSFT",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "10",
  "limitPrice": "450.00",
  "timeInForce": "GTC",
  "clientOrderId": "rebalance-2026-09-28"
}

-----
HTTP/1.1 201 Created
Location: http://localhost:8080/accounts/ACC-1001/orders/ORD-100002
ETag: "ORD-100002-v1"
Content-Type: application/hal+json
Idempotent-Replayed: false

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {

```

```

    "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "450.00",
  "status": "OPEN",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-28T23:33:22.811774Z",
  "updatedAt": "2026-09-28T23:33:22.811774Z",
  "remainingQuantity": "10"
}

```

claims-agent.json

```

{
  "sub": "brokerage-agent",
  "aud": "brokerage-api",
  "nbf": 1790638402,
  "scope": [
    "orders:read",
    "market-data:read",
    "orders:write",
    "accounts:read"
  ],
  "iss": "http://localhost:9000",
  "trading_limit": "5000",
  "exp": 1790638702,
  "customer_id": "cust-1001",
  "iat": 1790638402,
  "jti": "9b4140cb-b528-4dd3-a3e9-748817b58705"
}

```

Complete source code

Files in this appendix

E.1 The build

pom.xml	7
auth-server/pom.xml	8
brokerage-api/pom.xml	8
brokerage-client/pom.xml	10
brokerage-agent/pom.xml	10

E.2 The authorization server

auth/AuthServerApplication.java	11
auth-server/src/main/resources/application.yaml	12

E.3 The API

api/BrokerageApiApplication.java	13
api/RootController.java	14
api/platform/ApiException.java	15
api/platform/ApiExceptionHandler.java	15
api/platform/Caller.java	17
api/platform/CursorPage.java	18
api/platform/HypermediaConfig.java	19
api/platform/JsonConfig.java	20
api/platform/ProblemType.java	20
api/platform/RateLimitedException.java	21
api/platform/RateLimiter.java	21
api/platform/RequestIdFilter.java	23
api/platform/SecurityConfig.java	23
api/platform/WebConfig.java	25
api/market/Instrument.java	26
api/market/InstrumentController.java	26
api/market/MarketData.java	28
api/market/Quotes.java	31
api/accounts/Account.java	32
api/accounts/AccountController.java	32
api/accounts/Holdings.java	34
api/accounts/Ledger.java	35
api/accounts/Portfolio.java	38
api/accounts/Reservations.java	39
api/accounts/Transaction.java	40
api/orders/IdempotencyKeys.java	40

api/orders/Order.java	41
api/orders/OrderAmendment.java	43
api/orders/OrderController.java	43
api/orders/OrderEventController.java	45
api/orders/OrderEvents.java	46
api/orders/OrderModelAssembler.java	48
api/orders/OrderPreview.java	48
api/orders/OrderRequest.java	49
api/orders/OrderService.java	50
api/orders/TradingSession.java	56
brokerage-api/src/main/resources/application.yaml	56
E.4 The API's tests	
test: api/ApiTest.java	57
test: api/ContractTest.java	58
test: api/HypermediaTest.java	60
test: api/OrderLifecycleTest.java	62
test: api/ProtocolTest.java	64
test: api/RateLimitTest.java	66
E.5 The hypermedia client	
client/ApiProblem.java	67
client/BrokerageClient.java	68
client/ClientCredentials.java	70
client/Demo.java	71
client/HalResource.java	72
client/HypermediaClient.java	73
client/Model.java	74
test: client/BrokerageClientTest.java	75
E.6 The agent	
agent/AgentConsole.java	77
agent/BrokerageAgentApplication.java	79
agent/BrokerageTools.java	80
agent/PendingActions.java	81
agent/RefusalFallbacks.java	82
brokerage-agent/src/main/resources/application.yaml	83
test: agent/BrokerageToolsTest.java	84
test: agent/GuardrailsTest.java	86
E.7 The OpenAPI contract	
openapi/brokerage-api.yaml	87

E.1 The build

The parent POM and each module's POM.

pom.xml

64 lines

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!--
3   Brokerage APIs: a hypermedia (HATEOAS) brokerage REST API on Spring Boot.
4
5   auth-server      OAuth 2.1 authorization server (Spring Authorization Server)
6   brokerage-api    the REST API: HAL + HAL-FORMS, RFC 9457 errors, OAuth 2.1 scopes
7   brokerage-client a small hypermedia client that follows links instead of building URLs
8   brokerage-agent  an AI agent (Spring AI) that uses the client as its tools
9
10  Build and test everything: ./mvnw verify
11 -->
12 <PROJECT xmlns="http://maven.apache.org/POM/4.0.0"
13         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
14         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" >
15   <MODELVERSION > 4.0.0 </MODELVERSION >
16
17   <PARENT >
18     <GROUPID > org.springframework.boot </GROUPID >
19     <ARTIFACTID > spring-boot-starter-parent </ARTIFACTID >
20     <VERSION > 4.1.1 </VERSION >
21     <RELATIVEPATH > />
22   </PARENT >
23
24   <GROUPID > com.example.brokerage </GROUPID >
25   <ARTIFACTID > brokerage-apis </ARTIFACTID >
26   <VERSION > 1.0.0 </VERSION >
27   <PACKAGING > pom </PACKAGING >
28   <NAME > Brokerage APIs </NAME >
29
30   <MODULES >
31     <MODULE > auth-server </MODULE >
32     <MODULE > brokerage-api </MODULE >
33     <MODULE > brokerage-client </MODULE >
34     <MODULE > brokerage-agent </MODULE >
35   </MODULES >
36
37   <PROPERTIES >
38     <JAVA.VERSION > 27 </JAVA.VERSION >
39     <SPRING-AI.VERSION > 2.0.1 </SPRING-AI.VERSION >
40     <JSON-SCHEMA-VALIDATOR.VERSION > 3.0.7 </JSON-SCHEMA-VALIDATOR.VERSION >
41   </PROPERTIES >
42
43   <DEPENDENCYMANAGEMENT >
44     <DEPENDENCIES >
45       <DEPENDENCY >
46         <GROUPID > org.springframework.ai </GROUPID >
47         <ARTIFACTID > spring-ai-bom </ARTIFACTID >
48         <VERSION > ${spring-ai.version} </VERSION >
49         <TYPE > pom </TYPE >
50         <SCOPE > import </SCOPE >
51       </DEPENDENCY >
52       <DEPENDENCY >
53         <GROUPID > com.example.brokerage </GROUPID >
54         <ARTIFACTID > brokerage-client </ARTIFACTID >
```

```

55     <VERSION > ${project.version} </VERSION >
56 </DEPENDENCY >
57 <DEPENDENCY >
58     <GROUPID > com.networknt </GROUPID >
59     <ARTIFACTID > json-schema-validator </ARTIFACTID >
60     <VERSION > ${json-schema-validator.version} </VERSION >
61 </DEPENDENCY >
62 </DEPENDENCIES >
63 </DEPENDENCYMANAGEMENT >
64 </PROJECT >

```

auth-server/pom.xml

38 lines

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <PROJECT xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" >
5     <MODELVERSION > 4.0.0 </MODELVERSION >
6
7     <PARENT >
8         <GROUPID > com.example.brokerage </GROUPID >
9         <ARTIFACTID > brokerage-apis </ARTIFACTID >
10        <VERSION > 1.0.0 </VERSION >
11    </PARENT >
12
13    <ARTIFACTID > auth-server </ARTIFACTID >
14    <NAME > Brokerage Authorization Server </NAME >
15    <DESCRIPTION > OAuth 2.1 authorization server for local development </DESCRIPTION >
16
17    <DEPENDENCIES >
18        <DEPENDENCY >
19            <GROUPID > org.springframework.boot </GROUPID >
20            <ARTIFACTID > spring-boot-starter-security-oauth2-authorization-server </ARTIFACTID >
21        </DEPENDENCY >
22
23        <DEPENDENCY >
24            <GROUPID > org.springframework.boot </GROUPID >
25            <ARTIFACTID > spring-boot-starter-webmvc-test </ARTIFACTID >
26            <SCOPE > test </SCOPE >
27        </DEPENDENCY >
28    </DEPENDENCIES >
29
30    <BUILD >
31        <PLUGINS >
32            <PLUGIN >
33                <GROUPID > org.springframework.boot </GROUPID >
34                <ARTIFACTID > spring-boot-maven-plugin </ARTIFACTID >
35            </PLUGIN >
36        </PLUGINS >
37    </BUILD >
38 </PROJECT >

```

brokerage-api/pom.xml

75 lines

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <PROJECT xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" >
5     <MODELVERSION > 4.0.0 </MODELVERSION >
6
7     <PARENT >

```

```

8     <GROUPID > com.example.brokerage </GROUPID >
9     <ARTIFACTID > brokerage-apis </ARTIFACTID >
10    <VERSION > 1.0.0 </VERSION >
11    </PARENT >
12
13    <ARTIFACTID > brokerage-api </ARTIFACTID >
14    <NAME > Brokerage API </NAME >
15    <DESCRIPTION > The hypermedia brokerage REST API </DESCRIPTION >
16
17    <DEPENDENCIES >
18        <DEPENDENCY >
19            <GROUPID > org.springframework.boot </GROUPID >
20            <ARTIFACTID > spring-boot-starter-hateoas </ARTIFACTID >
21        </DEPENDENCY >
22        <DEPENDENCY >
23            <GROUPID > org.springframework.boot </GROUPID >
24            <ARTIFACTID > spring-boot-starter-validation </ARTIFACTID >
25        </DEPENDENCY >
26        <DEPENDENCY >
27            <GROUPID > org.springframework.boot </GROUPID >
28            <ARTIFACTID > spring-boot-starter-security-oauth2-resource-server </ARTIFACTID >
29        </DEPENDENCY >
30        <DEPENDENCY >
31            <GROUPID > org.springframework.boot </GROUPID >
32            <ARTIFACTID > spring-boot-starter-actuator </ARTIFACTID >
33        </DEPENDENCY >
34
35        <DEPENDENCY >
36            <GROUPID > org.springframework.boot </GROUPID >
37            <ARTIFACTID > spring-boot-starter-hateoas-test </ARTIFACTID >
38            <SCOPE > test </SCOPE >
39        </DEPENDENCY >
40        <DEPENDENCY >
41            <GROUPID > org.springframework.boot </GROUPID >
42            <ARTIFACTID > spring-boot-starter-security-oauth2-resource-server-test </ARTIFACTID >
43            <SCOPE > test </SCOPE >
44        </DEPENDENCY >
45        <DEPENDENCY >
46            <GROUPID > com.networknt </GROUPID >
47            <ARTIFACTID > json-schema-validator </ARTIFACTID >
48            <SCOPE > test </SCOPE >
49        </DEPENDENCY >
50        <DEPENDENCY >
51            <GROUPID > tools.jackson.dataformat </GROUPID >
52            <ARTIFACTID > jackson-dataformat-yaml </ARTIFACTID >
53            <SCOPE > test </SCOPE >
54        </DEPENDENCY >
55    </DEPENDENCIES >
56
57    <BUILD >
58        <RESOURCES >
59            <RESOURCE >
60                <DIRECTORY > src/main/resources </DIRECTORY >
61            </RESOURCE >
62            <!-- The OpenAPI contract ships inside the application and is served at /openapi.yaml -->
63            <RESOURCE >
64                <DIRECTORY > ${project.basedir}/../openapi </DIRECTORY >
65                <TARGETPATH > contract </TARGETPATH >
66            </RESOURCE >
67        </RESOURCES >

```

```

68     <PLUGINS >
69     <PLUGIN >
70         <GROUPID > org.springframework.boot </GROUPID >
71         <ARTIFACTID > spring-boot-maven-plugin </ARTIFACTID >
72     </PLUGIN >
73 </PLUGINS >
74 </BUILD >
75 </PROJECT >

```

brokerage-client/pom.xml

38 lines

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <PROJECT xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" >
5     <MODELVERSION > 4.0.0 </MODELVERSION >
6
7     <PARENT >
8         <GROUPID > com.example.brokerage </GROUPID >
9         <ARTIFACTID > brokerage-apis </ARTIFACTID >
10        <VERSION > 1.0.0 </VERSION >
11    </PARENT >
12
13    <ARTIFACTID > brokerage-client </ARTIFACTID >
14    <NAME > Brokerage Hypermedia Client </NAME >
15    <DESCRIPTION > A client that navigates the API by following links </DESCRIPTION >
16
17    <DEPENDENCIES >
18        <DEPENDENCY >
19            <GROUPID > org.springframework.boot </GROUPID >
20            <ARTIFACTID > spring-boot-starter-restclient </ARTIFACTID >
21        </DEPENDENCY >
22        <DEPENDENCY >
23            <GROUPID > org.springframework.boot </GROUPID >
24            <ARTIFACTID > spring-boot-starter-jackson </ARTIFACTID >
25        </DEPENDENCY >
26        <!-- Only for Link and RFC 6570 URI templates such as {?symbol,type} -->
27        <DEPENDENCY >
28            <GROUPID > org.springframework.hateoas </GROUPID >
29            <ARTIFACTID > spring-hateoas </ARTIFACTID >
30        </DEPENDENCY >
31
32        <DEPENDENCY >
33            <GROUPID > org.springframework.boot </GROUPID >
34            <ARTIFACTID > spring-boot-starter-restclient-test </ARTIFACTID >
35            <SCOPE > test </SCOPE >
36        </DEPENDENCY >
37    </DEPENDENCIES >
38 </PROJECT >

```

brokerage-agent/pom.xml

46 lines

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <PROJECT xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd" >
5     <MODELVERSION > 4.0.0 </MODELVERSION >
6
7     <PARENT >
8         <GROUPID > com.example.brokerage </GROUPID >
9         <ARTIFACTID > brokerage-apis </ARTIFACTID >

```

```

10     <VERSION > 1.0.0 </VERSION >
11 </PARENT >
12
13 <ARTIFACTID > brokerage-agent </ARTIFACTID >
14 <NAME > Brokerage AI Agent </NAME >
15 <DESCRIPTION > A Spring AI agent that uses the brokerage API through tools </DESCRIPTION >
16
17 <DEPENDENCIES >
18     <DEPENDENCY >
19         <GROUPID > com.example.brokerage </GROUPID >
20         <ARTIFACTID > brokerage-client </ARTIFACTID >
21     </DEPENDENCY >
22     <DEPENDENCY >
23         <GROUPID > org.springframework.ai </GROUPID >
24         <ARTIFACTID > spring-ai-starter-model-anthropic </ARTIFACTID >
25     </DEPENDENCY >
26     <DEPENDENCY >
27         <GROUPID > org.springframework.boot </GROUPID >
28         <ARTIFACTID > spring-boot-starter-security-oauth2-client </ARTIFACTID >
29     </DEPENDENCY >
30
31     <DEPENDENCY >
32         <GROUPID > org.springframework.boot </GROUPID >
33         <ARTIFACTID > spring-boot-starter-restclient-test </ARTIFACTID >
34         <SCOPE > test </SCOPE >
35     </DEPENDENCY >
36 </DEPENDENCIES >
37
38 <BUILD >
39     <PLUGINS >
40         <PLUGIN >
41             <GROUPID > org.springframework.boot </GROUPID >
42             <ARTIFACTID > spring-boot-maven-plugin </ARTIFACTID >
43         </PLUGIN >
44     </PLUGINS >
45 </BUILD >
46 </PROJECT >

```

E.2 The authorization server

auth-server: Spring Authorization Server, configured for three clients and one demo customer.

auth-server/src/main/java/com/example/brokerage/auth/AuthServerApplication.java

64 lines

```

1 package com.example.brokerage.auth;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5 import java.util.Map;
6
7 import org.springframework.boot.SpringApplication;
8 import org.springframework.boot.autoconfigure.SpringBootApplication;
9 import org.springframework.boot.context.properties.ConfigurationProperties;
10 import org.springframework.boot.context.properties.EnableConfigurationProperties;
11 import org.springframework.context.annotation.Bean;
12 import org.springframework.security.oauth2.server.authorization.OAuth2TokenType;
13 import org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
14 import org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
15
16 /**
17  * A development-only OAuth 2.1 authorization server. Clients, scopes and the demo user are
18  * declared in application.yaml; this class adds the two brokerage-specific claims that the

```

```

19  * API relies on.
20  */
21  @SpringBootApplication
22  @EnableConfigurationProperties(AuthServerApplication.Identities.class)
23  public class AuthServerApplication {
24
25      public static void main(String[] args) {
26          SpringApplication.run(AuthServerApplication.class, args);
27      }
28
29      /**
30       * Who each token is for. A user signs in as themselves; a confidential client that uses
31       * the client_credentials grant is registered to one customer, like a personal API key.
32       * An optional per-client trading limit caps the notional value of any single order.
33       */
34      @ConfigurationProperties("brokerage.identities")
35      record Identities(Map<String, String> customers, Map<String, BigDecimal> tradingLimits) {
36          Identities {
37              customers = customers == null ? Map.of() : customers;
38              tradingLimits = tradingLimits == null ? Map.of() : tradingLimits;
39          }
40      }
41
42      /** Stamps the audience, customer_id and (if configured) trading_limit into every access token. */
43      @Bean
44      OAuth2TokenCustomizer<JwtEncodingContext> brokerageClaims(Identities identities) {
45          return context -> {
46              if (!OAuth2TokenType.ACCESS_TOKEN.equals(context.getTokenType())) {
47                  return;
48              }
49              // Tokens are minted for one API, which checks that it is the intended audience.
50              context.getClaims().audience(List.of("brokerage-api"));
51              // The principal is the signed-in user, or the client itself for client_credentials.
52              String principal = context.getPrincipal().getName();
53              String customerId = identities.customers().get(principal);
54              if (customerId != null) {
55                  context.getClaims().claim("customer_id", customerId);
56              }
57              String clientId = context.getRegisteredClient().getClientId();
58              BigDecimal limit = identities.tradingLimits().get(clientId);
59              if (limit != null) {
60                  context.getClaims().claim("trading_limit", limit.toPlainString());
61              }
62          };
63      }
64  }

```

auth-server/src/main/resources/application.yaml

64 lines

```

1  # Development-only authorization server. Plain HTTP, in-memory keys and clients:
2  # never deploy this configuration.
3  server:
4      port: 9000
5
6  spring:
7      application:
8          name: auth-server
9      security:
10         user:
11             # The demo customer, for the browser-based authorization code flow.
12             name: alice
13             password: wonderland
14         oauth2:
15             authorizationserver:
16                 issuer: http://localhost:9000
17                 client:
18                     # A customer's own script or trading bot: the "personal API key" pattern.
19                     brokerage-cli:
20                         registration:

```

```

21     client-id: brokerage-cli
22     client-secret: "{noop}cli-secret"
23     client-authentication-methods: [client_secret_basic]
24     authorization-grant-types: [client_credentials]
25     scopes: [accounts:read, market-data:read, orders:read, orders:write]
26     token:
27         access-token-time-to-live: 15m
28     # The AI agent: same customer, but every order is capped by trading_limit.
29 brokerage-agent:
30     registration:
31         client-id: brokerage-agent
32         client-secret: "{noop}agent-secret"
33         client-authentication-methods: [client_secret_basic]
34         authorization-grant-types: [client_credentials]
35         scopes: [accounts:read, market-data:read, orders:read, orders:write]
36         token:
37             access-token-time-to-live: 5m
38     # A web app's backend (a "backend for frontend"): a confidential client that signs
39     # the customer in with the authorization code grant, PKCE and consent. Tokens stay
40     # on the server; the browser holds only a session cookie. Spring Authorization
41     # Server issues refresh tokens to confidential clients only.
42 brokerage-web:
43     registration:
44         client-id: brokerage-web
45         client-secret: "{noop}web-secret"
46         client-authentication-methods: [client_secret_basic]
47         authorization-grant-types: [authorization_code, refresh_token]
48         redirect-uris: [http://127.0.0.1:3000/callback]
49         scopes: [openid, accounts:read, market-data:read, orders:read, orders:write]
50     require-proof-key: true
51     require-authorization-consent: true
52     token:
53         access-token-time-to-live: 10m
54         refresh-token-time-to-live: 8h
55         reuse-refresh-tokens: false
56
57 brokerage:
58     identities:
59         customers:
60             alice: cust-1001
61             brokerage-cli: cust-1001
62             brokerage-agent: cust-1001
63     trading-limits:
64         brokerage-agent: 5000

```

E.3 The API

brokerage-api: the hypermedia REST API, package by package.

brokerage-api/src/main/java/com/example/brokerage/api/BrokerageApiApplication.java

29 lines

```

1  package com.example.brokerage.api;
2
3  import java.time.Clock;
4
5  import org.springframework.boot.SpringApplication;
6  import org.springframework.boot.autoconfigure.SpringBootApplication;
7  import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.scheduling.annotation.EnableScheduling;
10
11 /**
12  * The brokerage REST API: accounts, positions, transactions, instruments, quotes and
13  * orders, served as HAL (and HAL-FORMS) hypermedia and protected by OAuth 2.1 scopes.
14  */
15 @SpringBootApplication
16 @EnableScheduling

```

```

17 @ConfigurationPropertiesScan
18 public class BrokerageApiApplication {
19
20     public static void main(String[] args) {
21         SpringApplication.run(BrokerageApiApplication.class, args);
22     }
23
24     /** One clock for the whole application, so time can be controlled in tests. */
25     @Bean
26     Clock clock() {
27         return Clock.systemUTC();
28     }
29 }

```

brokerage-api/src/main/java/com/example/brokerage/api/RootController.java

57 lines

```

1 package com.example.brokerage.api;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6 import java.time.Duration;
7
8 import com.example.brokerage.api.accounts.AccountController;
9 import com.example.brokerage.api.market.InstrumentController;
10
11 import org.springframework.core.io.ClassPathResource;
12 import org.springframework.core.io.Resource;
13 import org.springframework.hateoas.Link;
14 import org.springframework.hateoas.RepresentationModel;
15 import org.springframework.http.CacheControl;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.web.bind.annotation.GetMapping;
18 import org.springframework.web.bind.annotation.RestController;
19 import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
20
21 /**
22  * The API root: the one URL a client needs to know. Everything else is discovered by
23  * following links from here, so the server is free to reorganize every other URI.
24  */
25 @RestController
26 public class RootController {
27
28     /** The root's own properties, beside its links. */
29     public static class ApiRoot extends RepresentationModel<ApiRoot> {
30
31         public final String name = "Brokerage API";
32         public final String description = "Accounts, positions, market data and trading, as hypermedia";
33     }
34
35     @GetMapping("/")
36     public ApiRoot root() {
37         String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
38         return new ApiRoot()
39             .add(linkTo(methodOn(RootController.class).root()).withSelfRel())
40             .add(linkTo(methodOn(AccountController.class).accounts(null)).withRel("accounts"))
41             .add(linkTo(methodOn(InstrumentController.class).search(null, null)).withRel("instruments"))
42             // RFC 8631: the machine-readable description of this API ...
43             .add(Link.of(base + "/openapi.yaml", "service-desc").withType("application/openapi+yaml"))
44             // ... and its human-readable documentation.
45             .add(Link.of("https://docs.brokerage.example", "service-doc").withType("text/html"))
46             // RFC 9728: where clients learn which authorization server issues tokens for this API.
47             .add(Link.of(base + "/.well-known/oauth-protected-resource", "oauth-protected-resource"));
48     }
49
50     /** The OpenAPI contract, served by the application it describes so the two ship together. */
51     @GetMapping(path = "/openapi.yaml", produces = "application/openapi+yaml")
52     public ResponseEntity<Resource> contract() {
53         return ResponseEntity.ok()

```

```

54         .cacheControl(CacheControl.maxAge(Duration.ofHours(1)))
55         .body(new ClassPathResource("contract/brokerage-api.yaml"));
56     }
57 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ApiException.java

36 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.util.LinkedHashMap;
4 import java.util.Map;
5
6 /**
7  * An error the API reports on purpose. It carries its problem type, a detail message for
8  * the developer, and optional extension members (for example the buying power available).
9  */
10 public class ApiException extends RuntimeException {
11
12     private final ProblemType type;
13     private final Map<String, Object> extensions = new LinkedHashMap<>();
14
15     public ApiException(ProblemType type, String detail) {
16         super(detail);
17         this.type = type;
18     }
19
20     public ApiException with(String name, Object value) {
21         extensions.put(name, value);
22         return this;
23     }
24
25     public ProblemType type() {
26         return type;
27     }
28
29     public Map<String, Object> extensions() {
30         return extensions;
31     }
32
33     public static ApiException notFound(String what, String id) {
34         return new ApiException(ProblemType.NOT_FOUND, what + " " + id + " was not found");
35     }
36 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ApiExceptionHandler.java

119 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.net.URI;
4 import java.util.Comparator;
5 import java.util.List;
6 import java.util.Map;
7
8 import jakarta.servlet.http.HttpServletRequest;
9
10 import org.springframework.http.HttpHeaders;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.http.ProblemDetail;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.access.AccessDeniedException;
15 import org.springframework.security.core.AuthenticationException;
16 import org.springframework.web.accept.InvalidApiVersionException;
17 import org.springframework.validation.FieldError;
18 import org.springframework.web.bind.MethodArgumentNotValidException;
19 import org.springframework.web.bind.annotation.ExceptionHandler;
20 import org.springframework.web.bind.annotation.RestControllerAdvice;
21 import org.springframework.web.context.request.ServletWebRequest;
22 import org.springframework.web.context.request.WebRequest;
23 import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

```

```

24
25 /**
26  * Turns every error into an RFC 9457 problem document (application/problem+json), with a
27  * stable type URI, the request path as "instance", and the request ID for support.
28  * Spring's own exceptions (bad JSON, 405, 406, unsupported versions...) are handled by the
29  * base class; this class adds the API's catalog and a consistent shape for all of them.
30  */
31 @RestControllerAdvice
32 public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
33
34     @ExceptionHandler(ApiException.class)
35     ResponseEntity<ProblemDetail> handleApi(ApiException ex, HttpServletRequest request) {
36         ProblemDetail problem = problem(ex.type(), ex.getMessage(), request);
37         ex.extensions().forEach(problem::setProperty);
38         HttpHeaders headers = new HttpHeaders();
39         if (ex instanceof RateLimitedException limited) {
40             headers.set(HttpHeaders.RETRY_AFTER, String.valueOf(limited.retryAfterSeconds()));
41         }
42         return ResponseEntity.status(ex.type().status()).headers(headers).body(problem);
43     }
44
45     /** Raised by the security filter chain when no valid token is present. */
46     @ExceptionHandler(AuthenticationException.class)
47     ResponseEntity<ProblemDetail> handleUnauthenticated(AuthenticationException ex,
48         HttpServletRequest request) {
49         return ResponseEntity.status(ProblemType.UNAUTHORIZED.status())
50             .body(problem(ProblemType.UNAUTHORIZED, "A valid bearer token is required", request));
51     }
52
53     /** Raised when the token is valid but does not carry the scope the operation needs. */
54     @ExceptionHandler(AccessDeniedException.class)
55     ResponseEntity<ProblemDetail> handleForbidden(AccessDeniedException ex, HttpServletRequest request) {
56         return ResponseEntity.status(ProblemType.INSUFFICIENT_SCOPE.status())
57             .body(problem(ProblemType.INSUFFICIENT_SCOPE,
58                 "The access token does not grant the scope this operation requires", request));
59     }
60
61     /** An API-Version header naming a version this server does not offer. */
62     @ExceptionHandler(InvalidApiVersionException.class)
63     ResponseEntity<ProblemDetail> handleVersion(InvalidApiVersionException ex, HttpServletRequest request) {
64         ProblemDetail problem = problem(ProblemType.UNSUPPORTED_API_VERSION,
65             "API version " + ex.getVersion() + " is not supported", request);
66         problem.setProperty("supportedVersions", List.of("1", "2"));
67         return ResponseEntity.badRequest().body(problem);
68     }
69
70     /** Bean Validation failures: one problem, with every invalid field listed. */
71     @Override
72     protected ResponseEntity<Object> handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
73         HttpHeaders headers, HttpStatus status, WebRequest request) {
74         ProblemDetail problem = problem(ProblemType.INVALID_REQUEST, "One or more fields are invalid",
75             ((ServletWebRequest) request).getRequest());
76         List<Map<String, String>> errors = ex.getBindingResult().getFieldErrors().stream()
77             .sorted(Comparator.comparing(FieldError::getField))
78             .map(error -> Map.of("field", error.getField(),
79                 "message", String.valueOf(error.getDefaultMessage())))
80             .toList();
81         problem.setProperty("errors", errors);
82         return ResponseEntity.badRequest().body(problem);
83     }
84
85     /** Everything Spring raises itself gets the same instance and requestId members. */
86     @Override
87     protected ResponseEntity<Object> createResponseEntity(Object body, HttpHeaders headers,
88         HttpStatus status, WebRequest request) {
89         if (body instanceof ProblemDetail problem) {
90             HttpServletRequest servletRequest = ((ServletWebRequest) request).getRequest();
91             if (problem.getType() == null || "about:blank".equals(problem.getType().toString())) {
92                 problem.setType(URI.create(ProblemType.BASE + slugFor(status)));

```

```

93     }
94     problem.setInstance(URI.create(servletRequest.getRequestURI()));
95     problem.setProperty("requestId", RequestIdFilter.currentId(servletRequest));
96 }
97 return super.createResponseEntity(body, headers, status, request);
98 }
99
100 static ProblemDetail problem(ProblemType type, String detail, HttpServletRequest request) {
101     ProblemDetail problem = ProblemDetail.forStatusAndDetail(type.status(), detail);
102     problem.setType(type.uri());
103     problem.setTitle(type.title());
104     problem.setInstance(URI.create(request.getRequestURI()));
105     problem.setProperty("requestId", RequestIdFilter.currentId(request));
106     return problem;
107 }
108
109 private static String slugFor(HttpStatusCode status) {
110     return switch (status.value()) {
111         case 400 -> "invalid-request";
112         case 404 -> "not-found";
113         case 405 -> "method-not-allowed";
114         case 406 -> "not-acceptable";
115         case 415 -> "unsupported-media-type";
116         default -> "http-" + status.value();
117     };
118 }
119 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/Caller.java

37 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.math.BigDecimal;
4 import java.util.Collection;
5 import java.util.List;
6 import java.util.Optional;
7
8 import org.springframework.security.oauth2.jwt.Jwt;
9
10 /**
11  * Who is calling, as established by the access token: the customer whose data may be
12  * read, the client application acting for them, what it may do, and any trading limit
13  * the customer delegated to that client.
14  */
15 public record Caller(String customerId, String clientId, Collection<String> scopes,
16     Optional<BigDecimal> tradingLimit) {
17
18     public static Caller of(Jwt jwt) {
19         // A valid token that is not bound to a customer can see no accounts at all.
20         String customerId = Optional.ofNullable(jwt.getClaimAsString("customer_id")).orElse("none");
21         // "scope" may be a space-delimited string (RFC 6749) or a JSON array.
22         Collection<String> scopes = jwt.getClaim("scope") instanceof String text
23             ? List.of(text.split(" "))
24             : Optional.ofNullable(jwt.getClaimAsStringList("scope")).orElse(List.of());
25         Optional<BigDecimal> limit = Optional.ofNullable(jwt.getClaimAsString("trading_limit"))
26             .map(BigDecimal::new);
27         return new Caller(customerId, jwt.getSubject(), scopes, limit);
28     }
29
30     public boolean canTrade() {
31         return scopes.contains("orders:write");
32     }
33
34     public boolean canReadOrders() {
35         return scopes.contains("orders:read");
36     }
37 }

```

```

1 package com.example.brokerage.api.platform;
2
3 import java.nio.charset.StandardCharsets;
4 import java.time.Instant;
5 import java.util.Base64;
6 import java.util.Comparator;
7 import java.util.List;
8 import java.util.function.Function;
9
10 /**
11  * One page of a newest-first list, with an opaque cursor for the next page.
12  *
13  * <p>The cursor encodes the sort key (timestamp and ID) of the last item returned, so a
14  * page never skips or repeats items when new ones are inserted at the top: something
15  * offset-based paging ("page=3") cannot promise. Clients never parse cursors; they
16  * follow the "next" link that carries one.
17  */
18 public record CursorPage<T>(List<T> items, String nextCursor) {
19
20     public static final int DEFAULT_LIMIT = 25;
21     public static final int MAX_LIMIT = 100;
22
23     public boolean hasNext() {
24         return nextCursor != null;
25     }
26
27     public static <T> CursorPage<T> of(List<T> all, Function<T, Instant> time, Function<T, String> id,
28         String cursor, Integer requestedLimit) {
29         int limit = requestedLimit == null ? DEFAULT_LIMIT : requestedLimit;
30         if (limit < 1 || limit > MAX_LIMIT) {
31             throw new ApiException(ProblemType.INVALID_REQUEST,
32                 "limit must be between 1 and " + MAX_LIMIT);
33         }
34         Comparator<T> newestFirst = Comparator.comparing(time).thenComparing(id).reversed();
35         List<T> sorted = all.stream().sorted(newestFirst).toList();
36
37         int start = 0;
38         if (cursor != null) {
39             Key after = Key.decode(cursor);
40             while (start < sorted.size() && !isOlder(sorted.get(start), after, time, id)) {
41                 start++;
42             }
43         }
44         int end = Math.min(start + limit, sorted.size());
45         List<T> items = sorted.subList(start, end);
46         String next = end < sorted.size()
47             ? new Key(time.apply(items.getLast()), id.apply(items.getLast())).encode()
48             : null;
49         return new CursorPage<>(items, next);
50     }
51
52     private static <T> boolean isOlder(T item, Key after, Function<T, Instant> time, Function<T, String> id) {
53         int byTime = time.apply(item).compareTo(after.time());
54         return byTime < 0 || (byTime == 0 && id.apply(item).compareTo(after.id()) < 0);
55     }
56
57     record Key(Instant time, String id) {
58
59         String encode() {
60             String raw = time.toEpochMilli() + ":" + id;
61             return Base64.getUrlEncoder().withoutPadding()
62                 .encodeToString(raw.getBytes(StandardCharsets.UTF_8));
63         }
64
65         static Key decode(String cursor) {
66             try {
67                 String raw = new String(Base64.getUrlDecoder().decode(cursor), StandardCharsets.UTF_8);

```

```

68         int colon = raw.indexOf(':');
69         return new Key(Instant.ofEpochMilli(Long.parseLong(raw.substring(0, colon))),
70             raw.substring(colon + 1));
71     } catch (RuntimeException ex) {
72         throw new ApiException(ProblemType.INVALID_REQUEST, "The cursor is malformed or expired");
73     }
74 }
75 }
76 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/HypermediaConfig.java

68 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.math.BigDecimal;
4  import java.util.Set;
5
6  import com.example.brokerage.api.orders.Order;
7  import com.example.brokerage.api.orders.OrderRequest;
8
9  import org.springframework.context.annotation.Bean;
10 import org.springframework.context.annotation.Configuration;
11 import org.springframework.hateoas.LinkRelation;
12 import org.springframework.hateoas.UriTemplate;
13 import org.springframework.hateoas.config.EnableHypermediaSupport;
14 import org.springframework.hateoas.config.EnableHypermediaSupport.HypermediaType;
15 import org.springframework.hateoas.mediatype.hal.CurieProvider;
16 import org.springframework.hateoas.mediatype.hal.DefaultCurieProvider;
17 import org.springframework.hateoas.mediatype.hal.HalLinkRelation;
18 import org.springframework.hateoas.mediatype.hal.forms.HalFormsConfiguration;
19 import org.springframework.hateoas.mediatype.hal.forms.HalFormsOptions;
20
21 /**
22  * Hypermedia formats and link relations.
23  *
24  * <p>HAL (application/hal+json) is the default. Clients that ask for HAL-FORMS
25  * (application/prs.hal-forms+json) also receive "_templates": machine-readable
26  * descriptions of the state transitions available right now, such as placing,
27  * changing or cancelling an order.
28  */
29 @Configuration(proxyBeanMethods = false)
30 @EnableHypermediaSupport(type = { HypermediaType.HAL, HypermediaType.HAL_FORMS })
31 public class HypermediaConfig {
32
33     /** Documentation for every custom link relation lives at this URI template. */
34     public static final String RELS = "https://docs.brokerage.example/rels/{rel}";
35
36     /**
37      * Relations that are not registered with IANA (like "orders") are custom, and HAL
38      * requires custom relations to be URIs. A CURIE keeps them short on the wire:
39      * "bk:orders" expands to https://docs.brokerage.example/rels/orders.
40      */
41     @Bean
42     CurieProvider curieProvider() {
43         return new DefaultCurieProvider("bk", UriTemplate.of(RELS)) {
44             @Override
45             public HalLinkRelation getNamespacedRelFor(LinkRelation relation) {
46                 // Registered by RFC 8631, but missing from Spring HATEOAS's list of IANA relations.
47                 return RFC_8631.contains(relation.value()) ? HalLinkRelation.uncuried(relation.value())
48                     : super.getNamespacedRelFor(relation);
49             }
50         };
51     }
52
53     private static final Set<String> RFC_8631 = Set.of("service-desc", "service-doc", "service-meta");
54
55     /** Offer the allowed values of enum fields inside HAL-FORMS templates. */
56     @Bean
57     HalFormsConfiguration halFormsConfiguration() {

```

```

58         return new HalFormsConfiguration()
59             // Decimals travel as strings; describe them as text that must look like a number.
60             .withPattern(BigDecimal.class, "[0-9]+(\\.([0-9]+)?$)")
61             .withOptions(OrderRequest.class, "side",
62                 metadata -> HalFormsOptions.inline(Order.Side.values()))
63             .withOptions(OrderRequest.class, "type",
64                 metadata -> HalFormsOptions.inline(Order.Type.values()))
65             .withOptions(OrderRequest.class, "timeInForce",
66                 metadata -> HalFormsOptions.inline(Order.TimeInForce.values()));
67     }
68 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/JsonConfig.java

27 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.math.BigDecimal;
4
5  import com.fasterxml.jackson.annotation.JsonFormat;
6
7  import org.springframework.boot.jackson.autoconfigure.JsonMapperBuilderCustomizer;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.context.annotation.Configuration;
10
11 /**
12  * JSON conventions that apply to every representation.
13  */
14 @Configuration(proxyBeanMethods = false)
15 public class JsonConfig {
16
17     /**
18      * Money, prices and quantities are written as decimal strings ("187.50"), never as
19      * JSON numbers. Many JSON parsers read numbers into binary floating point, where
20      * 0.1 + 0.2 is not 0.3; a string survives every parser intact.
21      */
22     @Bean
23     JsonMapperBuilderCustomizer decimalsAsStrings() {
24         return builder -> builder.withConfigOverride(BigDecimal.class,
25             override -> override.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)));
26     }
27 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ProblemType.java

64 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.net.URI;
4
5  import org.springframework.http.HttpStatus;
6
7  /**
8   * The catalog of error types this API can return. Each becomes the "type" URI of an
9   * RFC 9457 problem document, so clients branch on a stable identifier, never on the
10  * human-readable title or detail.
11  */
12 public enum ProblemType {
13
14     INVALID_REQUEST("invalid-request", HttpStatus.BAD_REQUEST, "The request is not valid"),
15     UNAUTHORIZED("unauthorized", HttpStatus.UNAUTHORIZED, "Authentication is required"),
16     INSUFFICIENT_SCOPE("insufficient-scope", HttpStatus.FORBIDDEN, "The access token lacks a required scope"),
17     NOT_FOUND("not-found", HttpStatus.NOT_FOUND, "The resource does not exist"),
18     IDEMPOTENCY_KEY_MISSING("idempotency-key-missing", HttpStatus.BAD_REQUEST,
19         "An Idempotency-Key header is required"),
20     IDEMPOTENCY_KEY_REUSED("idempotency-key-reused", HttpStatus.UNPROCESSABLE_CONTENT,
21         "The Idempotency-Key was already used for a different request"),
22     PRECONDITION_REQUIRED("precondition-required", HttpStatus.PRECONDITION_REQUIRED,
23         "An If-Match header is required"),
24     PRECONDITION_FAILED("precondition-failed", HttpStatus.PRECONDITION_FAILED, "The resource has changed"),
25     ORDER_NOT_CANCELLABLE("order-not-cancellable", HttpStatus.CONFLICT,

```

```

26         "The order can no longer be cancelled"),
27     ORDER_NOT_REPLACEABLE("order-not-replaceable", HttpStatus.CONFLICT, "The order can no longer be changed"),
28     INSUFFICIENT_BUYING_POWER("insufficient-buying-power", HttpStatus.UNPROCESSABLE_CONTENT,
29         "The account does not have enough buying power"),
30     INSUFFICIENT_POSITION("insufficient-position", HttpStatus.UNPROCESSABLE_CONTENT,
31         "The account does not hold enough of the instrument"),
32     INSTRUMENT_NOT_TRADABLE("instrument-not-tradable", HttpStatus.UNPROCESSABLE_CONTENT,
33         "The instrument cannot be traded this way"),
34     TRADING_LIMIT_EXCEEDED("trading-limit-exceeded", HttpStatus.FORBIDDEN,
35         "The order exceeds the trading limit delegated to this client"),
36     UNSUPPORTED_API_VERSION("unsupported-api-version", HttpStatus.BAD_REQUEST,
37         "The requested API version is not supported"),
38     RATE_LIMITED("rate-limited", HttpStatus.TOO_MANY_REQUESTS, "Too many requests");
39
40     /** Problem type URIs resolve to human-readable documentation for each error. */
41     public static final String BASE = "https://docs.brokerage.example/problems/";
42
43     private final String slug;
44     private final HttpStatus status;
45     private final String title;
46
47     ProblemType(String slug, HttpStatus status, String title) {
48         this.slug = slug;
49         this.status = status;
50         this.title = title;
51     }
52
53     public URI uri() {
54         return URI.create(BASE + slug);
55     }
56
57     public HttpStatus status() {
58         return status;
59     }
60
61     public String title() {
62         return title;
63     }
64 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/RateLimitedException.java

17 lines

```

1 package com.example.brokerage.api.platform;
2
3 /** Raised when a client has used up its request quota for the current window. */
4 public class RateLimitedException extends ApiException {
5
6     private final long retryAfterSeconds;
7
8     public RateLimitedException(String policy, long retryAfterSeconds) {
9         super(ProblemType.RATE_LIMITED, "The " + policy
10             + " request quota for this client is used up; retry in " + retryAfterSeconds + " seconds");
11         this.retryAfterSeconds = retryAfterSeconds;
12     }
13
14     public long retryAfterSeconds() {
15         return retryAfterSeconds;
16     }
17 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/RateLimiter.java

75 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.time.Clock;
4 import java.util.Map;
5 import java.util.concurrent.ConcurrentHashMap;
6
7 import jakarta.servlet.http.HttpServletRequest;

```

```

8 import jakarta.servlet.http.HttpServletResponse;
9
10 import org.springframework.boot.context.properties.ConfigurationProperties;
11 import org.springframework.http.HttpMethod;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.security.core.context.SecurityContextHolder;
14 import org.springframework.stereotype.Component;
15 import org.springframework.web.servlet.HandlerInterceptor;
16
17 /**
18  * A fixed-window rate limiter per client, with two policies: "default" for every request
19  * and a stricter "orders" policy for requests that change orders. Every response
20  * advertises the policy and what is left of it, using the RateLimit-Policy and RateLimit
21  * fields from the IETF httpapi working group's draft, so well-behaved clients can slow
22  * down before they are refused.
23  */
24 @Component
25 public class RateLimiter implements HandlerInterceptor {
26
27     @ConfigurationProperties("brokerage.rate-limit")
28     public record Quotas(int requestsPerMinute, int orderChangesPerMinute) {
29         public Quotas {
30             requestsPerMinute = requestsPerMinute > 0 ? requestsPerMinute : 120;
31             orderChangesPerMinute = orderChangesPerMinute > 0 ? orderChangesPerMinute : 30;
32         }
33     }
34
35     private static final int WINDOW_SECONDS = 60;
36
37     private final Quotas quotas;
38     private final Clock clock;
39     private final Map<String, Window> windows = new ConcurrentHashMap<>();
40
41     public RateLimiter(Quotas quotas) {
42         this.quotas = quotas;
43         this.clock = Clock.systemUTC();
44     }
45
46     @Override
47     public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
48         Authentication caller = SecurityContextHolder.getContext().getAuthentication();
49         if (caller == null || !caller.isAuthenticated()) {
50             return true;
51         }
52         boolean changesOrders = !HttpMethod.GET.matches(request.getMethod())
53             && request.getRequestURI().contains("/orders");
54         String policy = changesOrders ? "orders" : "default";
55         int quota = changesOrders ? quotas.orderChangesPerMinute() : quotas.requestsPerMinute();
56
57         long now = clock.instant().getEpochSecond();
58         long windowStart = now - (now % WINDOW_SECONDS);
59         Window window = windows.compute(caller.getName() + "|" + policy,
60             (key, existing) -> existing == null || existing.start() != windowStart
61                 ? new Window(windowStart, 1) : new Window(windowStart, existing.used() + 1));
62
63         long resetIn = windowStart + WINDOW_SECONDS - now;
64         int remaining = Math.max(0, quota - window.used());
65         response.setHeader("RateLimit-Policy", "\"%s\";q=%d;w=%d".formatted(policy, quota, WINDOW_SECONDS));
66         response.setHeader("RateLimit", "\"%s\";r=%d;t=%d".formatted(policy, remaining, resetIn));
67         if (window.used() > quota) {
68             throw new RateLimitedException(policy, resetIn);
69         }
70         return true;
71     }
72
73     private record Window(long start, int used) {
74     }
75 }

```

```

1 package com.example.brokerage.api.platform;
2
3 import java.io.IOException;
4 import java.util.UUID;
5 import java.util.regex.Pattern;
6
7 import jakarta.servlet.FilterChain;
8 import jakarta.servlet.ServletException;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11
12 import org.slf4j.MDC;
13 import org.springframework.core.Ordered;
14 import org.springframework.core.annotation.Order;
15 import org.springframework.stereotype.Component;
16 import org.springframework.web.filter.OncePerRequestFilter;
17
18 /**
19  * Gives every request an ID: the caller's X-Request-Id if it is well formed, otherwise a
20  * new UUID. The ID is echoed in the response, written to every log line through the MDC,
21  * and included in every problem document, so a support ticket can quote one value.
22  */
23 @Component
24 @Order(Ordered.HIGHEST_PRECEDENCE)
25 public class RequestIdFilter extends OncePerRequestFilter {
26
27     public static final String HEADER = "X-Request-Id";
28     private static final String ATTRIBUTE = RequestIdFilter.class.getName();
29     private static final Pattern SAFE = Pattern.compile("[A-Za-z0-9._-]{1,64}");
30
31     @Override
32     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
33         FilterChain chain) throws ServletException, IOException {
34         String supplied = request.getHeader(HEADER);
35         String id = supplied != null && SAFE.matcher(supplied).matches()
36             ? supplied : UUID.randomUUID().toString();
37         request.setAttribute(ATTRIBUTE, id);
38         response.setHeader(HEADER, id);
39         MDC.put("requestId", id);
40         try {
41             chain.doFilter(request, response);
42         } finally {
43             MDC.remove("requestId");
44         }
45     }
46
47     public static String currentId(HttpServletRequest request) {
48         Object id = request.getAttribute(ATTRIBUTE);
49         return id != null ? id.toString() : null;
50     }
51 }

```

```

1 package com.example.brokerage.api.platform;
2
3 import java.util.List;
4
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7
8 import org.springframework.beans.factory.annotation.Qualifier;
9 import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.context.annotation.Bean;
11 import org.springframework.context.annotation.Configuration;
12 import org.springframework.http.HttpMethod;
13 import org.springframework.security.access.AccessDeniedException;

```

```

14 import org.springframework.security.config.Customizer;
15 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
16 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
17 import org.springframework.security.config.http.SessionCreationPolicy;
18 import org.springframework.security.core.AuthenticationException;
19 import org.springframework.security.oauth2.server.resource.web.BearerTokenAuthenticationEntryPoint;
20 import org.springframework.security.oauth2.server.resource.web.access.BearerTokenAccessDeniedHandler;
21 import org.springframework.security.web.AuthenticationEntryPoint;
22 import org.springframework.security.web.SecurityFilterChain;
23 import org.springframework.security.web.access.AccessDeniedHandler;
24 import org.springframework.web.servlet.HandlerExceptionResolver;
25
26 /**
27  * OAuth 2.1 resource server. Every operation requires one scope; the rules below are the
28  * single place where an operation's required scope is decided.
29  *
30  * <pre>
31  * market-data:read    instruments, quotes, option chains
32  * accounts:read       accounts, balances, positions, transactions
33  * orders:read         orders, order previews, the order event stream
34  * orders:write        placing, changing and cancelling orders
35  * </pre>
36  */
37 @Configuration(proxyBeanMethods = false)
38 public class SecurityConfig {
39
40     public static final String MARKET_DATA_READ = "SCOPE_market-data:read";
41     public static final String ACCOUNTS_READ = "SCOPE_accounts:read";
42     public static final String ORDERS_READ = "SCOPE_orders:read";
43     public static final String ORDERS_WRITE = "SCOPE_orders:write";
44
45     @Bean
46     SecurityFilterChain api(HttpSecurity http, ProblemResponses problems,
47         @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer)
48         throws Exception {
49         return http
50             .authorizeHttpRequests(auth -> auth
51                 .requestMatchers(HttpMethod.GET, "/", "/openapi.yaml", "/actuator/health/**",
52                     "/.well-known/oauth-protected-resource").permitAll()
53                 .requestMatchers(HttpMethod.GET, "/instruments/**").hasAuthority(MARKET_DATA_READ)
54                 .requestMatchers(HttpMethod.GET, "/accounts/*/orders/**", "/accounts/*/order-events")
55                     .hasAuthority(ORDERS_READ)
56                 .requestMatchers(HttpMethod.POST, "/accounts/*/order-previews")
57                     .hasAuthority(ORDERS_READ)
58                 .requestMatchers("/accounts/*/orders/**").hasAuthority(ORDERS_WRITE)
59                 .requestMatchers(HttpMethod.GET, "/accounts/**").hasAuthority(ACCOUNTS_READ)
60                 .anyRequest().denyAll())
61             .oauth2ResourceServer(server -> server
62                 .jwt(Customizer.withDefaults())
63                 // RFC 9728: publish which authorization server and scopes this API uses.
64                 .protectedResourceMetadata(metadata -> metadata.protectedResourceMetadataCustomizer(
65                     resource -> resource.resourceName("Brokerage API")
66                         .authorizationServer(issuer)
67                         .tlsClientCertificateBoundAccessTokens(false)
68                         .scopes(scopes -> scopes.addAll(List.of("market-data:read",
69                             "accounts:read", "orders:read", "orders:write")))))
70                 .authenticationEntryPoint(problems)
71                 .accessDeniedHandler(problems))
72             // Bearer tokens are sent explicitly, never automatically like cookies, so
73             // there is no cross-site request forgery to defend against, and no session.
74             .csrf(AbstractHttpConfigurer::disable)
75             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
76             .build();
77     }
78
79     /**
80     * 401 and 403 responses keep the standard WWW-Authenticate header (RFC 6750) and get the
81     * same problem-document body as every other error, by handing the exception to the MVC
82     * exception handling in {@link ApiExceptionHandler}.

```

```

83     */
84     @Bean
85     ProblemResponses problemResponses(
86         @Qualifier("handlerExceptionResolver") HandlerExceptionResolver resolver) {
87         return new ProblemResponses(resolver);
88     }
89
90     public static final class ProblemResponses implements AuthenticationEntryPoint, AccessDeniedHandler {
91
92         private final HandlerExceptionResolver resolver;
93         private final BearerTokenAuthenticationEntryPoint challenge =
94             new BearerTokenAuthenticationEntryPoint();
95         private final BearerTokenAccessDeniedHandler denied = new BearerTokenAccessDeniedHandler();
96
97         ProblemResponses(HandlerExceptionResolver resolver) {
98             this.resolver = resolver;
99         }
100
101         @Override
102         public void commence(HttpServletRequest request, HttpServletResponse response,
103             AuthenticationException ex) {
104             challenge.commence(request, response, ex);
105             resolver.resolveException(request, response, null, ex);
106         }
107
108         @Override
109         public void handle(HttpServletRequest request, HttpServletResponse response,
110             AccessDeniedException ex) {
111             denied.handle(request, response, ex);
112             resolver.resolveException(request, response, null, ex);
113         }
114     }
115 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/WebConfig.java

49 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.net.URI;
4  import java.time.ZonedDateTime;
5
6  import org.springframework.context.annotation.Configuration;
7  import org.springframework.web.accept.StandardApiVersionDeprecationHandler;
8  import org.springframework.web.servlet.config.annotation.ApiVersionConfigurer;
9  import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
10 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
11
12 /**
13  * API versioning and request interceptors.
14  *
15  * <p>Clients choose a version with the API-Version request header. Version 1 is the
16  * default so that existing clients keep working, but it is deprecated: every version 1
17  * response carries Deprecation (RFC 9745) and Sunset (RFC 8594) headers, plus a link to
18  * the migration guide. Version 2 differs only in the shape of a quote.
19  */
20 @Configuration(proxyBeanMethods = false)
21 public class WebConfig implements WebMvcConfigurer {
22
23     public static final String VERSION_HEADER = "API-Version";
24
25     private final RateLimiter rateLimiter;
26
27     public WebConfig(RateLimiter rateLimiter) {
28         this.rateLimiter = rateLimiter;
29     }
30
31     @Override
32     public void configureApiVersioning(ApiVersionConfigurer versions) {
33         StandardApiVersionDeprecationHandler deprecations = new StandardApiVersionDeprecationHandler();

```

```

34     deprecations.configureVersion("1")
35         .setDeprecationDate(ZonedDateTime.parse("2026-09-01T00:00:00Z"))
36         .setSunsetDate(ZonedDateTime.parse("2027-03-01T00:00:00Z"))
37         .setSunsetLink(URI.create("https://docs.brokerage.example/migrations/v2"));
38
39     versions.useRequestHeader(VERSION_HEADER)
40         .addSupportedVersions("1", "2")
41         .setDefaultVersion("1")
42         .setDeprecationHandler(deprecations);
43 }
44
45 @Override
46 public void addInterceptors(InterceptorRegistry registry) {
47     registry.addInterceptor(rateLimiter).addPathPatterns("/accounts/**", "/instruments/**");
48 }
49 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/Instrument.java

34 lines

```

1  package com.example.brokerage.api.market;
2
3  import java.math.BigDecimal;
4  import java.time.LocalDate;
5
6  import com.fasterxml.jackson.annotation.JsonIgnore;
7
8  import org.springframework.hateoas.server.core.Relation;
9
10 /**
11  * Something that can be traded: a stock, an ETF, or an option contract. The ID is
12  * opaque: clients store it and send it back, but never take it apart.
13  *
14  * @param multiplier shares controlled by one unit: 1 for stocks, 100 for a US equity option
15  * @param option present only for option contracts
16  */
17 @Relation(collectionRelation = "instruments", itemRelation = "instrument")
18 public record Instrument(String id, String symbol, String name, Type type, String exchange,
19     String currency, boolean tradable, boolean fractionable, BigDecimal multiplier, OptionTerms option) {
20
21     public enum Type { EQUITY, ETF, OPTION }
22
23     public enum Right { CALL, PUT }
24
25     /** The contract terms of an option. */
26     public record OptionTerms(String underlyingId, String underlyingSymbol, Right right,
27         BigDecimal strike, LocalDate expiration, String style) {
28     }
29
30     @JsonIgnore // otherwise Jackson would also write a boolean "option" property
31     public boolean isOption() {
32         return type == Type.OPTION;
33     }
34 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/InstrumentController.java

133 lines

```

1  package com.example.brokerage.api.market;
2
3  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6  import java.time.LocalDate;
7  import java.util.List;
8  import java.util.concurrent.TimeUnit;
9
10 import com.example.brokerage.api.market.Instrument.Type;
11 import com.example.brokerage.api.market.Quotes.Quote;
12 import com.example.brokerage.api.market.Quotes.QuoteV1;
13

```

```

14 import org.springframework.format.annotation.DateTimeFormat;
15 import org.springframework.hateoas.CollectionModel;
16 import org.springframework.hateoas.EntityModel;
17 import org.springframework.http.CacheControl;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.web.bind.annotation.GetMapping;
20 import org.springframework.web.bind.annotation.PathVariable;
21 import org.springframework.web.bind.annotation.RequestParam;
22 import org.springframework.web.bind.annotation.RestController;
23 import org.springframework.web.context.request.WebRequest;
24
25 /**
26  * Instruments, quotes and option chains. Scope: market-data:read.
27  */
28 @RestController
29 public class InstrumentController {
30
31     private final MarketData market;
32
33     public InstrumentController(MarketData market) {
34         this.market = market;
35     }
36
37     @GetMapping("/instruments")
38     public CollectionModel<EntityModel<Instrument>> search(
39         @RequestParam(required = false) String symbol, @RequestParam(required = false) Type type) {
40         List<EntityModel<Instrument>> found = market.search(symbol, type).stream()
41             .map(this::toModel)
42             .toList();
43         return CollectionModel.of(found,
44             linkTo(methodOn(InstrumentController.class).search(symbol, type)).withSelfRel().expand());
45     }
46
47     @GetMapping("/instruments/{instrumentId}")
48     public EntityModel<Instrument> instrument(@PathVariable String instrumentId) {
49         return toModel(market.instrument(instrumentId));
50     }
51
52     /** Version 1 of a quote (deprecated; see WebConfig). */
53     @GetMapping(path = "/instruments/{instrumentId}/quote", version = "1")
54     public ResponseEntity<EntityModel<QuoteV1>> quoteV1(@PathVariable String instrumentId,
55         WebRequest request) {
56         Instrument instrument = market.instrument(instrumentId);
57         MarketData.Tick tick = market.tick(instrumentId);
58         return quoteResponse(instrument, tick, QuoteV1.of(instrument, tick), request);
59     }
60
61     @GetMapping(path = "/instruments/{instrumentId}/quote", version = "2")
62     public ResponseEntity<EntityModel<Quote>> quote(@PathVariable String instrumentId,
63         WebRequest request) {
64         Instrument instrument = market.instrument(instrumentId);
65         MarketData.Tick tick = market.tick(instrumentId);
66         return quoteResponse(instrument, tick, Quote.of(instrument, tick), request);
67     }
68
69     /**
70      * Quotes change every second, so they may be cached only briefly and only privately
71      * (market data is licensed per user). The ETag lets a client poll cheaply: an unchanged
72      * quote costs a 304 with no body.
73      */
74     private <T> ResponseEntity<EntityModel<T>> quoteResponse(Instrument instrument, MarketData.Tick tick,
75         T quote, WebRequest request) {
76         String etag = "\"q\" + tick.sequence() + \"\"";
77         if (request.checkNotModified(etag)) {
78             return null;
79         }
80         EntityModel<T> model = EntityModel.of(quote,
81             linkTo(methodOn(InstrumentController.class).quote(instrument.id(), null)).withSelfRel(),
82             linkTo(methodOn(InstrumentController.class).instrument(instrument.id()))

```

```

83         .withRel("instrument"));
84     return ResponseEntity.ok()
85         .cacheControl(CacheControl.maxAge(1, TimeUnit.SECONDS).cachePrivate())
86         .eTag(etag)
87         .body(model);
88 }
89
90 /**
91  * The option chain for one expiration. Every available expiration is offered as a
92  * link with the same relation, told apart by its "name": the client picks one instead
93  * of guessing dates.
94  */
95 @GetMapping("/instruments/{instrumentId}/option-chain")
96 public CollectionModel<EntityModel<Instrument>> optionChain(@PathVariable String instrumentId,
97     @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
98     LocalDate expiration) {
99     Instrument underlying = market.instrument(instrumentId);
100    List<LocalDate> expirations = market.expirations(underlying.id());
101    LocalDate chosen = expiration != null ? expiration
102        : expirations.isEmpty() ? null : expirations.getFirst();
103    List<EntityModel<Instrument>> contracts = chosen == null ? List.of()
104        : market.optionChain(underlying.id(), chosen).stream().map(this::toModel).toList();
105
106    CollectionModel<EntityModel<Instrument>> chain = CollectionModel.of(contracts,
107        linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), chosen))
108            .withSelfRel(),
109        linkTo(methodOn(InstrumentController.class).instrument(underlying.id()))
110            .withRel("underlying"));
111    for (LocalDate date : expirations) {
112        chain.add(linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), date))
113            .withRel("expiration").withName(date.toString()));
114    }
115    return chain;
116 }
117
118 private EntityModel<Instrument> toModel(Instrument instrument) {
119     EntityModel<Instrument> model = EntityModel.of(instrument,
120         linkTo(methodOn(InstrumentController.class).instrument(instrument.id())).withSelfRel(),
121         linkTo(methodOn(InstrumentController.class).quote(instrument.id(), null))
122             .withRel("quote"));
123     if (instrument.isOption()) {
124         String underlyingId = instrument.option().underlyingId();
125         model.add(linkTo(methodOn(InstrumentController.class).instrument(underlyingId))
126             .withRel("underlying"));
127     } else if (!market.expirations(instrument.id()).isEmpty()) {
128         model.add(linkTo(methodOn(InstrumentController.class).optionChain(instrument.id(), null))
129             .withRel("option-chain"));
130     }
131     return model;
132 }
133 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/MarketData.java

169 lines

```

1 package com.example.brokerage.api.market;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Clock;
6 import java.time.DayOfWeek;
7 import java.time.Instant;
8 import java.time.LocalDate;
9 import java.time.format.DateTimeFormatter;
10 import java.time.temporal.ChronoUnit;
11 import java.time.temporal.TemporalAdjusters;
12 import java.util.LinkedHashMap;
13 import java.util.List;
14 import java.util.Locale;
15 import java.util.Map;

```

```

16 import java.util.Random;
17 import java.util.concurrent.ConcurrentHashMap;
18
19 import com.example.brokerage.api.market.Instrument.OptionTerms;
20 import com.example.brokerage.api.market.Instrument.Right;
21 import com.example.brokerage.api.market.Instrument.Type;
22 import com.example.brokerage.api.platform.ApiException;
23
24 import org.springframework.stereotype.Component;
25
26 /**
27  * The instrument catalog and a simulated market. Prices are fictional: each stock takes a
28  * small random step every tick, and option prices follow their underlying. Option
29  * expirations are the third Friday of the next two months, so the demo never serves
30  * contracts that have already expired.
31  */
32 @Component
33 public class MarketData {
34
35     /** The market state of one instrument at one moment. */
36     public record Tick(BigDecimal last, BigDecimal bid, BigDecimal ask, BigDecimal bidSize,
37         BigDecimal askSize, BigDecimal previousClose, long volume, long sequence, Instant asOf) {
38     }
39
40     private static final DateTimeFormatter OPTION_DATE = DateTimeFormatter.ofPattern("yyMMdd", Locale.US);
41     private static final BigDecimal CENT = new BigDecimal("0.01");
42
43     private final Map<String, Instrument> instruments = new LinkedHashMap<>();
44     private final Map<String, Tick> ticks = new ConcurrentHashMap<>();
45     private final Random random = new Random(7);
46     private final Clock clock;
47
48     public MarketData(Clock clock) {
49         this.clock = clock;
50         stock("EQ-AAPL", "AAPL", "Apple Inc.", Type.EQUITY, "XNAS", "232.40", "230.10", true);
51         stock("EQ-MSFT", "MSFT", "Microsoft Corporation", Type.EQUITY, "XNAS", "512.75", "509.90", true);
52         stock("EQ-NVDA", "NVDA", "NVIDIA Corporation", Type.EQUITY, "XNAS", "186.20", "188.45", true);
53         stock("EQ-AMZN", "AMZN", "Amazon.com, Inc.", Type.EQUITY, "XNAS", "228.30", "226.00", true);
54         stock("ETF-SPY", "SPY", "SPDR S&P 500 ETF Trust", Type.ETF, "ARCX", "668.10", "664.80", true);
55         stock("EQ-HALT", "HALT", "Halted Holdings (suspended)", Type.EQUITY, "XNYS", "12.00", "12.00", false);
56
57         LocalDate today = LocalDate.now(clock);
58         for (int monthsAhead = 1; monthsAhead <= 2; monthsAhead++) {
59             LocalDate expiration = today.plusMonths(monthsAhead).withDayOfMonth(1)
60                 .with(TemporalAdjusters.dayOfWeekInMonth(3, DayOfWeek.FRIDAY));
61             for (int strike : new int[] { 220, 230, 240 }) {
62                 option(instruments.get("EQ-AAPL"), expiration, Right.CALL, strike);
63                 option(instruments.get("EQ-AAPL"), expiration, Right.PUT, strike);
64             }
65         }
66     }
67
68     // ----- catalog
69
70     public Instrument instrument(String id) {
71         Instrument instrument = instruments.get(id);
72         if (instrument == null) {
73             throw ApiException.notFound("Instrument", id);
74         }
75         return instrument;
76     }
77
78     public List<Instrument> search(String symbol, Type type) {
79         return instruments.values().stream()
80             .filter(i -> symbol == null || i.symbol().equalsIgnoreCase(symbol))
81             .filter(i -> type == null || i.type() == type)
82             .toList();
83     }
84

```

```

85     public List<LocalDate> expirations(String underlyingId) {
86         return options(underlyingId).stream()
87             .map(i -> i.option().expiration())
88             .distinct().sorted().toList();
89     }
90
91     public List<Instrument> optionChain(String underlyingId, LocalDate expiration) {
92         return options(underlyingId).stream()
93             .filter(i -> i.option().expiration().equals(expiration))
94             .toList();
95     }
96
97     private List<Instrument> options(String underlyingId) {
98         return instruments.values().stream()
99             .filter(i -> i.isOption() && i.option().underlyingId().equals(underlyingId))
100             .toList();
101     }
102
103     // ----- prices
104
105     public Tick tick(String instrumentId) {
106         return ticks.get(instrument(instrumentId).id());
107     }
108
109     /** One step of the simulated market. Called on a schedule by the order matcher. */
110     public synchronized void move() {
111         for (Instrument instrument : instruments.values()) {
112             if (instrument.isOption() || !instrument.tradable()) {
113                 continue;
114             }
115             Tick tick = ticks.get(instrument.id());
116             double step = (random.nextGaussian() * 0.0008) * tick.last().doubleValue();
117             BigDecimal last = tick.last().add(BigDecimal.valueOf(step))
118                 .setScale(2, RoundingMode.HALF_EVEN);
119             long volume = tick.volume() + random.nextInt(400);
120             ticks.put(instrument.id(), stockTick(last, tick.previousClose(), volume, tick.sequence() + 1));
121         }
122         instruments.values().stream().filter(Instrument::isOption).forEach(this::priceOption);
123     }
124
125     // ----- seeding
126
127     private void stock(String id, String symbol, String name, Type type, String exchange,
128         String last, String previousClose, boolean tradable) {
129         instruments.put(id, new Instrument(id, symbol, name, type, exchange, "USD", tradable, tradable,
130             BigDecimal.ONE, null));
131         ticks.put(id, stockTick(new BigDecimal(last), new BigDecimal(previousClose), 1_250_000, 1));
132     }
133
134     private void option(Instrument underlying, LocalDate expiration, Right right, int strike) {
135         String id = "OPT-%s-%s-%s-%d".formatted(underlying.symbol(), expiration.format(OPTION_DATE),
136             right == Right.CALL ? "C" : "P", strike);
137         String name = "%s %s %s %d %s".formatted(underlying.symbol(), expiration, strike, right);
138         OptionTerms terms = new OptionTerms(underlying.id(), underlying.symbol(), right,
139             BigDecimal.valueOf(strike), expiration, "AMERICAN");
140         instruments.put(id, new Instrument(id, underlying.symbol(), name, Type.OPTION, "OPRA", "USD",
141             true, false, BigDecimal.valueOf(100), terms));
142         priceOption(instruments.get(id));
143     }
144
145     private Tick stockTick(BigDecimal last, BigDecimal previousClose, long volume, long sequence) {
146         BigDecimal halfSpread = last.multiply(new BigDecimal("0.0001")).max(CENT)
147             .setScale(2, RoundingMode.HALF_EVEN);
148         return new Tick(last, last.subtract(halfSpread), last.add(halfSpread), BigDecimal.valueOf(500),
149             BigDecimal.valueOf(500), previousClose, volume, sequence, clock.instant());
150     }
151
152     /** Intrinsic value plus a simple time value: enough to look plausible, not a pricing model. */
153     private void priceOption(Instrument option) {

```

```

154     OptionTerms terms = option.option();
155     BigDecimal underlying = ticks.get(terms.underlyingId()).last();
156     BigDecimal intrinsic = (terms.right() == Right.CALL
157         ? underlying.subtract(terms.strike())
158         : terms.strike().subtract(underlying)).max(BigDecimal.ZERO);
159     long days = Math.max(1, ChronoUnit.DAYS.between(LocalDate.now(clock), terms.expiration()));
160     BigDecimal timeValue = underlying.multiply(BigDecimal.valueOf(0.012 * Math.sqrt(days / 30.0)));
161     BigDecimal mid = intrinsic.add(timeValue).setScale(2, RoundingMode.HALF_EVEN);
162     Tick previous = ticks.get(option.id());
163     long sequence = previous == null ? 1 : previous.sequence() + 1;
164     BigDecimal previousClose = previous == null ? mid : previous.previousClose();
165     BigDecimal nickel = new BigDecimal("0.05");
166     ticks.put(option.id(), new Tick(mid, mid.subtract(nickel), mid.add(nickel), BigDecimal.valueOf(50),
167         BigDecimal.valueOf(50), previousClose, 3_400, sequence, clock.instant()));
168 }
169 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/Quotes.java

47 lines

```

1 package com.example.brokerage.api.market;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Instant;
6
7 import org.springframework.hateoas.server.core.Relation;
8
9 /**
10  * The two representations of a quote. Version 2 regroups price and size into bid and ask
11  * objects and adds the day's change: a breaking change of shape, so it needs a new API
12  * version rather than a new optional field.
13  */
14 public final class Quotes {
15
16     private Quotes() {
17     }
18
19     /** API version 1 (deprecated): flat fields. */
20     @Relation(itemRelation = "quote")
21     public record QuoteV1(String instrumentId, String symbol, BigDecimal last, BigDecimal bid,
22         BigDecimal ask, BigDecimal bidSize, BigDecimal askSize, long volume, Instant asOf) {
23
24         public static QuoteV1 of(Instrument instrument, MarketData.Tick tick) {
25             return new QuoteV1(instrument.id(), instrument.symbol(), tick.last(), tick.bid(), tick.ask(),
26                 tick.bidSize(), tick.askSize(), tick.volume(), tick.asOf());
27         }
28     }
29
30     /** API version 2: bid and ask as price levels, plus the change since the previous close. */
31     @Relation(itemRelation = "quote")
32     public record Quote(String instrumentId, String symbol, BigDecimal last, PriceLevel bid, PriceLevel ask,
33         BigDecimal change, BigDecimal changePercent, long volume, Instant asOf) {
34
35         public record PriceLevel(BigDecimal price, BigDecimal size) {
36         }
37
38         public static Quote of(Instrument instrument, MarketData.Tick tick) {
39             BigDecimal change = tick.last().subtract(tick.previousClose());
40             BigDecimal percent = change.multiply(BigDecimal.valueOf(100))
41                 .divide(tick.previousClose(), 2, RoundingMode.HALF_EVEN);
42             return new Quote(instrument.id(), instrument.symbol(), tick.last(),
43                 new PriceLevel(tick.bid(), tick.bidSize()), new PriceLevel(tick.ask(), tick.askSize()),
44                 change, percent, tick.volume(), tick.asOf());
45         }
46     }
47 }

```

```

1 package com.example.brokerage.api.accounts;
2
3 import java.time.LocalDate;
4
5 import com.fasterxml.jackson.annotation.JsonIgnore;
6
7 import org.springframework.hateoas.server.core.Relation;
8
9 /**
10  * A brokerage account. A customer can own several (an individual margin account and a
11  * retirement account, say); every other resource hangs off one of them.
12  *
13  * @param number the account number, masked: only the last four digits are ever sent
14  */
15 @Relation(collectionRelation = "accounts", itemRelation = "account")
16 public record Account(String id, @JsonIgnore String customerId, String number, String nickname, Type type,
17     Registration registration, String currency, LocalDate openedOn) {
18
19     /** How purchases are funded: settled cash only, or cash plus a margin loan. */
20     public enum Type { CASH, MARGIN }
21
22     /** The legal form of the account. */
23     public enum Registration { INDIVIDUAL, JOINT, ROTH_IRA, TRADITIONAL_IRA }
24 }

```

```

1 package com.example.brokerage.api.accounts;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.afford;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
5 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
6
7 import java.time.LocalDate;
8 import java.time.ZoneOffset;
9 import java.util.List;
10
11 import com.example.brokerage.api.accounts.Holdings.Balances;
12 import com.example.brokerage.api.accounts.Holdings.Position;
13 import com.example.brokerage.api.market.InstrumentController;
14 import com.example.brokerage.api.orders.OrderController;
15 import com.example.brokerage.api.orders.OrderEventController;
16 import com.example.brokerage.api.platform.Caller;
17 import com.example.brokerage.api.platform.CursorPage;
18
19 import org.springframework.format.annotation.DateTimeFormat;
20 import org.springframework.hateoas.CollectionModel;
21 import org.springframework.hateoas.EntityModel;
22 import org.springframework.hateoas.Link;
23 import org.springframework.security.core.annotation.AuthenticationPrincipal;
24 import org.springframework.security.oauth2.jwt.Jwt;
25 import org.springframework.web.bind.annotation.GetMapping;
26 import org.springframework.web.bind.annotation.PathVariable;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29
30 /**
31  * Accounts and everything read-only beneath them. Scope: accounts:read.
32  */
33 @RestController
34 public class AccountController {
35
36     private final Ledger ledger;
37     private final Portfolio portfolio;
38
39     public AccountController(Ledger ledger, Portfolio portfolio) {
40         this.ledger = ledger;

```

```

41     this.portfolio = portfolio;
42 }
43
44 @GetMapping("/accounts")
45 public CollectionModel<EntityModel<Account>> accounts(@AuthenticationPrincipal Jwt jwt) {
46     Caller caller = Caller.of(jwt);
47     List<EntityModel<Account>> accounts = ledger.accounts(caller.customerId()).stream()
48         .map(account -> toModel(account, caller))
49         .toList();
50     return CollectionModel.of(accounts,
51         linkTo(methodOn(AccountController.class).accounts(null)).withSelfRel());
52 }
53
54 @GetMapping("/accounts/{accountId}")
55 public EntityModel<Account> account(@PathVariable String accountId,
56     @AuthenticationPrincipal Jwt jwt) {
57     Caller caller = Caller.of(jwt);
58     return toModel(ledger.account(caller.customerId(), accountId), caller);
59 }
60
61 @GetMapping("/accounts/{accountId}/balances")
62 public EntityModel<Balances> balances(@PathVariable String accountId,
63     @AuthenticationPrincipal Jwt jwt) {
64     Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
65     return EntityModel.of(portfolio.balances(account),
66         linkTo(methodOn(AccountController.class).balances(accountId, null)).withSelfRel(),
67         linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"),
68         linkTo(methodOn(AccountController.class).positions(accountId, null)).withRel("positions"));
69 }
70
71 @GetMapping("/accounts/{accountId}/positions")
72 public CollectionModel<EntityModel<Position>> positions(@PathVariable String accountId,
73     @AuthenticationPrincipal Jwt jwt) {
74     Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
75     List<EntityModel<Position>> positions = portfolio.positions(account).stream()
76         .map(position -> toModel(accountId, position))
77         .toList();
78     return CollectionModel.of(positions,
79         linkTo(methodOn(AccountController.class).positions(accountId, null)).withSelfRel(),
80         linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
81 }
82
83 @GetMapping("/accounts/{accountId}/positions/{instrumentId}")
84 public EntityModel<Position> position(@PathVariable String accountId,
85     @PathVariable String instrumentId, @AuthenticationPrincipal Jwt jwt) {
86     Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
87     return toModel(accountId, portfolio.position(account, instrumentId));
88 }
89
90 /**
91  * Transaction history, newest first, filterable by type and by an inclusive date range
92  * (UTC), paged with an opaque cursor.
93  */
94 @GetMapping("/accounts/{accountId}/transactions")
95 public CollectionModel<EntityModel<Transaction>> transactions(@PathVariable String accountId,
96     @RequestParam(required = false) Transaction.Type type,
97     @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate from,
98     @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate to,
99     @RequestParam(required = false) String cursor, @RequestParam(required = false) Integer limit,
100     @AuthenticationPrincipal Jwt jwt) {
101     Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
102     List<Transaction> matching = ledger.transactions(account.id()).stream()
103         .filter(t -> type == null || t.type() == type)
104         .filter(t -> from == null || !utcDate(t).isBefore(from))
105         .filter(t -> to == null || !utcDate(t).isAfter(to))
106         .toList();
107     CursorPage<Transaction> page = CursorPage.of(matching, Transaction::occurredAt, Transaction::id,
108         cursor, limit);
109 }

```

```

110     CollectionModel<EntityModel<Transaction>> model = CollectionModel.of(
111         page.items().stream().map(EntityModel::of).toList(),
112         linkTo(methodOn(AccountController.class)
113             .transactions(accountId, type, from, to, cursor, limit, null))
114             .withSelfRel().expand(),
115         linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
116     if (page.hasNext()) {
117         // expand(): "next" is a concrete URL, not a template with the unused filters left open
118         model.add(linkTo(methodOn(AccountController.class)
119             .transactions(accountId, type, from, to, page.nextCursor(), limit, null))
120             .withRel("next").expand());
121     }
122     return model;
123 }
124
125 private static LocalDate utcDate(Transaction transaction) {
126     return transaction.occurredAt().atZone(ZoneOffset.UTC).toLocalDate();
127 }
128
129 // ----- representations
130
131 /**
132  * An account's links depend on what the caller may do: a token without orders:read is
133  * not shown the orders at all, and only a token with orders:write is offered the
134  * "placeOrder" affordance. Clients discover their capabilities instead of hard-coding them.
135  */
136 EntityModel<Account> toModel(Account account, Caller caller) {
137     String id = account.id();
138     EntityModel<Account> model = EntityModel.of(account,
139         linkTo(methodOn(AccountController.class).account(id, null)).withSelfRel(),
140         linkTo(methodOn(AccountController.class).balances(id, null)).withRel("balances"),
141         linkTo(methodOn(AccountController.class).positions(id, null)).withRel("positions"),
142         linkTo(methodOn(AccountController.class).transactions(id, null, null, null, null, null))
143             .withRel("transactions"));
144     if (caller.canReadOrders()) {
145         Link orders = linkTo(methodOn(OrderController.class).orders(id, null, null, null, null, null))
146             .withRel("orders");
147         if (caller.canTrade()) {
148             orders = orders.andAffordance(
149                 afford(methodOn(OrderController.class).placeOrder(id, null, null, null)));
150         }
151         model.add(orders,
152             linkTo(methodOn(OrderController.class).previewOrder(id, null, null))
153                 .withRel("order-previews"),
154             linkTo(methodOn(OrderEventController.class).events(id, null, null))
155                 .withRel("order-events"));
156     }
157     return model;
158 }
159
160 private EntityModel<Position> toModel(String accountId, Position position) {
161     return EntityModel.of(position,
162         linkTo(methodOn(AccountController.class).position(accountId, position.instrumentId(), null))
163             .withSelfRel(),
164         linkTo(methodOn(InstrumentController.class).instrument(position.instrumentId()))
165             .withRel("instrument"),
166         linkTo(methodOn(InstrumentController.class).quote(position.instrumentId(), null))
167             .withRel("quote"));
168 }
169 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Holdings.java

47 lines

```

1 package com.example.brokerage.api.accounts;
2
3 import java.math.BigDecimal;
4 import java.time.Instant;
5
6 import com.example.brokerage.api.market.Instrument;

```

```

7
8 import org.springframework.hateoas.server.core.Relation;
9
10 /**
11  * The read models derived from the ledger and the market: positions and balances.
12  */
13 public final class Holdings {
14
15     private Holdings() {
16     }
17
18     /**
19      * A holding valued at the current market price.
20      *
21      * @param averageCost per share, or per contract share for options
22      */
23     @Relation(collectionRelation = "positions", itemRelation = "position")
24     public record Position(String instrumentId, String symbol, Instrument.Type instrumentType,
25         BigDecimal quantity, BigDecimal averageCost, BigDecimal costBasis, BigDecimal lastPrice,
26         BigDecimal marketValue, BigDecimal unrealizedPnl, BigDecimal unrealizedPnlPercent) {
27     }
28
29     /**
30      * What the account is worth and what it can buy.
31      *
32      * @param reservedForOrders cash held back for open buy orders that have not filled yet
33      */
34     public record Balances(String accountId, String currency, BigDecimal cash, BigDecimal reservedForOrders,
35         BigDecimal buyingPower, BigDecimal positionsMarketValue, BigDecimal totalEquity,
36         BigDecimal unrealizedPnl, Instant asOf) {
37     }
38
39     /**
40      * Buying power: unreserved cash in a cash account, twice that in a margin account
41      * (the 50% initial margin of Regulation T). A deliberate simplification.
42      */
43     public static BigDecimal buyingPower(Account.Type type, BigDecimal cash, BigDecimal reserved) {
44         BigDecimal free = cash.subtract(reserved).max(BigDecimal.ZERO);
45         return type == Account.Type.MARGIN ? free.multiply(BigDecimal.TWO) : free;
46     }
47 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Ledger.java

214 lines

```

1 package com.example.brokerage.api.accounts;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Clock;
6 import java.time.DayOfWeek;
7 import java.time.Duration;
8 import java.time.Instant;
9 import java.time.LocalDate;
10 import java.time.ZoneOffset;
11 import java.util.ArrayList;
12 import java.util.Comparator;
13 import java.util.LinkedHashMap;
14 import java.util.List;
15 import java.util.Map;
16 import java.util.concurrent.atomic.AtomicInteger;
17
18 import com.example.brokerage.api.accounts.Account.Registration;
19 import com.example.brokerage.api.accounts.Transaction.Type;
20 import com.example.brokerage.api.market.Instrument;
21 import com.example.brokerage.api.market.MarketData;
22 import com.example.brokerage.api.platform.ApiException;
23
24 import org.springframework.stereotype.Component;
25

```

```

26 /**
27  * The books of record: accounts, cash, holdings and transaction history, held in memory
28  * and seeded with two demo customers. Every mutation goes through {@link #recordTrade},
29  * which keeps cash, holdings and history consistent with one another.
30  */
31 @Component
32 public class Ledger {
33
34     /** One holding: how much, and at what average cost per unit. */
35     public record Holding(String instrumentId, BigDecimal quantity, BigDecimal averageCost) {
36     }
37
38     private static final class Book {
39         BigDecimal cash;
40         final Map<String, Holding> holdings = new LinkedHashMap<>();
41         final List<Transaction> transactions = new ArrayList<>();
42
43         Book(BigDecimal cash) {
44             this.cash = cash;
45         }
46     }
47
48     private final Map<String, Account> accounts = new LinkedHashMap<>();
49     private final Map<String, Book> books = new LinkedHashMap<>();
50     private final AtomicInteger transactionIds = new AtomicInteger(1000);
51     private final MarketData market;
52     private final Clock clock;
53
54     public Ledger(MarketData market, Clock clock) {
55         this.market = market;
56         this.clock = clock;
57         seed();
58     }
59
60     // ----- reads
61
62     public List<Account> accounts(String customerId) {
63         return accounts.values().stream().filter(a -> a.customerId().equals(customerId)).toList();
64     }
65
66     /**
67      * An account the customer owns. Someone else's account is reported as not found,
68      * not as forbidden: a 403 would confirm that the account exists.
69      */
70     public Account account(String customerId, String accountId) {
71         Account account = accounts.get(accountId);
72         if (account == null || !account.customerId().equals(customerId)) {
73             throw ApiException.notFound("Account", accountId);
74         }
75         return account;
76     }
77
78     public synchronized BigDecimal cash(String accountId) {
79         return books.get(accountId).cash;
80     }
81
82     public synchronized List<Holding> holdings(String accountId) {
83         return List.copyOf(books.get(accountId).holdings.values());
84     }
85
86     public synchronized BigDecimal quantityHeld(String accountId, String instrumentId) {
87         Holding holding = books.get(accountId).holdings.get(instrumentId);
88         return holding == null ? BigDecimal.ZERO : holding.quantity();
89     }
90
91     public synchronized List<Transaction> transactions(String accountId) {
92         return List.copyOf(books.get(accountId).transactions);
93     }
94

```

```

95 // ----- writes
96
97 /**
98  * Books one execution. A positive quantity is a purchase, a negative one a sale.
99  * Cash moves by quantity x price x multiplier, plus or minus commission.
100  */
101 public synchronized Transaction recordTrade(String accountId, Instrument instrument,
102     BigDecimal signedQuantity, BigDecimal price, BigDecimal commission, String orderId,
103     Instant at) {
104     Book book = books.get(accountId);
105     BigDecimal quantity = signedQuantity.abs();
106     BigDecimal gross = quantity.multiply(price).multiply(instrument.multiplier());
107     boolean buy = signedQuantity.signum() > 0;
108     BigDecimal cashChange = (buy ? gross.negate() : gross).subtract(commission)
109         .setScale(2, RoundingMode.HALF_EVEN);
110     book.cash = book.cash.add(cashChange);
111
112     Holding old = book.holdings.get(instrument.id());
113     BigDecimal oldQuantity = old == null ? BigDecimal.ZERO : old.quantity();
114     BigDecimal newQuantity = oldQuantity.add(signedQuantity);
115     if (newQuantity.signum() == 0) {
116         book.holdings.remove(instrument.id());
117     } else if (buy) {
118         BigDecimal oldCost = old == null ? BigDecimal.ZERO
119             : old.quantity().multiply(old.averageCost());
120         BigDecimal averageCost = oldCost.add(quantity.multiply(price))
121             .divide(newQuantity, 4, RoundingMode.HALF_EVEN);
122         book.holdings.put(instrument.id(), new Holding(instrument.id(), newQuantity, averageCost));
123     } else {
124         BigDecimal averageCost = old == null ? price : old.averageCost();
125         book.holdings.put(instrument.id(), new Holding(instrument.id(), newQuantity, averageCost));
126     }
127
128     String what = instrument.symbol() + (instrument.isOption() ? " option" : "");
129     String description = "%s %s %s @ %s".formatted(buy ? "Bought" : "Sold",
130         quantity.stripTrailingZeros().toPlainString(), what, price.toPlainString());
131     Transaction transaction = new Transaction(nextTransactionId(), Type.TRADE, description, cashChange,
132         instrument.id(), instrument.symbol(), quantity, price, orderId, at, settlementDate(at));
133     book.transactions.add(transaction);
134     return transaction;
135 }
136
137 /** US equity and option trades settle one business day after the trade date (T+1). */
138 private static LocalDate settlementDate(Instant tradeTime) {
139     LocalDate date = tradeTime.atZone(ZoneOffset.UTC).toLocalDate().plusDays(1);
140     while (date.getDayOfWeek() == DayOfWeek.SATURDAY || date.getDayOfWeek() == DayOfWeek.SUNDAY) {
141         date = date.plusDays(1);
142     }
143     return date;
144 }
145
146 private String nextTransactionId() {
147     return "TXN-" + transactionIds.incrementAndGet();
148 }
149
150 // ----- seed data
151
152 private void seed() {
153     open(new Account("ACC-1001", "cust-1001", "****4821", "Individual Margin", Account.Type.MARGIN,
154         Registration.INDIVIDUAL, "USD", LocalDate.of(2021, 3, 15)));
155     open(new Account("ACC-1002", "cust-1001", "****7304", "Roth IRA", Account.Type.CASH,
156         Registration.ROTH_IRA, "USD", LocalDate.of(2023, 1, 9)));
157     open(new Account("ACC-2001", "cust-2002", "****5518", "Individual Cash", Account.Type.CASH,
158         Registration.INDIVIDUAL, "USD", LocalDate.of(2024, 6, 2)));
159
160     // Every cash movement is a transaction, so balances reconcile with the history.
161     deposit("ACC-1001", "50000.00", 400);
162     deposit("ACC-1001", "10000.00", 120);
163     historicalBuy("ACC-1001", "EQ-AAPL", "50", "180.25", 360);

```

```

164     historicalBuy("ACC-1001", "EQ-MSFT", "20", "402.10", 300);
165     historicalBuy("ACC-1001", "ETF-SPY", "15", "540.00", 200);
166     dividend("ACC-1001", "EQ-AAPL", "13.00", 45);
167     dividend("ACC-1001", "ETF-SPY", "26.85", 20);
168     historicalBuy("ACC-1001", "EQ-AMZN", "12", "205.40", 9);
169
170     deposit("ACC-1002", "12000.00", 600);
171     historicalBuy("ACC-1002", "EQ-NVDA", "40", "120.50", 500);
172     dividend("ACC-1002", "EQ-NVDA", "0.40", 60);
173
174     deposit("ACC-2001", "5000.00", 90);
175     historicalBuy("ACC-2001", "EQ-AMZN", "10", "190.00", 80);
176     books.values().forEach(book -> book.transactions.sort(Comparator.comparing(Transaction::occurredAt)));
177 }
178
179 private void open(Account account) {
180     accounts.put(account.id(), account);
181     books.put(account.id(), new Book(BigDecimal.ZERO));
182 }
183
184 private Instant daysAgo(int days) {
185     return clock.instant().minus(Duration.ofDays(days)).minus(Duration.ofMinutes(days * 7L % 300));
186 }
187
188 private void deposit(String accountId, String amount, int daysAgo) {
189     Book book = books.get(accountId);
190     BigDecimal cash = new BigDecimal(amount);
191     book.cash = book.cash.add(cash);
192     Instant at = daysAgo(daysAgo);
193     book.transactions.add(new Transaction(nextTransactionId(), Type.DEPOSIT, "ACH deposit",
194         cash, null, null, null, null, null, at, settlementDate(at)));
195 }
196
197 private void dividend(String accountId, String instrumentId, String amount, int daysAgo) {
198     Instrument instrument = market.instrument(instrumentId);
199     Book book = books.get(accountId);
200     BigDecimal cash = new BigDecimal(amount);
201     book.cash = book.cash.add(cash);
202     Instant at = daysAgo(daysAgo);
203     LocalDate paid = at.atZone(ZoneOffset.UTC).toLocalDate(); // dividends are paid in cash, settled
204     book.transactions.add(new Transaction(nextTransactionId(), Type.DIVIDEND,
205         "Cash dividend " + instrument.symbol(), cash, instrument.id(), instrument.symbol(),
206         null, null, null, at, paid));
207 }
208
209 private void historicalBuy(String accountId, String instrumentId, String quantity, String price,
210     int daysAgo) {
211     recordTrade(accountId, market.instrument(instrumentId), new BigDecimal(quantity),
212         new BigDecimal(price), BigDecimal.ZERO, null, daysAgo(daysAgo));
213 }
214 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Portfolio.java

77 lines

```

1 package com.example.brokerage.api.accounts;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Clock;
6 import java.util.List;
7
8 import com.example.brokerage.api.accounts.Holdings.Balances;
9 import com.example.brokerage.api.accounts.Holdings.Position;
10 import com.example.brokerage.api.market.Instrument;
11 import com.example.brokerage.api.market.MarketData;
12 import com.example.brokerage.api.platform.ApiException;
13
14 import org.springframework.stereotype.Service;
15

```

```

16 /**
17  * Values the ledger at current market prices: positions and balances are computed on
18  * every request, never stored, so they can never drift from the books of record.
19  */
20 @Service
21 public class Portfolio {
22
23     private final Ledger ledger;
24     private final MarketData market;
25     private final Reservations reservations;
26     private final Clock clock;
27
28     public Portfolio(Ledger ledger, MarketData market, Reservations reservations, Clock clock) {
29         this.ledger = ledger;
30         this.market = market;
31         this.reservations = reservations;
32         this.clock = clock;
33     }
34
35     public List<Position> positions(Account account) {
36         return ledger.holdings(account.id()).stream().map(this::value).toList();
37     }
38
39     public Position position(Account account, String instrumentId) {
40         return ledger.holdings(account.id()).stream()
41             .filter(h -> h.instrumentId().equals(instrumentId))
42             .findFirst()
43             .map(this::value)
44             .orElseThrow(() -> ApiException.notFound("Position", instrumentId));
45     }
46
47     public Balances balances(Account account) {
48         List<Position> positions = positions(account);
49         BigDecimal cash = ledger.cash(account.id());
50         BigDecimal reserved = reservations.reservedCash(account.id());
51         BigDecimal marketValue = positions.stream().map(Position::marketValue)
52             .reduce(BigDecimal.ZERO, BigDecimal::add);
53         BigDecimal unrealized = positions.stream().map(Position::unrealizedPnl)
54             .reduce(BigDecimal.ZERO, BigDecimal::add);
55         return new Balances(account.id(), account.currency(), cash, reserved,
56             Holdings.buyingPower(account.type(), cash, reserved), marketValue, cash.add(marketValue),
57             unrealized, clock.instant());
58     }
59
60     private Position value(Ledger.Holding holding) {
61         Instrument instrument = market.instrument(holding.instrumentId());
62         BigDecimal last = market.tick(instrument.id()).last();
63         BigDecimal multiplier = instrument.multiplier();
64         BigDecimal costBasis = money(holding.quantity().multiply(holding.averageCost()).multiply(multiplier));
65         BigDecimal marketValue = money(holding.quantity().multiply(last).multiply(multiplier));
66         BigDecimal pnl = marketValue.subtract(costBasis);
67         BigDecimal pnlPercent = costBasis.signum() == 0 ? BigDecimal.ZERO
68             : pnl.multiply(BigDecimal.valueOf(100)).divide(costBasis, 2, RoundingMode.HALF_EVEN);
69         return new Position(instrument.id(), instrument.symbol(), instrument.type(), holding.quantity(),
70             holding.averageCost().setScale(2, RoundingMode.HALF_EVEN), costBasis, last, marketValue, pnl,
71             pnlPercent);
72     }
73
74     private static BigDecimal money(BigDecimal amount) {
75         return amount.setScale(2, RoundingMode.HALF_EVEN);
76     }
77 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Reservations.java

16 lines

```

1 package com.example.brokerage.api.accounts;
2
3 import java.math.BigDecimal;
4

```

```

5 /**
6  * What open orders have already claimed. Implemented by the order service; declared here
7  * so that the accounts package does not depend on the orders package.
8  */
9  public interface Reservations {
10
11      /** Cash held back for open buy orders. */
12      BigDecimal reservedCash(String accountId);
13
14      /** Units of an instrument already promised to open sell orders. */
15      BigDecimal reservedQuantity(String accountId, String instrumentId);
16  }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Transaction.java

22 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4  import java.time.Instant;
5  import java.time.LocalDate;
6
7  import org.springframework.hateoas.server.core.Relation;
8
9  /**
10 * One entry in an account's history. The amount is signed: money into the account is
11 * positive, money out is negative, so a client can sum a statement without a lookup table.
12 *
13 * @param settlesOn when the cash actually moves: US equity trades settle one business day
14 *                   after the trade (T+1)
15 */
16 @Relation(collectionRelation = "transactions", itemRelation = "transaction")
17 public record Transaction(String id, Type type, String description, BigDecimal amount, String instrumentId,
18     String symbol, BigDecimal quantity, BigDecimal price, String orderId, Instant occurredAt,
19     LocalDate settlesOn) {
20
21     public enum Type { DEPOSIT, WITHDRAWAL, TRADE, DIVIDEND, FEE }
22 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/IdempotencyKeys.java

56 lines

```

1  package com.example.brokerage.api.orders;
2
3  import java.time.Clock;
4  import java.time.Duration;
5  import java.time.Instant;
6  import java.util.Map;
7  import java.util.Optional;
8  import java.util.concurrent.ConcurrentHashMap;
9
10 import com.example.brokerage.api.platform.ApiException;
11 import com.example.brokerage.api.platform.ProblemType;
12
13 import org.springframework.stereotype.Component;
14
15 /**
16 * Remembers which order each Idempotency-Key created, for 24 hours.
17 *
18 * <p>A network timeout leaves a client unsure whether its order was placed. With a key,
19 * retrying is always safe: the same key and the same request return the original order
20 * instead of buying twice. The same key with a different request is a client bug, and is
21 * refused rather than guessed at.
22 */
23 @Component
24 public class IdempotencyKeys {
25
26     static final Duration RETENTION = Duration.ofHours(24);
27
28     private record Entry(String fingerprint, String orderId, Instant createdAt) {
29

```

```

30
31 private final Map<String, Entry> entries = new ConcurrentHashMap<>();
32 private final Clock clock;
33
34 public IdempotencyKeys(Clock clock) {
35     this.clock = clock;
36 }
37
38 /** The order already created with this key, if any. */
39 Optional<String> existingOrder(String scope, String key, String fingerprint) {
40     Instant now = clock.instant();
41     entries.values().removeIf(entry -> entry.createdAt().plus(RETENTION).isBefore(now));
42     Entry entry = entries.get(scope + "|" + key);
43     if (entry == null) {
44         return Optional.empty();
45     }
46     if (!entry.fingerprint().equals(fingerprint)) {
47         throw new ApiException(ProblemType.IDEMPOTENCY_KEY_REUSED,
48             "Idempotency-Key " + key + " was already used for a different order request");
49     }
50     return Optional.of(entry.orderId());
51 }
52
53 void remember(String scope, String key, String fingerprint, String orderId) {
54     entries.put(scope + "|" + key, new Entry(fingerprint, orderId, clock.instant()));
55 }
56 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/Order.java

105 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Instant;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 import com.example.brokerage.api.market.Instrument;
10 import com.fasterxml.jackson.annotation.JsonIgnore;
11 import com.fasterxml.jackson.annotation.JsonProperty;
12
13 import org.springframework.hateoas.server.core.Relation;
14
15 /**
16  * An order and its lifecycle. Orders are never deleted: a cancelled or filled order stays
17  * readable, which is why cancelling is a state change (POST ../cancellation), not DELETE.
18  *
19  * <pre>
20  * OPEN —fill→ PARTIALLY_FILLED —fill→ FILLED
21  *   |               |
22  *   └cancel→ PENDING_CANCEL ←cancel┐ → CANCELLED
23  * </pre>
24  *
25  * @param version incremented on every change; exposed only as the ETag header
26  */
27 @Relation(collectionRelation = "orders", itemRelation = "order")
28 public record Order(String id, String accountId, String clientOrderId, String instrumentId, String symbol,
29     Instrument.Type instrumentType, Side side, Type type, TimeInForce timeInForce, BigDecimal quantity,
30     BigDecimal limitPrice, BigDecimal stopPrice, Status status, BigDecimal filledQuantity,
31     BigDecimal averageFillPrice, List<Fill> fills, Instant createdAt, Instant updatedAt,
32     @JsonIgnore boolean stopTriggered, @JsonIgnore long version) {
33
34     public enum Side { BUY, SELL }
35
36     public enum Type { MARKET, LIMIT, STOP, STOP_LIMIT }
37
38     /** DAY orders expire at the close; GTC ("good 'til cancelled") orders stay working. */
39     public enum TimeInForce { DAY, GTC }

```

```

40
41 public enum Status {
42     OPEN, PARTIALLY_FILLED, FILLED, PENDING_CANCEL, CANCELLED, REJECTED;
43
44     /** Can still fill, and so can still be changed or cancelled. */
45     public boolean isWorking() {
46         return this == OPEN || this == PARTIALLY_FILLED;
47     }
48
49     /** Not yet in a final state. */
50     public boolean isOpen() {
51         return isWorking() || this == PENDING_CANCEL;
52     }
53 }
54
55 /** One execution against the order. */
56 public record Fill(String id, BigDecimal quantity, BigDecimal price, Instant executedAt) {
57 }
58
59 @JsonProperty
60 public BigDecimal remainingQuantity() {
61     return quantity.subtract(filledQuantity);
62 }
63
64 /** A strong entity tag: changes whenever the order changes. */
65 @JsonIgnore
66 public String etag() {
67     return "\"" + id + "-v" + version + "\"";
68 }
69
70 // ----- state changes
71
72 Order withStatus(Status newStatus, Instant at) {
73     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
74         timeInForce, quantity, limitPrice, stopPrice, newStatus, filledQuantity,
75         averageFillPrice, fills, createdAt, at, stopTriggered, version + 1);
76 }
77
78 Order withStopTriggered() {
79     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
80         timeInForce, quantity, limitPrice, stopPrice, status, filledQuantity, averageFillPrice, fills,
81         createdAt, updatedAt, true, version);
82 }
83
84 Order withFill(Fill fill) {
85     BigDecimal filled = filledQuantity.add(fill.quantity());
86     BigDecimal cost = averageFillPrice == null ? BigDecimal.ZERO
87         : averageFillPrice.multiply(filledQuantity);
88     BigDecimal average = cost.add(fill.price().multiply(fill.quantity()))
89         .divide(filled, 4, RoundingMode.HALF_EVEN);
90     List<Fill> allFills = new ArrayList<>(fills);
91     allFills.add(fill);
92     Status newStatus = filled.compareTo(quantity) == 0 ? Status.FILLED
93         : status == Status.PENDING_CANCEL ? Status.PENDING_CANCEL : Status.PARTIALLY_FILLED;
94     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
95         timeInForce, quantity, limitPrice, stopPrice, newStatus, filled, average,
96         List.copyOf(allFills), createdAt, fill.executedAt(), stopTriggered, version + 1);
97 }
98
99 Order withTerms(BigDecimal newQuantity, BigDecimal newLimit, BigDecimal newStop, TimeInForce newTif,
100     Instant at) {
101     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
102         newTif, newQuantity, newLimit, newStop, status, filledQuantity, averageFillPrice, fills,
103         createdAt, at, stopTriggered, version + 1);
104 }
105 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4
5 import jakarta.validation.constraints.Digits;
6 import jakarta.validation.constraints.Positive;
7
8 import org.springframework.hateoas.InputType;
9
10 /**
11  * The body of PATCH ../orders/{orderId}, sent as application/merge-patch+json (RFC 7396):
12  * only the fields present are changed. The instrument, side and type of a working order
13  * cannot change; cancel it and place a new one instead.
14  */
15 public class OrderAmendment {
16
17     @InputType("text") @Positive @Digits(integer = 9, fraction = 6)
18     private BigDecimal quantity;
19     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
20     private BigDecimal limitPrice;
21     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
22     private BigDecimal stopPrice;
23     private Order.TimeInForce timeInForce;
24
25     public BigDecimal getQuantity() { return quantity; }
26     public void setQuantity(BigDecimal quantity) { this.quantity = quantity; }
27     public BigDecimal getLimitPrice() { return limitPrice; }
28     public void setLimitPrice(BigDecimal limitPrice) { this.limitPrice = limitPrice; }
29     public BigDecimal getStopPrice() { return stopPrice; }
30     public void setStopPrice(BigDecimal stopPrice) { this.stopPrice = stopPrice; }
31     public Order.TimeInForce getTimeInForce() { return timeInForce; }
32     public void setTimeInForce(Order.TimeInForce timeInForce) { this.timeInForce = timeInForce; }
33 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6 import java.util.List;
7
8 import jakarta.validation.Valid;
9
10 import com.example.brokerage.api.accounts.AccountController;
11 import com.example.brokerage.api.platform.Caller;
12 import com.example.brokerage.api.platform.CursorPage;
13
14 import org.springframework.hateoas.CollectionModel;
15 import org.springframework.hateoas.EntityModel;
16 import org.springframework.hateoas.IanaLinkRelations;
17 import org.springframework.http.HttpHeaders;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.security.core.annotation.AuthenticationPrincipal;
20 import org.springframework.security.oauth2.jwt.Jwt;
21 import org.springframework.web.bind.annotation.GetMapping;
22 import org.springframework.web.bind.annotation.PatchMapping;
23 import org.springframework.web.bind.annotation.PathVariable;
24 import org.springframework.web.bind.annotation.PostMapping;
25 import org.springframework.web.bind.annotation.RequestBody;
26 import org.springframework.web.bind.annotation.RequestHeader;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29 import org.springframework.web.context.request.WebRequest;
30
31 /**

```

```

32  * Orders. Reading needs orders:read; placing, changing and cancelling need orders:write.
33  */
34  @RestController
35  public class OrderController {
36
37      public static final String IDEMPOTENCY_KEY = "Idempotency-Key";
38      public static final String MERGE_PATCH_JSON = "application/merge-patch+json";
39
40      private final OrderService service;
41      private final OrderModelAssembler assembler;
42
43      public OrderController(OrderService service, OrderModelAssembler assembler) {
44          this.service = service;
45          this.assembler = assembler;
46      }
47
48      /** Orders, newest first, optionally filtered by status (repeatable) and symbol. */
49      @GetMapping("/accounts/{accountId}/orders")
50      public CollectionModel<EntityModel<Order>> orders(@PathVariable String accountId,
51          @RequestParam(required = false) List<Order.Status> status,
52          @RequestParam(required = false) String symbol,
53          @RequestParam(required = false) String cursor, @RequestParam(required = false) Integer limit,
54          @AuthenticationPrincipal Jwt jwt) {
55          Caller caller = Caller.of(jwt);
56          CursorPage<Order> page = service.orders(caller, accountId, status, symbol, cursor, limit);
57          CollectionModel<EntityModel<Order>> model = CollectionModel.of(
58              page.items().stream().map(order -> assembler.toModel(order, caller)).toList(),
59              linkTo(methodOn(OrderController.class).orders(accountId, status, symbol, cursor, limit, null))
60                  .withSelfRel().expand(),
61              linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
62          if (page.hasNext()) {
63              model.add(linkTo(methodOn(OrderController.class)
64                  .orders(accountId, status, symbol, page.nextCursor(), limit, null))
65                  .withRel(IanaLinkRelations.NEXT).expand());
66          }
67          return model;
68      }
69
70      /**
71       * Places an order: 201 Created with a Location header and the order's ETag. Retrying
72       * with the same Idempotency-Key returns the original order, marked Idempotent-Replayed.
73       */
74      @PostMapping("/accounts/{accountId}/orders")
75      public ResponseEntity<EntityModel<Order>> placeOrder(@PathVariable String accountId,
76          @RequestHeader(name = IDEMPOTENCY_KEY, required = false) String idempotencyKey,
77          @Valid @RequestBody OrderRequest request, @AuthenticationPrincipal Jwt jwt) {
78          Caller caller = Caller.of(jwt);
79          OrderService.Placement placement = service.place(caller, accountId, request, idempotencyKey);
80          EntityModel<Order> model = assembler.toModel(placement.order(), caller);
81          return ResponseEntity.created(model.getRequiredLink(IanaLinkRelations.SELF).toUri())
82              .eTag(placement.order().eTag())
83              .header("Idempotent-Replayed", Boolean.toString(placement.replayed()))
84              .body(model);
85      }
86
87      /** One order. Its ETag supports cheap polling (If-None-Match) and safe edits (If-Match). */
88      @GetMapping("/accounts/{accountId}/orders/{orderId}")
89      public ResponseEntity<EntityModel<Order>> order(@PathVariable String accountId,
90          @PathVariable String orderId, WebRequest request, @AuthenticationPrincipal Jwt jwt) {
91          Caller caller = Caller.of(jwt);
92          Order order = service.order(caller, accountId, orderId);
93          if (request.checkNotModified(order.eTag())) {
94              return null;
95          }
96          return ResponseEntity.ok().eTag(order.eTag()).body(assembler.toModel(order, caller));
97      }
98
99      /** Changes a working order's quantity, prices or time in force. Requires If-Match. */
100     @PatchMapping(path = "/accounts/{accountId}/orders/{orderId}",

```

```

101         consumes = { MERGE_PATCH_JSON, "application/json" })
102     public ResponseEntity<EntityModel<Order>> replaceOrder(@PathVariable String accountId,
103         @PathVariable String orderId,
104         @RequestHeader(name = HttpHeaders.IF_MATCH, required = false) String ifMatch,
105         @Valid @RequestBody OrderAmendment change, @AuthenticationPrincipal Jwt jwt) {
106         Caller caller = Caller.of(jwt);
107         Order order = service.amend(caller, accountId, orderId, change, ifMatch);
108         return ResponseEntity.ok().eTag(order.etag()).body(assembler.toModel(order, caller));
109     }
110
111     /**
112      * Requests cancellation. 202 Accepted: the cancel is in progress, and the Location
113      * header points at the order, whose status will become CANCELLED.
114      */
115     @PostMapping("/accounts/{accountId}/orders/{orderId}/cancellation")
116     public ResponseEntity<EntityModel<Order>> cancelOrder(@PathVariable String accountId,
117         @PathVariable String orderId, @AuthenticationPrincipal Jwt jwt) {
118         Caller caller = Caller.of(jwt);
119         Order order = service.cancel(caller, accountId, orderId);
120         EntityModel<Order> model = assembler.toModel(order, caller);
121         return ResponseEntity.accepted()
122             .location(linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
123                 .toUri())
124             .eTag(order.etag())
125             .body(model);
126     }
127
128     /**
129      * Estimates an order without placing it. Safe to call as often as a user edits a
130      * ticket; when the order would be accepted, a link to place it is included.
131      */
132     @PostMapping("/accounts/{accountId}/order-previews")
133     public EntityModel<OrderPreview> previewOrder(@PathVariable String accountId,
134         @Valid @RequestBody OrderRequest request, @AuthenticationPrincipal Jwt jwt) {
135         Caller caller = Caller.of(jwt);
136         OrderPreview preview = service.preview(caller, accountId, request);
137         EntityModel<OrderPreview> model = EntityModel.of(preview,
138             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
139         if (preview.acceptable() && caller.canTrade()) {
140             model.add(linkTo(methodOn(OrderController.class).orders(accountId, null, null, null, null, null))
141                 .withRel("place-order"));
142         }
143         return model;
144     }
145 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderEventController.java

58 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.time.Instant;
5 import java.util.Map;
6
7 import com.example.brokerage.api.accounts.Ledger;
8 import com.example.brokerage.api.platform.Caller;
9 import com.fasterxml.jackson.annotation.JsonProperty;
10
11 import org.springframework.http.MediaType;
12 import org.springframework.security.core.annotation.AuthenticationPrincipal;
13 import org.springframework.security.oauth2.jwt.Jwt;
14 import org.springframework.web.bind.annotation.GetMapping;
15 import org.springframework.web.bind.annotation.PathVariable;
16 import org.springframework.web.bind.annotation.RequestHeader;
17 import org.springframework.web.bind.annotation.RestController;
18 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
19 import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
20
21 /**

```

```

22  * A live stream of an account's order events (text/event-stream). Scope: orders:read.
23  */
24  @RestController
25  public class OrderEventController {
26
27      /** The data of one event: a summary, and a link to the full order. */
28      public record OrderEventData(String orderId, Order.Status status, BigDecimal filledQuantity,
29          BigDecimal remainingQuantity, BigDecimal averageFillPrice, Instant occurredAt,
30          @JsonProperty("_links") Map<String, Map<String, String>> links) {
31      }
32
33      private final OrderEvents events;
34      private final Ledger ledger;
35
36      public OrderEventController(OrderEvents events, Ledger ledger) {
37          this.events = events;
38          this.ledger = ledger;
39      }
40
41      @GetMapping(path = "/accounts/{accountId}/order-events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
42      public SseEmitter events(@PathVariable String accountId,
43          @RequestHeader(name = "Last-Event-ID", required = false) Long lastEventId,
44          @AuthenticationPrincipal Jwt jwt) {
45          ledger.account(Caller.of(jwt).customerId(), accountId);
46          // Events are sent later, outside this request, so capture the base URI now.
47          String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
48          return events.subscribe(accountId, lastEventId, event -> render(base, event));
49      }
50
51      static OrderEventData render(String base, OrderEvents.Event event) {
52          Order order = event.order();
53          String href = base + "/accounts/" + order.accountId() + "/orders/" + order.id();
54          return new OrderEventData(order.id(), order.status(), order.filledQuantity(),
55              order.remainingQuantity(), order.averageFillPrice(), event.at(),
56              Map.of("order", Map.of("href", href)));
57      }
58  }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderEvents.java

96 lines

```

1  package com.example.brokerage.api.orders;
2
3  import java.io.IOException;
4  import java.time.Clock;
5  import java.time.Instant;
6  import java.util.ArrayDeque;
7  import java.util.Deque;
8  import java.util.List;
9  import java.util.Map;
10 import java.util.concurrent.ConcurrentHashMap;
11 import java.util.concurrent.CopyOnWriteArrayList;
12 import java.util.concurrent.atomic.AtomicLong;
13 import java.util.function.Function;
14
15 import org.springframework.http.MediaType;
16 import org.springframework.stereotype.Component;
17 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
18
19 /**
20  * Order lifecycle events, pushed to subscribers as Server-Sent Events.
21  *
22  * <p>Every event gets an increasing ID, and the last 500 events of each account are kept.
23  * A client that loses its connection reconnects with a Last-Event-ID header and receives
24  * exactly the events it missed: no polling, and no gaps.
25  */
26 @Component
27 public class OrderEvents {
28
29     /** Something happened to an order. The type is order.created, order.filled, etc. */

```

```

30     public record Event(long id, String type, Order order, Instant at) {
31     }
32
33     private record Subscriber(SseEmitter emitter, Function<Event, Object> renderer) {
34     }
35
36     private static final int HISTORY = 500;
37     private static final long TIMEOUT_MILLIS = 30 * 60 * 1000L;
38
39     private final AtomicLong sequence = new AtomicLong();
40     private final Map<String, Deque<Event>> history = new ConcurrentHashMap<>();
41     private final Map<String, List<Subscriber>> subscribers = new ConcurrentHashMap<>();
42     private final Clock clock;
43
44     public OrderEvents(Clock clock) {
45         this.clock = clock;
46     }
47
48     void publish(String type, Order order) {
49         Event event = new Event(sequence.incrementAndGet(), type, order, clock.instant());
50         Deque<Event> events = history.computeIfAbsent(order.accountId(), id -> new ArrayDeque<>());
51         synchronized (events) {
52             events.addLast(event);
53             if (events.size() > HISTORY) {
54                 events.removeFirst();
55             }
56         }
57         for (Subscriber subscriber : subscribers.getOrDefault(order.accountId(), List.of())) {
58             send(order.accountId(), subscriber, event);
59         }
60     }
61
62     /** Opens a stream for one account, first replaying anything after lastEventId. */
63     public SseEmitter subscribe(String accountId, Long lastEventId, Function<Event, Object> renderer) {
64         Subscriber subscriber = new Subscriber(new SseEmitter(TIMEOUT_MILLIS), renderer);
65         subscriber.emitter().onCompletion(() -> unsubscribe(accountId, subscriber));
66         subscriber.emitter().onTimeout(() -> unsubscribe(accountId, subscriber));
67         if (lastEventId != null) {
68             replay(accountId, lastEventId).forEach(event -> send(accountId, subscriber, event));
69         }
70         subscribers.computeIfAbsent(accountId, id -> new CopyOnWriteArrayList<>()).add(subscriber);
71         return subscriber.emitter();
72     }
73
74     /** Events for an account with an ID greater than lastEventId, oldest first. */
75     public List<Event> replay(String accountId, long lastEventId) {
76         Deque<Event> events = history.getOrDefault(accountId, new ArrayDeque<>());
77         synchronized (events) {
78             return events.stream().filter(event -> event.id() > lastEventId).toList();
79         }
80     }
81
82     private void send(String accountId, Subscriber subscriber, Event event) {
83         try {
84             subscriber.emitter().send(SseEmitter.event()
85                 .id(Long.toString(event.id()))
86                 .name(event.type())
87                 .data(subscriber.renderer().apply(event), MediaType.APPLICATION_JSON));
88         } catch (IOException | IllegalStateException gone) {
89             unsubscribe(accountId, subscriber);
90         }
91     }
92
93     private void unsubscribe(String accountId, Subscriber subscriber) {
94         subscribers.getOrDefault(accountId, List.of()).remove(subscriber);
95     }
96 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.afford;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
5 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
6
7 import com.example.brokerage.api.accounts.AccountController;
8 import com.example.brokerage.api.market.InstrumentController;
9 import com.example.brokerage.api.platform.Caller;
10
11 import org.springframework.hateoas.EntityModel;
12 import org.springframework.hateoas.IanaLinkRelations;
13 import org.springframework.hateoas.Link;
14 import org.springframework.stereotype.Component;
15
16 /**
17  * Builds an order's representation. This is where hypermedia earns its keep: the links
18  * an order carries depend on its state and on the caller's token.
19  *
20  * <ul>
21  * <li>A working order, seen by a token with orders:write, offers "edit" (change it with
22  * PATCH) and "bk:cancel"; in HAL-FORMS both come with a template describing the
23  * request.</li>
24  * <li>A filled or cancelled order offers neither. A client that shows buttons only for
25  * links that are present can never offer an action the server would refuse.</li>
26  * </ul>
27  */
28 @Component
29 public class OrderModelAssembler {
30
31     public EntityModel<Order> toModel(Order order, Caller caller) {
32         String accountId = order.accountId();
33         String orderId = order.id();
34         boolean changeable = order.status().isWorking() && caller.canTrade();
35
36         Link self = linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
37             .withSelfRel();
38         if (changeable) {
39             self = self.andAffordance(afford(methodOn(OrderController.class)
40                 .replaceOrder(accountId, orderId, null, null, null)));
41         }
42         EntityModel<Order> model = EntityModel.of(order, self,
43             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"),
44             linkTo(methodOn(InstrumentController.class).instrument(order.instrumentId()))
45                 .withRel("instrument"));
46
47         if (changeable) {
48             model.add(linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
49                 .withRel(IanaLinkRelations.EDIT));
50             model.add(linkTo(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))
51                 .withRel("cancel")
52                 .andAffordance(
53                     afford(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))));
54         }
55         return model;
56     }
57 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.net.URI;
5 import java.util.List;
6
7 /**

```

```

8  * What an order would cost and whether it would be accepted, without placing it. A preview
9  * that fails a business rule is still a successful response: "acceptable" is false and each
10 * issue carries the same problem type URI that placing the order would return.
11 *
12 * @param estimatedTotal for a purchase, the cash needed; for a sale, the expected proceeds
13 */
14 public record OrderPreview(String accountId, String instrumentId, String symbol, Order.Side side,
15     Order.Type type, BigDecimal quantity, BigDecimal estimatedPrice, BigDecimal estimatedNotional,
16     BigDecimal estimatedCommission, BigDecimal estimatedTotal, BigDecimal buyingPower,
17     boolean acceptable, List<Issue> issues) {
18
19     /** A reason the order would be refused. */
20     public record Issue(URI type, String detail) {
21     }
22 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderRequest.java

88 lines

```

1  package com.example.brokerage.api.orders;
2
3  import java.math.BigDecimal;
4
5  import jakarta.validation.constraints.Digits;
6  import jakarta.validation.constraints.NotBlank;
7  import jakarta.validation.constraints.NotNull;
8  import jakarta.validation.constraints.Pattern;
9  import jakarta.validation.constraints.Positive;
10 import jakarta.validation.constraints.Size;
11
12 import org.springframework.hateoas.InputType;
13
14 /**
15  * The body of POST .../orders and POST .../order-previews. Field-level rules are declared
16  * here and checked by Bean Validation; rules that involve several fields or the account
17  * (a limit order needs a limit price, a sale needs shares to sell) live in OrderService.
18  *
19  * <p>This is a JavaBean rather than a record on purpose: Spring HATEOAS derives HAL-FORMS
20  * templates from bean properties, and marks a property without a setter as read-only.
21  */
22 public class OrderRequest {
23
24     @NotBlank
25     private String instrumentId;
26     @NotNull
27     private Order.Side side;
28     @NotNull
29     private Order.Type type;
30     private Order.TimeInForce timeInForce;
31     @InputType("text") @NotNull @Positive @Digits(integer = 9, fraction = 6)
32     private BigDecimal quantity;
33     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
34     private BigDecimal limitPrice;
35     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
36     private BigDecimal stopPrice;
37     /** The caller's own reference for the order, echoed back unchanged. */
38     @InputType("text") @Size(max = 64) @Pattern(regexp = "[A-Za-z0-9._]*")
39     private String clientOrderId;
40
41     public OrderRequest() {
42     }
43
44     public OrderRequest(String instrumentId, Order.Side side, Order.Type type, Order.TimeInForce timeInForce,
45         BigDecimal quantity, BigDecimal limitPrice, BigDecimal stopPrice, String clientOrderId) {
46         this.instrumentId = instrumentId;
47         this.side = side;
48         this.type = type;
49         this.timeInForce = timeInForce;
50         this.quantity = quantity;
51         this.limitPrice = limitPrice;

```

```

52         this.stopPrice = stopPrice;
53         this.clientOrderId = clientOrderId;
54     }
55
56     public Order.TimeInForce timeInForceOrDefault() {
57         return timeInForce == null ? Order.TimeInForce.DAY : timeInForce;
58     }
59
60     /** A stable fingerprint, so a retried request can be recognized as the same request. */
61     String fingerprint() {
62         return String.join("|", instrumentId, side.name(), type.name(), timeInForceOrDefault().name(),
63             plain(quantity), plain(limitPrice), plain(stopPrice), String.valueOf(clientOrderId));
64     }
65
66     private static String plain(BigDecimal value) {
67         return value == null ? "" : value.stripTrailingZeros().toPlainString();
68     }
69
70     // ----- accessors
71
72     public String getInstrumentId() { return instrumentId; }
73     public void setInstrumentId(String instrumentId) { this.instrumentId = instrumentId; }
74     public Order.Side getSide() { return side; }
75     public void setSide(Order.Side side) { this.side = side; }
76     public Order.Type getType() { return type; }
77     public void setType(Order.Type type) { this.type = type; }
78     public Order.TimeInForce getTimeInForce() { return timeInForce; }
79     public void setTimeInForce(Order.TimeInForce timeInForce) { this.timeInForce = timeInForce; }
80     public BigDecimal getQuantity() { return quantity; }
81     public void setQuantity(BigDecimal quantity) { this.quantity = quantity; }
82     public BigDecimal getLimitPrice() { return limitPrice; }
83     public void setLimitPrice(BigDecimal limitPrice) { this.limitPrice = limitPrice; }
84     public BigDecimal getStopPrice() { return stopPrice; }
85     public void setStopPrice(BigDecimal stopPrice) { this.stopPrice = stopPrice; }
86     public String getClientOrderId() { return clientOrderId; }
87     public void setClientId(String clientOrderId) { this.clientOrderId = clientOrderId; }
88 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderService.java

402 lines

```

1  package com.example.brokerage.api.orders;
2
3  import java.math.BigDecimal;
4  import java.math.RoundingMode;
5  import java.time.Clock;
6  import java.time.Instant;
7  import java.util.ArrayList;
8  import java.util.Collection;
9  import java.util.LinkedHashMap;
10 import java.util.List;
11 import java.util.Map;
12 import java.util.Optional;
13 import java.util.concurrent.atomic.AtomicInteger;
14
15 import com.example.brokerage.api.accounts.Account;
16 import com.example.brokerage.api.accounts.Holdings;
17 import com.example.brokerage.api.accounts.Ledger;
18 import com.example.brokerage.api.accounts.Reservations;
19 import com.example.brokerage.api.market.Instrument;
20 import com.example.brokerage.api.market.MarketData;
21 import com.example.brokerage.api.orders.Order.Side;
22 import com.example.brokerage.api.orders.Order.Status;
23 import com.example.brokerage.api.orders.Order.TimeInForce;
24 import com.example.brokerage.api.orders.Order.Type;
25 import com.example.brokerage.api.platform.ApiException;
26 import com.example.brokerage.api.platform.Caller;
27 import com.example.brokerage.api.platform.CursorPage;
28 import com.example.brokerage.api.platform.ProblemType;
29

```

```

30 import org.springframework.stereotype.Service;
31
32 /**
33  * Order placement, preview, amendment, cancellation and matching.
34  *
35  * <p>All state changes are serialized by one lock: simple and obviously correct, which
36  * is what an example should be. A real system would partition by account.
37  */
38 @Service
39 public class OrderService implements Reservations {
40
41     /** Placement result: the order, and whether this was a replay of an earlier request. */
42     public record Placement(Order order, boolean replayed) {
43     }
44
45     private static final BigDecimal OPTION_COMMISSION = new BigDecimal("0.65");
46
47     private final Map<String, Order> orders = new LinkedHashMap<>();
48     private final AtomicInteger orderIds = new AtomicInteger(100_000);
49     private final AtomicInteger fillIds = new AtomicInteger(500_000);
50     private final Ledger ledger;
51     private final MarketData market;
52     private final IdempotencyKeys idempotencyKeys;
53     private final OrderEvents events;
54     private final Clock clock;
55
56     public OrderService(Ledger ledger, MarketData market, IdempotencyKeys idempotencyKeys,
57         OrderEvents events, Clock clock) {
58         this.ledger = ledger;
59         this.market = market;
60         this.idempotencyKeys = idempotencyKeys;
61         this.events = events;
62         this.clock = clock;
63         // A resting limit order, so that there is something to change and cancel.
64         Instant yesterday = clock.instant().minusSeconds(86_400);
65         save(new Order(nextOrderId(), "ACC-1001", "rebalance-msft-01", "EQ-MSFT", "MSFT",
66             Instrument.Type.EQUITY, Side.BUY, Type.LIMIT, TimeInForce.GTC, new BigDecimal("10"),
67             new BigDecimal("480.00"), null, Status.OPEN, BigDecimal.ZERO, null, List.of(),
68             yesterday, yesterday, false, 1));
69     }
70
71     // ----- queries
72
73     public synchronized Order order(Caller caller, String accountId, String orderId) {
74         ledger.account(caller.customerId(), accountId);
75         Order order = orders.get(orderId);
76         if (order == null || !order.accountId().equals(accountId)) {
77             throw ApiException.notFound("Order", orderId);
78         }
79         return order;
80     }
81
82     public synchronized CursorPage<Order> orders(Caller caller, String accountId,
83         Collection<Status> statuses, String symbol, String cursor, Integer limit) {
84         ledger.account(caller.customerId(), accountId);
85         List<Order> matching = orders.values().stream()
86             .filter(o -> o.accountId().equals(accountId))
87             .filter(o -> statuses == null || statuses.isEmpty() || statuses.contains(o.status()))
88             .filter(o -> symbol == null || o.symbol().equalsIgnoreCase(symbol))
89             .toList();
90         return CursorPage.of(matching, Order::createdAt, Order::id, cursor, limit);
91     }
92
93     // ----- commands
94
95     public synchronized Placement place(Caller caller, String accountId, OrderRequest request,
96         String idempotencyKey) {
97         Account account = ledger.account(caller.customerId(), accountId);
98         if (idempotencyKey == null || idempotencyKey.isBlank() || idempotencyKey.length() > 255) {

```

```

99         throw new ApiException(ProblemType.IDEMPOTENCY_KEY_MISSING,
100             "Send a unique Idempotency-Key header (up to 255 characters) with every new order");
101     }
102     String scope = caller.clientId() + "|" + accountId;
103     String fingerprint = request.fingerprint();
104     Optional<String> earlier = idempotencyKeys.existingOrder(scope, idempotencyKey, fingerprint);
105     if (earlier.isPresent()) {
106         return new Placement(orders.get(earlier.get()), true);
107     }
108
109     Assessment assessment = assess(caller, account, request);
110     assessment.issues().stream().findFirst().ifPresent(issue -> {
111         throw issue;
112     });
113
114     Instant now = clock.instant();
115     Instrument instrument = assessment.instrument();
116     Order order = new Order(nextOrderId(), accountId, request.getClientOrderId(), instrument.id(),
117         instrument.symbol(), instrument.type(), request.getSide(), request.getType(),
118         request.timeInForceOrDefault(), request.getQuantity(), request.getLimitPrice(),
119         request.getStopPrice(), Status.OPEN, BigDecimal.ZERO, null, List.of(), now, now,
120         false, 1);
121     save(order);
122     idempotencyKeys.remember(scope, idempotencyKey, fingerprint, order.id());
123     events.publish("order.created", order);
124     return new Placement(order, false);
125 }
126
127 public synchronized OrderPreview preview(Caller caller, String accountId, OrderRequest request) {
128     Account account = ledger.account(caller.customerId(), accountId);
129     Assessment assessment = assess(caller, account, request);
130     List<OrderPreview.Issue> issues = assessment.issues().stream()
131         .map(issue -> new OrderPreview.Issue(issue.type().uri(), issue.getMessage()))
132         .toList();
133     Instrument instrument = assessment.instrument();
134     return new OrderPreview(accountId, request.getInstrumentId(),
135         instrument == null ? null : instrument.symbol(), request.getSide(), request.getType(),
136         request.getQuantity(), assessment.price(), assessment.notional(), assessment.commission(),
137         assessment.total(), assessment.buyingPower(), issues.isEmpty(), issues);
138 }
139
140 /**
141  * Changes a working order. The If-Match header must carry the order's current ETag:
142  * two clients editing the same order cannot silently overwrite one another.
143  */
144 public synchronized Order amend(Caller caller, String accountId, String orderId,
145     OrderAmendment change, String ifMatch) {
146     Order order = order(caller, accountId, orderId);
147     if (ifMatch == null) {
148         throw new ApiException(ProblemType.PRECONDITION_REQUIRED,
149             "Send If-Match with the order's current ETag to change it");
150     }
151     if (!ifMatch.equals(order.etag()) && !ifMatch.equals("*")) {
152         throw new ApiException(ProblemType.PRECONDITION_FAILED,
153             "The order has changed since you read it; fetch it again and reapply your change")
154             .with("currentEtag", order.etag());
155     }
156     if (!order.status().isWorking()) {
157         throw new ApiException(ProblemType.ORDER_NOT_REPLACEABLE,
158             "Order " + orderId + " is " + order.status() + " and can no longer be changed");
159     }
160     BigDecimal quantity = orElse(change.getQuantity(), order.quantity());
161     if (quantity.compareTo(order.filledQuantity()) <= 0) {
162         throw new ApiException(ProblemType.INVALID_REQUEST, "quantity must be greater than the "
163             + order.filledQuantity().toPlainString() + " already filled");
164     }
165     OrderRequest amended = new OrderRequest(order.instrumentId(), order.side(), order.type(),
166         orElse(change.getTimeInForce(), order.timeInForce()), quantity,
167         orElse(change.getLimitPrice(), order.limitPrice()),

```

```

168         orElse(change.getStopPrice(), order.stopPrice(), order.clientOrderId());
169
170     Account account = ledger.account(caller.customerId(), accountId);
171     orders.remove(orderId); // assess the new terms without this order's own reservation
172     try {
173         assess(caller, account, amended).issues().stream().findFirst().ifPresent(issue -> {
174             throw issue;
175         });
176     } finally {
177         orders.put(orderId, order);
178     }
179     Order updated = order.withTerms(amended.getQuantity(), amended.getLimitPrice(),
180         amended.getStopPrice(), amended.getTimeInForce(), clock.instant());
181     save(updated);
182     events.publish("order.replaced", updated);
183     return updated;
184 }
185
186 /**
187  * Requests cancellation. Like a real exchange, the cancel is asynchronous: the order
188  * moves to PENDING_CANCEL now and to CANCELLED at the next matching pass, and it may
189  * still fill in between.
190  */
191 public synchronized Order cancel(Caller caller, String accountId, String orderId) {
192     Order order = order(caller, accountId, orderId);
193     if (order.status() == Status.PENDING_CANCEL) {
194         return order;
195     }
196     if (!order.status().isWorking()) {
197         throw new ApiException(ProblemType.ORDER_NOT_CANCELLABLE,
198             "Order " + orderId + " is " + order.status() + " and cannot be cancelled");
199     }
200     Order pending = order.withStatus(Status.PENDING_CANCEL, clock.instant());
201     save(pending);
202     events.publish("order.cancel-requested", pending);
203     return pending;
204 }
205
206 // ----- matching
207
208 /** One pass of the simulated exchange over every open order. */
209 public synchronized void match() {
210     for (Order order : List.copyOf(orders.values())) {
211         if (order.status() == Status.PENDING_CANCEL) {
212             Order cancelled = order.withStatus(Status.CANCELLED, clock.instant());
213             save(cancelled);
214             events.publish("order.cancelled", cancelled);
215         } else if (order.status().isWorking()) {
216             tryToFill(order);
217         }
218     }
219 }
220
221 private void tryToFill(Order order) {
222     Instrument instrument = market.instrument(order.instrumentId());
223     MarketData.Tick tick = market.tick(instrument.id());
224     boolean buy = order.side() == Side.BUY;
225     Order current = order;
226
227     if ((order.type() == Type.STOP || order.type() == Type.STOP_LIMIT) && !order.stopTriggered()) {
228         boolean triggered = buy ? tick.last().compareTo(order.stopPrice()) >= 0
229             : tick.last().compareTo(order.stopPrice()) <= 0;
230         if (!triggered) {
231             return;
232         }
233         current = current.withStopTriggered();
234         save(current);
235     }
236 }

```

```

237     BigDecimal price = buy ? tick.ask() : tick.bid();
238     if (order.type() == Type.LIMIT || order.type() == Type.STOP_LIMIT) {
239         boolean marketable = buy ? price.compareTo(order.limitPrice()) <= 0
240             : price.compareTo(order.limitPrice()) >= 0;
241         if (!marketable) {
242             return;
243         }
244     }
245
246     BigDecimal displayed = buy ? tick.askSize() : tick.bidSize();
247     BigDecimal quantity = current.remainingQuantity().min(displayed);
248     Instant now = clock.instant();
249     ledger.recordTrade(order.accountId(), instrument, buy ? quantity : quantity.negate(), price,
250         commission(instrument, quantity), order.id(), now);
251     String fillId = "FIL-" + fillIds.incrementAndGet();
252     Order filled = current.withFill(new Order.Fill(fillId, quantity, price, now));
253     save(filled);
254     boolean complete = filled.status() == Status.FILLED;
255     events.publish(complete ? "order.filled" : "order.partially-filled", filled);
256 }
257
258 // ----- reservations
259
260 @Override
261 public synchronized BigDecimal reservedCash(String accountId) {
262     return orders.values().stream()
263         .filter(o -> o.accountId().equals(accountId))
264         .filter(o -> o.side() == Side.BUY && o.status().isOpen())
265         .map(o -> {
266             Instrument instrument = market.instrument(o.instrumentId());
267             BigDecimal price = referencePrice(o.type(), Side.BUY, o.limitPrice(),
268                 o.stopPrice(), instrument);
269             return price.multiply(o.remainingQuantity()).multiply(instrument.multiplier())
270                 .add(commission(instrument, o.remainingQuantity()));
271         })
272         .reduce(BigDecimal.ZERO, BigDecimal::add)
273         .setScale(2, RoundingMode.HALF_EVEN);
274 }
275
276 @Override
277 public synchronized BigDecimal reservedQuantity(String accountId, String instrumentId) {
278     return orders.values().stream()
279         .filter(o -> o.accountId().equals(accountId) && o.instrumentId().equals(instrumentId))
280         .filter(o -> o.side() == Side.SELL && o.status().isOpen())
281         .map(Order::remainingQuantity)
282         .reduce(BigDecimal.ZERO, BigDecimal::add);
283 }
284
285 // ----- the rules
286
287 /** The outcome of checking an order request against the rules, without acting on it. */
288 private record Assessment(Instrument instrument, BigDecimal price, BigDecimal notional,
289     BigDecimal commission, BigDecimal total, BigDecimal buyingPower,
290     List<ApiException> issues) {
291
292     static Assessment failed(Instrument instrument, List<ApiException> issues) {
293         return new Assessment(instrument, null, null, null, null, null, issues);
294     }
295 }
296
297 private Assessment assess(Caller caller, Account account, OrderRequest request) {
298     List<ApiException> issues = new ArrayList<>();
299     Instrument instrument;
300     try {
301         instrument = market.instrument(request.getInstrumentId());
302     } catch (ApiException notFound) {
303         issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,
304             "Instrument " + request.getInstrumentId() + " does not exist"));
305         return Assessment.failed(null, issues);

```

```

306     }
307
308     checkPrices(request, issues);
309     if (!instrument.tradable()) {
310         issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,
311             instrument.symbol() + " is not tradable"));
312     }
313     boolean wholeUnits = request.getQuantity().stripTrailingZeros().scale() <= 0;
314     if (!wholeUnits && (!instrument.fractionable() || request.getType() != Type.MARKET)) {
315         issues.add(new ApiException(ProblemType.INVALID_REQUEST,
316             "Fractional quantities are allowed only in market orders for fractionable stocks"));
317     }
318     if (!issues.isEmpty()) {
319         return Assessment.failed(instrument, issues);
320     }
321
322     BigDecimal price = referencePrice(request.getType(), request.getSide(),
323         request.getLimitPrice(), request.getStopPrice(), instrument);
324     BigDecimal notional = price.multiply(request.getQuantity())
325         .multiply(instrument.multiplier())
326         .setScale(2, RoundingMode.HALF_EVEN);
327     BigDecimal commission = commission(instrument, request.getQuantity());
328     boolean buy = request.getSide() == Side.BUY;
329     BigDecimal total = buy ? notional.add(commission) : notional.subtract(commission);
330     BigDecimal buyingPower = Holdings.buyingPower(account.type(), ledger.cash(account.id()),
331         reservedCash(account.id()));
332
333     if (buy && total.compareTo(buyingPower) > 0) {
334         issues.add(new ApiException(ProblemType.INSUFFICIENT_BUYING_POWER,
335             "This order needs " + total.toPlainString() + " but only "
336             + buyingPower.toPlainString() + " is available")
337             .with("required", total).with("available", buyingPower));
338     }
339     if (!buy) {
340         BigDecimal available = ledger.quantityHeld(account.id(), instrument.id())
341             .subtract(reservedQuantity(account.id(), instrument.id()));
342         if (request.getQuantity().compareTo(available) > 0) {
343             issues.add(new ApiException(ProblemType.INSUFFICIENT_POSITION,
344                 "Only " + available.toPlainString() + " " + instrument.symbol()
345                 + " are available to sell; short selling is not supported")
346                 .with("available", available));
347         }
348     }
349     caller.tradingLimit().filter(limit -> notional.compareTo(limit) > 0).ifPresent(limit ->
350         issues.add(new ApiException(ProblemType.TRADING_LIMIT_EXCEEDED,
351             "This client may place orders up to " + limit.toPlainString()
352             + " USD; this one is " + notional.toPlainString()
353             .with("tradingLimit", limit)));
354     return new Assessment(instrument, price, notional, commission, total, buyingPower, issues);
355 }
356
357 private static void checkPrices(OrderRequest request, List<ApiException> issues) {
358     Type type = request.getType();
359     boolean needsLimit = type == Type.LIMIT || type == Type.STOP_LIMIT;
360     boolean needsStop = type == Type.STOP || type == Type.STOP_LIMIT;
361     if (needsLimit != (request.getLimitPrice() != null)) {
362         issues.add(new ApiException(ProblemType.INVALID_REQUEST, needsLimit
363             ? "limitPrice is required for " + type + " orders"
364             : "limitPrice is not allowed for " + type + " orders"));
365     }
366     if (needsStop != (request.getStopPrice() != null)) {
367         issues.add(new ApiException(ProblemType.INVALID_REQUEST, needsStop
368             ? "stopPrice is required for " + type + " orders"
369             : "stopPrice is not allowed for " + type + " orders"));
370     }
371 }
372
373 /** The price used to estimate cost and reserve cash: the worst price the order can get. */
374 private BigDecimal referencePrice(Type type, Side side, BigDecimal limit, BigDecimal stop,

```

```

375         Instrument instrument) {
376     return switch (type) {
377         case LIMIT, STOP_LIMIT -> limit;
378         case STOP -> stop;
379         case MARKET -> {
380             MarketData.Tick tick = market.tick(instrument.id());
381             yield side == Side.BUY ? tick.ask() : tick.bid();
382         }
383     };
384 }
385
386 private static BigDecimal commission(Instrument instrument, BigDecimal quantity) {
387     BigDecimal perUnit = instrument.isOption() ? OPTION_COMMISSION : BigDecimal.ZERO;
388     return perUnit.multiply(quantity).setScale(2, RoundingMode.HALF_EVEN);
389 }
390
391 private static <T> T orElse(T value, T fallback) {
392     return value != null ? value : fallback;
393 }
394
395 private void save(Order order) {
396     orders.put(order.id(), order);
397 }
398
399 private String nextOrderId() {
400     return "ORD-" + orderIds.incrementAndGet();
401 }
402 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/TradingSession.java

31 lines

```

1 package com.example.brokerage.api.orders;
2
3 import com.example.brokerage.api.market.MarketData;
4
5 import org.springframework.boot.autoconfigure.condition.ConditionalOnBooleanProperty;
6 import org.springframework.scheduling.annotation.Scheduled;
7 import org.springframework.stereotype.Component;
8
9 /**
10  * Drives the simulated market: every tick, prices move and open orders are matched.
11  * Tests switch this off (brokerage.simulator.enabled=false) and call match() directly,
12  * so that every price is known in advance.
13  */
14 @Component
15 @ConditionalOnBooleanProperty(name = "brokerage.simulator.enabled", matchIfMissing = true)
16 class TradingSession {
17
18     private final MarketData market;
19     private final OrderService orders;
20
21     TradingSession(MarketData market, OrderService orders) {
22         this.market = market;
23         this.orders = orders;
24     }
25
26     @Scheduled(fixedDelayString = "${brokerage.simulator.tick-millis:1000}")
27     void tick() {
28         market.move();
29         orders.match();
30     }
31 }

```

brokerage-api/src/main/resources/application.yaml

34 lines

```

1 server:
2   port: 8080
3
4 spring:

```

```

5  application:
6    name: brokerage-api
7  jackson:
8    # Absent optional fields are omitted rather than sent as null.
9    default-property-inclusion: non_null
10 security:
11   oauth2:
12     resourceserver:
13       jwt:
14         issuer-uri: http://localhost:9000
15         # Tokens must be minted for this API, not merely by a trusted issuer.
16         audiences: brokerage-api
17
18 brokerage:
19   simulator:
20     enabled: true
21     tick-millis: 1000
22   rate-limit:
23     requests-per-minute: 120
24     order-changes-per-minute: 30
25
26 management:
27   endpoints:
28     web:
29       exposure:
30         include: health
31
32 logging:
33   pattern:
34     level: "%5p [%X{requestId:-}]"

```

E.4 The API's tests

brokerage-api/src/test: protocol, hypermedia, lifecycle, rate limit and contract tests.

brokerage-api/src/test/java/com/example/brokerage/api/ApiTest.java

64 lines

```

1  package com.example.brokerage.api;
2
3  import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.jwt;
4
5  import java.lang.annotation.ElementType;
6  import java.lang.annotation.Retention;
7  import java.lang.annotation.RetentionPolicy;
8  import java.lang.annotation.Target;
9  import java.util.ArrayList;
10 import java.util.List;
11
12 import org.springframework.boot.test.context.SpringBootTest;
13 import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
14 import org.springframework.test.web.servlet.request.RequestPostProcessor;
15
16 /**
17  * The whole application, with the simulated market frozen: prices never move by
18  * themselves, and orders fill only when a test calls OrderService.match().
19  */
20 @Target(ElementType.TYPE)
21 @Retention(RetentionPolicy.RUNTIME)
22 @SpringBootTest(properties = "brokerage.simulator.enabled=false")
23 @AutoConfigureMockMvc
24 public @interface ApiTest {
25
26     /** Access tokens for tests, as the authorization server would issue them. */
27     final class Tokens {
28
29         public static final String MARKET_DATA = "market-data:read";
30         public static final String ACCOUNTS = "accounts:read";
31     }
32 }

```

```

31     public static final String ORDERS_READ = "orders:read";
32     public static final String ORDERS_WRITE = "orders:write";
33     public static final String[] ALL = { MARKET_DATA, ACCOUNTS, ORDERS_READ, ORDERS_WRITE };
34
35     private Tokens() {
36     }
37
38     /** Alice (customer cust-1001) through her own script, the brokerage-cli client. */
39     public static RequestPostProcessor alice(String... scopes) {
40         return token("brokerage-cli", "cust-1001", null, scopes);
41     }
42
43     /** Alice's AI agent: same customer, orders capped at the given notional. */
44     public static RequestPostProcessor agent(String tradingLimit, String... scopes) {
45         return token("brokerage-agent", "cust-1001", tradingLimit, scopes);
46     }
47
48     /** Bob (customer cust-2002), who must never see Alice's accounts. */
49     public static RequestPostProcessor bob(String... scopes) {
50         return token("bob-app", "cust-2002", null, scopes);
51     }
52
53     private static RequestPostProcessor token(String client, String customer, String limit,
54         String... scopes) {
55         return jwt().jwt(token -> {
56             token.subject(client).audience(List.of("brokerage-api")).claim("customer_id", customer)
57                 .claim("scope", new ArrayList<>(List.of(scopes)));
58             if (limit != null) {
59                 token.claim("trading_limit", limit);
60             }
61         });
62     }
63 }
64 }

```

brokerage-api/src/test/java/com/example/brokerage/api/ContractTest.java

149 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4  import static com.example.brokerage.api.ApiTest.Tokens.alice;
5  import static org.assertj.core.api.Assertions.assertThat;
6
7  import java.io.IOException;
8  import java.io.InputStream;
9  import java.util.List;
10 import java.util.Map;
11 import java.util.UUID;
12
13 import com.networknt.schema.Error;
14 import com.networknt.schema.SchemaLocation;
15 import com.networknt.schema.SchemaRegistry;
16 import com.networknt.schema.SpecificationVersion;
17 import tools.jackson.databind.JsonNode;
18 import tools.jackson.databind.json.JsonMapper;
19 import tools.jackson.databind.node.ObjectNode;
20 import tools.jackson.dataformat.yaml.YAMLMapper;
21
22 import org.junit.jupiter.api.BeforeAll;
23 import org.junit.jupiter.api.Test;
24 import org.springframework.beans.factory.annotation.Autowired;
25 import org.springframework.core.io.ClassPathResource;
26 import org.springframework.http.MediaType;
27 import org.springframework.test.web.servlet.assertj.MockMvcTester;
28 import org.springframework.test.web.servlet.assertj.MvcTestResult;
29
30 /**
31  * The OpenAPI document is a contract, so it is tested like code: it must be a valid
32  * OpenAPI 3.2 document, its examples must match its own schemas, and real responses from

```

```

33  * the running application must match the schemas it promises.
34  */
35  @ApiTest
36  class ContractTest {
37
38      static final String CONTRACT = "https://api.brokerage.example/openapi.yaml";
39      static final String OAS_32 = "https://spec.openapis.org/oas/3.2/schema/2025-09-17";
40
41      static JsonNode contract;
42      static SchemaRegistry schemas;
43
44      @Autowired
45      MockMvcTester mvc;
46
47      @BeforeAll
48      static void loadTheContract() throws IOException {
49          try (InputStream yaml = new ClassPathResource("contract/brokerage-api.yaml").getInputStream();
50               InputStream meta = new ClassPathResource("oas-3.2-schema.json").getInputStream()) {
51              contract = YAMLMapper.builder().build().readTree(yaml);
52              String metaSchema = new String(meta.readAllBytes());
53              String contractJson = JsonMapper.builder().build().writeValueAsString(contract);
54              schemas = SchemaRegistry.withDefaultDialect(SpecificationVersion.DRAFT_2020_12,
55                  registry -> registry.schemas(Map.of(CONTRACT, contractJson, OAS_32, metaSchema)));
56          }
57      }
58
59      @Test
60      void theContractIsAValidOpenApi32Document() {
61          List<Error> errors = schemas.getSchema(SchemaLocation.of(OAS_32)).validate(contract);
62          assertThat(errors).as("violations of the OpenAPI 3.2 meta-schema").isEmpty();
63      }
64
65      @Test
66      void theMetaSchemaCheckCatchesABrokenContract() {
67          ObjectNode broken = (ObjectNode) contract.deepCopy();
68          broken.remove("info");
69          broken.put("openapi", "3.2");
70          assertThat(schemas.getSchema(SchemaLocation.of(OAS_32)).validate(broken)).isNotEmpty();
71      }
72
73      @Test
74      void everyExampleMatchesItsOwnSchema() {
75          JsonNode placeOrder = contract.at("/paths/~1accounts~1{accountId}~1orders/post/requestBody/content/"
76              + "application~1json/examples/limitBuy/dataValue");
77          assertMatches("OrderRequest", placeOrder);
78          for (String response : List.of("BadRequest", "UnprocessableContent")) {
79              contract.at("/components/responses/" + response + "/content/application~1problem+json/examples")
80                  .forEach(example -> assertMatches("Problem", example.get("dataValue")));
81          }
82      }
83
84      @Test
85      void readOnlyResourcesMatchTheirSchemas() throws Exception {
86          assertMatches("ApiRoot", get("/"));
87          assertMatches("AccountCollection", get("/accounts"));
88          assertMatches("Account", get("/accounts/ACC-1001"));
89          assertMatches("Balances", get("/accounts/ACC-1001/balances"));
90          assertMatches("PositionCollection", get("/accounts/ACC-1001/positions"));
91          assertMatches("Position", get("/accounts/ACC-1001/positions/EQ-AAPL"));
92          assertMatches("TransactionPage", get("/accounts/ACC-1001/transactions?limit=5"));
93          assertMatches("InstrumentCollection", get("/instruments?type=EQUITY"));
94          assertMatches("InstrumentCollection", get("/instruments/EQ-AAPL/option-chain"));
95          assertMatches("QuoteV1", get("/instruments/EQ-AAPL/quote"));
96          assertMatches("OrderPage", get("/accounts/ACC-1001/orders"));
97          assertMatches("ProtectedResourceMetadata", get("/.well-known/oauth-protected-resource"));
98      }
99
100      @Test
101      void ordersPreviewsAndProblemsMatchTheirSchemas() throws Exception {

```

```

102     MvcTestResult placed = mvc.post().uri("/accounts/ACC-1001/orders").with(alice(ALL))
103         .header("Idempotency-Key", UUID.randomUUID().toString())
104         .contentType(MediaType.APPLICATION_JSON)
105         .content("""
106             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "LIMIT", "quantity": "1",
107              "limitPrice": "200.00", "clientOrderId": "contract-test"}""")
108         .exchange();
109     assertMatches("Order", json(placed));
110
111     MvcTestResult preview = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
112         .contentType(MediaType.APPLICATION_JSON)
113         .content("""
114             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "3"}""")
115         .exchange();
116     assertMatches("OrderPreview", json(preview));
117
118     MvcTestResult problem = mvc.post().uri("/accounts/ACC-1001/orders").with(alice(ALL))
119         .header("Idempotency-Key", UUID.randomUUID().toString())
120         .contentType(MediaType.APPLICATION_JSON)
121         .content("""
122             {"instrumentId": "EQ-HALT", "side": "BUY", "type": "MARKET", "quantity": "1"}""")
123         .exchange();
124     assertThat(problem).hasStatus(422);
125     assertMatches("Problem", json(problem));
126
127     MvcTestResult v2 = mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "2")
128         .with(alice(ALL)).exchange();
129     assertMatches("Quote", json(v2));
130 }
131
132 // ----- helpers
133
134 private JsonNode get(String uri) throws Exception {
135     MvcTestResult result = mvc.get().uri(uri).with(alice(ALL)).exchange();
136     assertThat(result).as(uri).hasStatusOk();
137     return json(result);
138 }
139
140 private static JsonNode json(MvcTestResult result) throws Exception {
141     return JsonMapper.builder().build().readTree(result.getResponse().getContentAsString());
142 }
143
144 private static void assertMatches(String schemaName, JsonNode instance) {
145     SchemaLocation location = SchemaLocation.of(CONTRACT + "#/components/schemas/" + schemaName);
146     List<Error> errors = schemas.getSchema(location).validate(instance);
147     assertThat(errors).as("%s does not match the contract:%n%s", schemaName, instance).isEmpty();
148 }
149 }

```

brokerage-api/src/test/java/com/example/brokerage/api/HypermediaTest.java

98 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ACCOUNTS;
4  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
5  import static com.example.brokerage.api.ApiTest.Tokens.ORDERS_READ;
6  import static com.example.brokerage.api.ApiTest.Tokens.alice;
7  import static com.example.brokerage.api.ApiTest.Tokens.bob;
8  import static org.assertj.core.api.Assertions.assertThat;
9
10 import org.junit.jupiter.api.Test;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.hateoas.MediaType;
13 import org.springframework.http.HttpStatus;
14 import org.springframework.test.web.servlet.assertj.MockMvcTester;
15 import org.springframework.test.web.servlet.assertj.MvcTestResult;
16
17 /**
18  * Discovery, link relations and the security boundary: what a client can learn, and

```

```

19  * what it is allowed to see.
20  */
21  @ApiTest
22  class HypermediaTest {
23
24      static final String PROBLEMS = "https://docs.brokerage.example/problems/";
25
26      @Autowired
27      MockMvcTester mvc;
28
29      @Test
30      void theRootIsPublicAndLinksToEverything() {
31          MvcTestResult root = mvc.get().uri("/").exchange();
32          assertThat(root).hasStatusOk().hasContentType(MediaType.HAL_JSON);
33          assertThat(root).bodyJson().extractingPath("$. _links['bk:accounts'].href").asString()
34              .endsWith("/accounts");
35          assertThat(root).bodyJson().extractingPath("$. _links['bk:instruments'].templated").isEqualTo(true);
36          assertThat(root).bodyJson().extractingPath("$. _links['service-desc'].href").asString()
37              .endsWith("/openapi.yaml");
38      }
39
40      @Test
41      void theContractIsServedBesideTheApi() {
42          assertThat(mvc.get().uri("/openapi.yaml")).hasStatusOk().bodyText().contains("openapi: 3.2.1");
43      }
44
45      @Test
46      void protectedResourceMetadataNamesTheAuthorizationServer() {
47          assertThat(mvc.get().uri("/.well-known/oauth-protected-resource")).hasStatusOk()
48              .bodyJson().extractingPath("$.authorization_servers[0]").isEqualTo("http://localhost:9000");
49      }
50
51      @Test
52      void aRequestWithoutATokenGetsAProblemAndAChallenge() {
53          MvcTestResult result = mvc.get().uri("/accounts").exchange();
54          assertThat(result).hasStatus(HttpStatus.UNAUTHORIZED).hasContentType("application/problem+json");
55          assertThat(result.getResponse().getHeader("WWW-Authenticate")).startsWith("Bearer")
56              .contains("resource_metadata=");
57          assertThat(result).bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "unauthorized");
58      }
59
60      @Test
61      void aTokenWithoutTheScopeIsForbidden() {
62          assertThat(mvc.get().uri("/accounts").with(alice(ORDERS_READ)))
63              .hasStatus(HttpStatus.FORBIDDEN)
64              .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "insufficient-scope");
65      }
66
67      @Test
68      void anotherCustomersAccountDoesNotExist() {
69          assertThat(mvc.get().uri("/accounts/ACC-1001").with(bob(ALL)))
70              .hasStatus(HttpStatus.NOT_FOUND)
71              .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "not-found");
72      }
73
74      @Test
75      void linksReflectWhatTheTokenMayDo() {
76          // Read-only: accounts are visible, but orders are not even mentioned.
77          MvcTestResult readOnly = mvc.get().uri("/accounts/ACC-1001").with(alice(ACCOUNTS)).exchange();
78          assertThat(readOnly).hasStatusOk();
79          assertThat(readOnly).bodyJson()
80              .hasPath("$. _links['bk:positions']")
81              .doesNotHavePath("$. _links['bk:orders']");
82
83          // A full trading token, asking for HAL-FORMS: the placeOrder template appears.
84          MvcTestResult trading = mvc.get().uri("/accounts/ACC-1001").with(alice(ALL))
85              .accept(MediaType.HAL_FORMS_JSON).exchange();
86          assertThat(trading).hasStatusOk();
87          assertThat(trading).bodyJson().hasPath("$. _links['bk:orders']");

```

```

88         assertThat(trading).bodyJson().extractingPath("$.templates.placeOrder.method").isEqualTo("POST");
89     }
90
91     @Test
92     void anOptionChainOffersEveryExpirationAsANamedLink() {
93         MvcTestResult chain = mvc.get().uri("/instruments/EQ-AAPL/option-chain").with(alice(ALL)).exchange();
94         assertThat(chain).hasStatusOk();
95         assertThat(chain).bodyJson().extractingPath("$.links['bk:expiration']").asArray().hasSize(2);
96         assertThat(chain).bodyJson().extractingPath("$.embedded['bk:instruments']").asArray().hasSize(6);
97     }
98 }

```

brokerage-api/src/test/java/com/example/brokerage/api/OrderLifecycleTest.java

183 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4  import static com.example.brokerage.api.ApiTest.Tokens.agent;
5  import static com.example.brokerage.api.ApiTest.Tokens.alice;
6  import static org.assertj.core.api.Assertions.assertThat;
7
8  import java.util.UUID;
9
10 import com.example.brokerage.api.orders.OrderService;
11
12 import org.junit.jupiter.api.Test;
13 import org.springframework.beans.factory.annotation.Autowired;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.MediaType;
16 import org.springframework.test.web.servlet.assertj.MockMvcTester;
17 import org.springframework.test.web.servlet.assertj.MvcTestResult;
18
19 /**
20  * Placing, retrying, changing and cancelling orders, and the links that follow each state.
21  */
22 @ApiTest
23 class OrderLifecycleTest {
24
25     static final String ORDERS = "/accounts/ACC-1001/orders";
26     static final String PROBLEMS = "https://docs.brokerage.example/problems/";
27     static final MediaType MERGE_PATCH = MediaType.parseMediaType("application/merge-patch+json");
28
29     @Autowired
30     MockMvcTester mvc;
31
32     @Autowired
33     OrderService orders;
34
35     @Test
36     void aMarketOrderIsCreatedThenFilledAndLosesItsCancelLink() {
37         MvcTestResult placed = place(UUID.randomUUID().toString(), ""
38             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "2"}""");
39         assertThat(placed).hasStatus(HttpStatus.CREATED);
40         assertThat(placed.getResponse().getHeader("Location")).contains(ORDERS + "/ORD-");
41         assertThat(placed.getResponse().getHeader("ETag")).isNotBlank();
42         assertThat(placed).bodyJson().extractingPath("$.status").isEqualTo("OPEN");
43         assertThat(placed).bodyJson().hasPath("$.links['bk:cancel']").hasPath("$.links.edit");
44
45         orders.match();
46
47         String url = placed.getResponse().getHeader("Location");
48         MvcTestResult filled = mvc.get().uri(url).with(alice(ALL)).exchange();
49         assertThat(filled).bodyJson().extractingPath("$.status").isEqualTo("FILLED");
50         // AAPL is frozen at 232.40, so the ask is 232.42.
51         assertThat(filled).bodyJson().extractingPath("$.averageFillPrice").isEqualTo("232.4200");
52         assertThat(filled).bodyJson()
53             .doesNotHavePath("$.links['bk:cancel']")
54             .doesNotHavePath("$.links.edit");
55     }

```

```

56
57 @Test
58 void retryingWithTheSameKeyReturnsTheSameOrder() {
59     String key = UUID.randomUUID().toString();
60     String body = ""
61         {"instrumentId": "EQ-MSFT", "side": "BUY", "type": "LIMIT", "quantity": "1",
62          "limitPrice": "400.00"}"";
63     MvcTestResult first = place(key, body);
64     MvcTestResult retry = place(key, body);
65
66     assertThat(retry).hasStatus(HttpStatus.CREATED);
67     assertThat(retry.getResponse().getHeader("Location"))
68         .isEqualTo(first.getResponse().getHeader("Location"));
69     assertThat(retry.getResponse().getHeader("Idempotent-Replayed")).isEqualTo("true");
70
71     MvcTestResult different = place(key, body.replace("\1", "\2"));
72     assertThat(different).hasStatus(HttpStatus.UNPROCESSABLE_CONTENT)
73         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "idempotency-key-reused");
74 }
75
76 @Test
77 void anOrderWithoutAnIdempotencyKeyIsRefused() {
78     assertThat(mvc.post().uri(ORDERS).with(alice(ALL)).contentType(MediaType.APPLICATION_JSON)
79         .content("""
80             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "1"}"""))
81         .hasStatus(HttpStatus.BAD_REQUEST)
82         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "idempotency-key-missing");
83 }
84
85 @Test
86 void businessRuleViolationsAre422WithTheDetailsAClientNeeds() {
87     MvcTestResult tooBig = place(UUID.randomUUID().toString(), ""
88         {"instrumentId": "EQ-MSFT", "side": "BUY", "type": "LIMIT", "quantity": "1000",
89          "limitPrice": "500"}""");
90     assertThat(tooBig).hasStatus(HttpStatus.UNPROCESSABLE_CONTENT);
91     assertThat(tooBig.bodyJson().extractingPath("$.type"))
92         .isEqualTo(PROBLEMS + "insufficient-buying-power");
93     assertThat(tooBig.bodyJson().extractingPath("$.required")).isEqualTo("500000.00");
94     assertThat(tooBig.bodyJson().hasPath("$.available"));
95 }
96
97 @Test
98 void theAgentsDelegatedTradingLimitIsEnforcedByTheApi() {
99     // 30 x 232.42 = 6,972.60, above the agent's 5,000 limit.
100     assertThat(mvc.post().uri(ORDERS).with(agent("5000", ALL))
101         .header("Idempotency-Key", UUID.randomUUID().toString())
102         .contentType(MediaType.APPLICATION_JSON)
103         .content("""
104             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "30"}"""))
105         .hasStatus(HttpStatus.FORBIDDEN)
106         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "trading-limit-exceeded");
107 }
108
109 @Test
110 void changingAnOrderRequiresItsCurrentEtag() {
111     MvcTestResult placed = place(UUID.randomUUID().toString(), ""
112         {"instrumentId": "EQ-NVDA", "side": "BUY", "type": "LIMIT", "quantity": "5",
113          "limitPrice": "150.00"}""");
114     String url = placed.getResponse().getHeader("Location");
115     String etag = placed.getResponse().getHeader("ETag");
116
117     // Polling an unchanged order costs a 304 and no body.
118     assertThat(mvc.get().uri(url).with(alice(ALL)).header("If-None-Match", etag))
119         .hasStatus(HttpStatus.NOT_MODIFIED);
120
121     assertThat(amend(url, null, ""
122         {"limitPrice": "151.00"}"")).hasStatus(HttpStatus.PRECONDITION_REQUIRED);
123
124     MvcTestResult changed = amend(url, etag, ""

```

```

125         {"limitPrice": "151.00"}""");
126     assertThat(changed).hasStatusOk();
127     assertThat(changed).bodyJson().extractingPath("$.limitPrice").isEqualTo("151.00");
128     assertThat(changed.getResponse().getHeader("ETag")).isNotEqualTo(etag);
129
130     // The old ETag is now stale: a second writer holding it is refused, not merged.
131     assertThat(amend(url, etag, ""
132         {"limitPrice": "152.00"}"""))
133         .hasStatus(HttpStatus.PRECONDITION_FAILED)
134         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "precondition-failed");
135 }
136
137 @Test
138 void cancellationIsAcceptedThenCompletes() {
139     MvcTestResult placed = place(UUID.randomUUID().toString(), ""
140         {"instrumentId": "ETF-SPY", "side": "SELL", "type": "LIMIT", "quantity": "1",
141         "limitPrice": "900.00"}""");
142     String url = placed.getResponse().getHeader("Location");
143
144     MvcTestResult cancel = mvc.post().uri(url + "/cancellation").with(alice(ALL)).exchange();
145     assertThat(cancel).hasStatus(HttpStatus.ACCEPTED);
146     assertThat(cancel.getResponse().getHeader("Location")).isEqualTo(url);
147     assertThat(cancel.bodyJson().extractingPath("$.status").isEqualTo("PENDING_CANCEL");
148
149     orders.match();
150
151     assertThat(mvc.get().uri(url).with(alice(ALL)))
152         .bodyJson().extractingPath("$.status").isEqualTo("CANCELLED");
153     assertThat(mvc.post().uri(url + "/cancellation").with(alice(ALL)))
154         .hasStatus(HttpStatus.CONFLICT)
155         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "order-not-cancellable");
156 }
157
158 @Test
159 void aPreviewReportsProblemsWithoutPlacingAnything() {
160     MvcTestResult preview = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
161         .contentType(MediaType.APPLICATION_JSON)
162         .content("""
163             {"instrumentId": "EQ-AMZN", "side": "SELL", "type": "MARKET", "quantity": "500"}""")
164         .exchange();
165     assertThat(preview).hasStatusOk();
166     assertThat(preview.bodyJson().extractingPath("$.acceptable").isEqualTo(false);
167     assertThat(preview.bodyJson().extractingPath("$.issues[0].type")
168         .isEqualTo(PROBLEMS + "insufficient-position");
169     assertThat(preview.bodyJson().doesNotHavePath("$.links['bk:place-order']");
170 }
171
172 // ----- helpers
173
174 private MvcTestResult place(String idempotencyKey, String json) {
175     return mvc.post().uri(ORDERS).with(alice(ALL)).header("Idempotency-Key", idempotencyKey)
176         .contentType(MediaType.APPLICATION_JSON).content(json).exchange();
177 }
178
179 private MvcTestResult amend(String url, String ifMatch, String json) {
180     var request = mvc.patch().uri(url).with(alice(ALL)).contentType(MERGE_PATCH).content(json);
181     return (ifMatch == null ? request : request.header("If-Match", ifMatch)).exchange();
182 }
183 }

```

brokerage-api/src/test/java/com/example/brokerage/api/ProtocolTest.java

107 lines

```

1 package com.example.brokerage.api;
2
3 import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4 import static com.example.brokerage.api.ApiTest.Tokens.alice;
5 import static org.assertj.core.api.Assertions.assertThat;
6
7 import java.util.List;

```

```

8
9 import com.example.brokerage.api.orders.OrderEvents;
10 import com.jayway.jsonpath.JsonPath;
11
12 import org.junit.jupiter.api.Test;
13 import org.springframework.beans.factory.annotation.Autowired;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.test.web.servlet.assertj.MockMvcTester;
16 import org.springframework.test.web.servlet.assertj.MvcTestResult;
17
18 /**
19  * Pagination, versioning, caching and event replay.
20  */
21 @ApiTest
22 class ProtocolTest {
23
24     static final String PROBLEMS = "https://docs.brokerage.example/problems/";
25
26     @Autowired
27     MockMvcTester mvc;
28
29     @Autowired
30     OrderEvents events;
31
32     @Test
33     void transactionsArePagedWithAnOpaqueCursor() throws Exception {
34         MvcTestResult first = mvc.get().uri("/accounts/ACC-1001/transactions?type=TRADE&limit=2")
35             .with(alice(ALL)).exchange();
36         assertThat(first).hasStatusOk();
37         List<String> firstIds = ids(first);
38         assertThat(firstIds).hasSize(2);
39
40         String next = JsonPath.read(first.getResponse().getContentAsString(), "$._links.next.href");
41         assertThat(next).contains("cursor=").doesNotContain("{}");
42
43         MvcTestResult second = mvc.get().uri(next).with(alice(ALL)).exchange();
44         assertThat(ids(second)).doesNotContainAnyElementsOf(firstIds);
45     }
46
47     @Test
48     void aTamperedCursorIsABadRequest() {
49         assertThat(mvc.get().uri("/accounts/ACC-1001/transactions?cursor=not-a-cursor").with(alice(ALL)))
50             .hasStatus(HttpStatus.BAD_REQUEST)
51             .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "invalid-request");
52     }
53
54     @Test
55     void versionOneQuotesAreFlatAndDeprecated() {
56         MvcTestResult v1 = mvc.get().uri("/instruments/EQ-AAPL/quote").with(alice(ALL)).exchange();
57         assertThat(v1).hasStatusOk();
58         assertThat(v1.bodyJson().extractingPath("$.bid").isEqualTo("232.38"));
59         assertThat(v1.getResponse().getHeader("Deprecation")).startsWith("@");
60         assertThat(v1.getResponse().getHeader("Sunset")).contains("2027");
61         assertThat(v1.getResponse().getHeader("Cache-Control")).contains("max-age=1").contains("private");
62     }
63
64     @Test
65     void versionTwoQuotesNestPriceLevels() {
66         MvcTestResult v2 = mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "2")
67             .with(alice(ALL)).exchange();
68         assertThat(v2).hasStatusOk();
69         assertThat(v2.bodyJson().extractingPath("$.bid.price").isEqualTo("232.38"));
70         assertThat(v2.bodyJson().extractingPath("$.changePercent").isEqualTo("1.00"));
71         assertThat(v2.getResponse().getHeader("Deprecation")).isNull();
72     }
73
74     @Test
75     void anUnknownVersionIsRefusedWithTheSupportedOnes() {
76         assertThat(mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "7").with(alice(ALL)))

```

```

77         .hasStatus(HttpStatus.BAD_REQUEST)
78         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "unsupported-api-version");
79     }
80
81     @Test
82     void anUnchangedQuoteCostsA304() {
83         MvcTestResult quote = mvc.get().uri("/instruments/EQ-MSFT/quote").with(alice(ALL)).exchange();
84         assertThat(mvc.get().uri("/instruments/EQ-MSFT/quote").with(alice(ALL))
85             .header("If-None-Match", quote.getResponse().getHeader("ETag")))
86             .hasStatus(HttpStatus.NOT_MODIFIED);
87     }
88
89     @Test
90     void aReconnectingClientReceivesOnlyTheEventsItMissed() {
91         mvc.post().uri("/accounts/ACC-1002/orders").with(alice(ALL)).header("Idempotency-Key", "events-1")
92             .contentType("application/json")
93             .content("""
94                 {"instrumentId": "EQ-NVDA", "side": "SELL", "type": "LIMIT", "quantity": "1",
95                  "limitPrice": "999"}""")
96             .exchange();
97         List<OrderEvents.Event> all = events.replay("ACC-1002", 0);
98         assertThat(all).isNotEmpty();
99         long last = all.getLast().id();
100        assertThat(events.replay("ACC-1002", last)).isEmpty();
101        assertThat(events.replay("ACC-1002", last - 1)).hasSize(1);
102    }
103
104    private static List<String> ids(MvcTestResult page) throws Exception {
105        return JsonPath.read(page.getResponse().getContentAsString(), "$._embedded['bk:transactions'][*].id");
106    }
107 }

```

brokerage-api/src/test/java/com/example/brokerage/api/RateLimitTest.java

42 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4  import static com.example.brokerage.api.ApiTest.Tokens.alice;
5  import static org.assertj.core.api.Assertions.assertThat;
6
7  import org.junit.jupiter.api.Test;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.boot.test.context.SpringBootTest;
10 import org.springframework.boot.test.autoconfigure.AutoConfigureMockMvc;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.test.web.servlet.assertj.MockMvcTester;
13 import org.springframework.test.web.servlet.assertj.MvcTestResult;
14
15 /**
16  * Quotas are advertised on every response, and enforced with 429 and Retry-After.
17  */
18 @SpringBootTest(properties = {
19     "brokerage.simulator.enabled=false",
20     "brokerage.rate-limit.requests-per-minute=3" })
21 @AutoConfigureMockMvc
22 class RateLimitTest {
23
24     @Autowired
25     MockMvcTester mvc;
26
27     @Test
28     void aClientOverItsQuotaIsToldWhenToComeBack() {
29         MvcTestResult first = mvc.get().uri("/instruments").with(alice(ALL)).exchange();
30         assertThat(first.getResponse().getHeader("RateLimit-Policy")).isEqualTo("\"default\";q=3;w=60");
31         assertThat(first.getResponse().getHeader("RateLimit")).startsWith("\"default\";r=2;");
32
33         mvc.get().uri("/instruments").with(alice(ALL)).exchange();
34         mvc.get().uri("/instruments").with(alice(ALL)).exchange();
35         MvcTestResult refused = mvc.get().uri("/instruments").with(alice(ALL)).exchange();

```

```

36
37     assertThat(refused).hasStatus(HttpStatus.TOO_MANY_REQUESTS);
38     assertThat(refused.getResponse().getHeader("Retry-After")).isNotBlank();
39     assertThat(refused.bodyJson().extractingPath("$.type")
40         .isEqualTo("https://docs.brokerage.example/problems/rate-limited"));
41 }
42 }

```

E.5 The hypermedia client

brokerage-client: a Java client that follows links.

brokerage-client/src/main/java/com/example/brokerage/client/ApiProblem.java

59 lines

```

1 package com.example.brokerage.client;
2
3 import java.io.IOException;
4 import java.util.Optional;
5
6 import tools.jackson.databind.JsonNode;
7 import tools.jackson.databind.json.JsonMapper;
8
9 import org.springframework.http.client.ClientHttpResponse;
10
11 /**
12  * An RFC 9457 problem returned by the API, as an exception. Callers branch on {@link #type()},
13  * the stable identifier, and may show {@link #detail()} to a person.
14  */
15 public class ApiProblem extends RuntimeException {
16
17     private final int status;
18     private final String type;
19     private final String title;
20     private final JsonNode body;
21     private final Optional<Long> retryAfterSeconds;
22
23     ApiProblem(int status, String type, String title, String detail, JsonNode body,
24         Optional<Long> retryAfter) {
25         super(status + " " + title + (detail == null ? "" : ": " + detail));
26         this.status = status;
27         this.type = type;
28         this.title = title;
29         this.body = body;
30         this.retryAfterSeconds = retryAfter;
31     }
32
33     static ApiProblem from(ClientHttpResponse response, JsonMapper json) throws IOException {
34         JsonNode body = json.readTree(response.getBody());
35         int status = response.getStatusCode().value();
36         Optional<Long> retryAfter = Optional.ofNullable(response.getHeaders().getFirst("Retry-After"))
37             .map(Long::valueOf);
38         return new ApiProblem(status, body.path("type").asString("about:blank"),
39             body.path("title").asString(response.getStatusText()), body.path("detail").asString(null),
40             body, retryAfter);
41     }
42
43     /** Worth retrying later with the same request (and the same Idempotency-Key). */
44     public boolean isTransient() {
45         return status == 429 || status == 502 || status == 503 || status == 504;
46     }
47
48     /** The problem type's short name, such as "insufficient-buying-power". */
49     public String typeName() {
50         return type.substring(type.lastIndexOf('/') + 1);
51     }
52
53     public int status() { return status; }

```

```

54     public String type() { return type; }
55     public String title() { return title; }
56     public String detail() { return body.path("detail").asString(null); }
57     public JsonNode body() { return body; }
58     public Optional<Long> retryAfterSeconds() { return retryAfterSeconds; }
59 }

```

brokerage-client/src/main/java/com/example/brokerage/client/BrokerageClient.java

145 lines

```

1  package com.example.brokerage.client;
2
3  import java.net.URI;
4  import java.time.Duration;
5  import java.util.List;
6  import java.util.Map;
7
8  import com.example.brokerage.client.Model.Account;
9  import com.example.brokerage.client.Model.Balances;
10 import com.example.brokerage.client.Model.Instrument;
11 import com.example.brokerage.client.Model.Order;
12 import com.example.brokerage.client.Model.OrderPreview;
13 import com.example.brokerage.client.Model.OrderTicket;
14 import com.example.brokerage.client.Model.Position;
15 import com.example.brokerage.client.Model.Quote;
16
17 import org.springframework.hateoas.Link;
18 import org.springframework.web.client.ResourceAccessException;
19
20 /**
21  * The brokerage, as a Java API. Every method starts from the API root and follows link
22  * relations; not one URL is built here. If the server moves a resource, this class keeps
23  * working, and if the server withholds a link, the corresponding action is unavailable.
24  */
25 public class BrokerageClient {
26
27     private static final int ATTEMPTS = 3;
28
29     private final HypermediaClient http;
30     private final URI root;
31
32     public BrokerageClient(HypermediaClient http, URI root) {
33         this.http = http;
34         this.root = root;
35     }
36
37     // ----- accounts
38
39     public List<Account> accounts() {
40         return accountResources().stream().map(account -> account.as(Account.class)).toList();
41     }
42
43     public Balances balances(String accountId) {
44         return follow(account(accountId), "bk:balances").as(Balances.class);
45     }
46
47     public List<Position> positions(String accountId) {
48         return follow(account(accountId), "bk:positions").embedded("bk:positions").stream()
49             .map(position -> position.as(Position.class)).toList();
50     }
51
52     // ----- market data
53
54     public Instrument instrument(String symbol) {
55         return instrumentResource(symbol).as(Instrument.class);
56     }
57
58     /** The latest quote, in the version 2 shape that the client pins on every request. */
59     public Quote quote(String symbol) {
60         Link quote = instrumentResource(symbol).requiredLink("bk:quote");

```

```

61     return http.get(quote.toUri()).as(Quote.class);
62 }
63
64 // ----- trading
65
66 public OrderPreview preview(String accountId, OrderTicket ticket) {
67     Link previews = account(accountId).requiredLink("bk:order-previews");
68     return http.post(previews, ticket, headers -> { }).as(OrderPreview.class);
69 }
70
71 /**
72  * Places an order. On a timeout or a transient error the request is retried with the
73  * same Idempotency-Key, so a retry can never place a second order.
74  */
75 public Order place(String accountId, OrderTicket ticket, String idempotencyKey) {
76     Link orders = account(accountId).requiredLink("bk:orders").expand();
77     for (int attempt = 1; ; attempt++) {
78         try {
79             return http.post(orders, ticket, headers -> headers.set("Idempotency-Key", idempotencyKey))
80                 .as(Order.class);
81         } catch (ResourceAccessException | ApiProblem failure) {
82             boolean retryable = !(failure instanceof ApiProblem problem) || problem.isTransient();
83             if (!retryable || attempt == ATTEMPTS) {
84                 throw failure;
85             }
86             long seconds = failure instanceof ApiProblem problem
87                 ? problem.retryAfterSeconds().orElse(1L) : attempt;
88             pause(seconds);
89         }
90     }
91 }
92
93 public List<Order> openOrders(String accountId) {
94     Link orders = account(accountId).requiredLink("bk:orders");
95     URI open = orders.expand(Map.of("status", List.of("OPEN", "PARTIALLY_FILLED"))).toUri();
96     return http.get(open).embedded("bk:orders").stream().map(order -> order.as(Order.class)).toList();
97 }
98
99 /** Re-reads an order through its own self link. */
100 public Order refresh(Order order) {
101     return http.get(order.self()).as(Order.class);
102 }
103
104 /** Cancels an order, if and only if the server still offers the cancel link. */
105 public Order cancel(Order order) {
106     URI cancel = order.link("bk:cancel").orElseThrow(() -> new IllegalStateException(
107         "Order " + order.id() + " is " + order.status() + " and can no longer be cancelled"));
108     return http.post(Link.of(cancel.toString()), null, headers -> { }).as(Order.class);
109 }
110
111 // ----- navigation
112
113 private List<HalResource> accountResources() {
114     HalResource start = http.get(root);
115     return follow(start, "bk:accounts").embedded("bk:accounts");
116 }
117
118 private HalResource account(String accountId) {
119     return accountResources().stream()
120         .filter(account -> accountId.equals(account.body().path("id").asString()))
121         .findFirst()
122         .orElseThrow(() -> new IllegalArgumentException(
123             "No account " + accountId + " for this customer"));
124 }
125
126 private HalResource instrumentResource(String symbol) {
127     Link search = http.get(root).requiredLink("bk:instruments");
128     return http.get(search.expand(Map.of("symbol", symbol)).toUri()).embedded("bk:instruments").stream()
129         .findFirst()

```

```

130         .orElseThrow(() -> new IllegalArgumentException("Unknown symbol " + symbol));
131     }
132
133     private HalResource follow(HalResource from, String rel) {
134         return http.get(from.requiredLink(rel).expand().toUri());
135     }
136
137     private static void pause(long seconds) {
138         try {
139             Thread.sleep(Duration.ofSeconds(Math.min(seconds, 5)));
140         } catch (InterruptedException interrupted) {
141             Thread.currentThread().interrupt();
142             throw new IllegalStateException(interrupted);
143         }
144     }
145 }

```

brokerage-client/src/main/java/com/example/brokerage/client/ClientCredentials.java

53 lines

```

1 package com.example.brokerage.client;
2
3 import java.net.URI;
4 import java.time.Instant;
5 import java.util.function.Supplier;
6
7 import tools.jackson.databind.JsonNode;
8
9 import org.springframework.http.MediaType;
10 import org.springframework.util.LinkedMultiValueMap;
11 import org.springframework.util.MultiValueMap;
12 import org.springframework.web.client.RestClient;
13
14 /**
15  * Access tokens by the OAuth 2.1 client credentials grant, cached until shortly before they
16  * expire. Enough for scripts and the demo; a Spring application should use Spring
17  * Security's OAuth 2.0 client instead, as the agent does.
18  */
19 public class ClientCredentials implements Supplier<String> {
20
21     private final RestClient http = RestClient.create();
22     private final URI tokenEndpoint;
23     private final String clientId;
24     private final String clientSecret;
25     private final String scope;
26     private String token;
27     private Instant expiresAt = Instant.EPOCH;
28
29     public ClientCredentials(URI tokenEndpoint, String clientId, String clientSecret, String scope) {
30         this.tokenEndpoint = tokenEndpoint;
31         this.clientId = clientId;
32         this.clientSecret = clientSecret;
33         this.scope = scope;
34     }
35
36     @Override
37     public synchronized String get() {
38         if (token == null || Instant.now().isAfter(expiresAt.minusSeconds(30))) {
39             MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
40             form.add("grant_type", "client_credentials");
41             form.add("scope", scope);
42             JsonNode response = http.post().uri(tokenEndpoint)
43                 .headers(headers -> headers.setBasicAuth(clientId, clientSecret))
44                 .contentType(MediaType.APPLICATION_FORM_URLENCODED)
45                 .body(form)
46                 .retrieve()
47                 .body(JsonNode.class);
48             token = response.get("access_token").asString();
49             expiresAt = Instant.now().plusSeconds(response.path("expires_in").asLong(60));
50         }
51     }
52 }

```

```

51         return token;
52     }
53 }

```

brokerage-client/src/main/java/com/example/brokerage/client/Demo.java

71 lines

```

1  package com.example.brokerage.client;
2
3  import java.math.BigDecimal;
4  import java.net.URI;
5  import java.util.UUID;
6
7  import com.example.brokerage.client.Model.Order;
8  import com.example.brokerage.client.Model.OrderTicket;
9
10 import org.springframework.web.client.RestClient;
11
12 /**
13  * A guided tour of the API through the hypermedia client. Start the authorization server
14  * and the API first, then run:
15  *
16  * <pre>
17  * ./mvnw -q -pl brokerage-client exec:java -Dexec.mainClass=com.example.brokerage.client.Demo
18  * </pre>
19  */
20 public final class Demo {
21
22     public static void main(String[] args) throws InterruptedException {
23         ClientCredentials tokens = new ClientCredentials(URI.create("http://localhost:9000/oauth2/token"),
24             "brokerage-cli", "cli-secret", "accounts:read market-data:read orders:read orders:write");
25         HypermediaClient http = new HypermediaClient(RestClient.builder()
26             .requestInterceptor((request, body, execution) -> {
27                 request.getHeaders().setBearerAuth(tokens.get());
28                 return execution.execute(request, body);
29             }));
30         BrokerageClient brokerage = new BrokerageClient(http, URI.create("http://localhost:8080/"));
31
32         System.out.println("Accounts");
33         brokerage.accounts().forEach(account -> System.out.printf(" %9s %-18s %s%n",
34             account.id(), account.nickname(), account.type()));
35
36         var balances = brokerage.balances("ACC-1001");
37         System.out.printf("%nACC-1001: cash %s, buying power %s, equity %s%n",
38             balances.cash(), balances.buyingPower(), balances.totalEquity());
39
40         System.out.println("\nPositions");
41         brokerage.positions("ACC-1001").forEach(p -> System.out.printf(" %5s %6s @ %-9s P&L %s%n",
42             p.symbol(), p.quantity(), p.lastPrice(), p.unrealizedPnL()));
43
44         var quote = brokerage.quote("AAPL");
45         System.out.printf("%nAAPL %s bid %s ask %s (%s%)%n",
46             quote.last(), quote.bid().price(), quote.ask().price(), quote.changePercent());
47
48         // A limit order well below the market, so that it rests and can be cancelled.
49         BigDecimal price = quote.last().multiply(new BigDecimal("0.90"))
50             .setScale(2, java.math.RoundingMode.DOWN);
51         OrderTicket ticket = OrderTicket.limit("EQ-AAPL", "BUY", new BigDecimal("5"), price);
52         var preview = brokerage.preview("ACC-1001", ticket);
53         System.out.printf("%nPreview: 5 AAPL at %s costs %s (acceptable: %s)%n",
54             price, preview.estimatedTotal(), preview.acceptable());
55
56         Order order = brokerage.place("ACC-1001", ticket, UUID.randomUUID().toString());
57         System.out.printf("Placed %s: %s, cancellable: %s%n", order.id(), order.status(),
58             order.isCancellable());
59
60         order = brokerage.cancel(order);
61         System.out.printf("Cancel requested: %s%n", order.status());
62         while (order.isCancellable() || "PENDING_CANCEL".equals(order.status())) {
63             Thread.sleep(500);

```

```

64         order = brokerage.refresh(order);
65     }
66     System.out.printf("Now %s, cancellable: %s\n", order.status(), order.isCancellable());
67 }
68
69 private Demo() {
70 }
71 }

```

brokerage-client/src/main/java/com/example/brokerage/client/HalResource.java

77 lines

```

1 package com.example.brokerage.client;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Optional;
6
7 import tools.jackson.databind.JsonNode;
8 import tools.jackson.databind.json.JsonMapper;
9
10 import org.springframework.hateoas.Link;
11 import org.springframework.http.HttpHeaders;
12
13 /**
14  * A HAL representation as a client sees it: properties, links by relation, embedded
15  * resources, and the response headers that came with it.
16  */
17 public final class HalResource {
18
19     private final JsonNode body;
20     private final HttpHeaders headers;
21     private final JsonMapper json;
22
23     HalResource(JsonNode body, HttpHeaders headers, JsonMapper json) {
24         this.body = body;
25         this.headers = headers;
26         this.json = json;
27     }
28
29     /** The first link with this relation, if the server offered one. */
30     public Optional<Link> link(String rel) {
31         return links(rel).stream().findFirst();
32     }
33
34     /** The link with this relation; its absence means the action is not available now. */
35     public Link requiredLink(String rel) {
36         return link(rel).orElseThrow(() -> new IllegalStateException(
37             "The server offered no '" + rel + "' link, so that action is not available"));
38     }
39
40     /** Every link with this relation. HAL sends one link as an object, several as an array. */
41     public List<Link> links(String rel) {
42         JsonNode node = body.path("_links").path(rel);
43         List<Link> links = new ArrayList<>();
44         for (JsonNode each : node.isArray() ? node : List.of(node)) {
45             if (each.hasNonNull("href")) {
46                 Link link = Link.of(each.get("href").asString(), rel);
47                 links.add(each.hasNonNull("name") ? link.withName(each.get("name").asString()) : link);
48             }
49         }
50         return links;
51     }
52
53     /** Resources embedded under a relation, such as the items of a collection. */
54     public List<HalResource> embedded(String rel) {
55         List<HalResource> items = new ArrayList<>();
56         body.path("_embedded").path(rel)
57             .forEach(item -> items.add(new HalResource(item, new HttpHeaders(), json)));
58         return items;

```

```

59     }
60
61     /** The properties as a Java type. Unknown properties are ignored: a tolerant reader. */
62     public <T> T as(Class<T> type) {
63         return json.treeToValue(body, type);
64     }
65
66     public String etag() {
67         return headers.getETag();
68     }
69
70     public HttpHeaders headers() {
71         return headers;
72     }
73
74     public JsonNode body() {
75         return body;
76     }
77 }

```

brokerage-client/src/main/java/com/example/brokerage/client/HypermediaClient.java

80 lines

```

1  package com.example.brokerage.client;
2
3  import java.math.BigDecimal;
4  import java.net.URI;
5  import java.util.function.Consumer;
6
7  import com.fasterxml.jackson.annotation.JsonFormat;
8  import com.fasterxml.jackson.annotation.JsonInclude;
9  import tools.jackson.databind.DeserializationFeature;
10 import tools.jackson.databind.json.JsonMapper;
11
12 import org.springframework.hateoas.Link;
13 import org.springframework.http.HttpHeaders;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.MediaType;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.web.client.RestClient;
18
19 /**
20  * HTTP for hypermedia: every request targets a link the server provided. There is no
21  * method that takes a path to append to a base URL, by design.
22  */
23 public class HypermediaClient {
24
25     public static final MediaType HAL_JSON = MediaType.parseMediaType("application/hal+json");
26     public static final MediaType MERGE_PATCH = MediaType.parseMediaType("application/merge-patch+json");
27
28     private final RestClient http;
29     private final JsonMapper json;
30
31     public HypermediaClient(RestClient.Builder builder) {
32         this.json = JsonMapper.builder()
33             // Tolerant reader: new properties on the server must never break this client.
34             .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
35             // The API's convention: decimals are strings, and absent fields are omitted.
36             .withConfigOverride(BigDecimal.class,
37                 o -> o.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)))
38             .changeDefaultPropertyInclusion(i -> i.withValueInclusion(JsonInclude.Include.NON_NULL))
39             .build();
40         this.http = builder
41             .defaultHeader(HttpHeaders.ACCEPT, HAL_JSON + ", application/problem+json")
42             // Pin the API version on every request, so a new default can never change
43             // what this client receives.
44             .defaultHeader("API-Version", "2")
45             .defaultStatusHandler(HttpStatus::isError, (request, response) -> {
46                 throw ApiProblem.from(response, json);
47             })

```

```

48         .build();
49     }
50
51     public HalResource get(Uri target) {
52         return get(target, headers -> { });
53     }
54
55     public HalResource get(Uri target, Consumer<HttpHeaders> headers) {
56         return read(http.get().uri(target).headers(headers).retrieve().toEntity(String.class));
57     }
58
59     public HalResource post(Link target, Object body, Consumer<HttpHeaders> headers) {
60         RestClient.RequestBodySpec request = http.post().uri(target.toUri()).headers(headers);
61         if (body != null) {
62             request.contentType(MediaType.APPLICATION_JSON).body(json.writeValueAsString(body));
63         }
64         return read(request.retrieve().toEntity(String.class));
65     }
66
67     /** A JSON merge patch, sent only if the resource still has the ETag the caller read. */
68     public HalResource patch(Link target, Object changes, String ifMatch) {
69         return read(http.patch().uri(target.toUri())
70             .contentType(MERGE_PATCH)
71             .header(HttpHeaders.IF_MATCH, ifMatch)
72             .body(json.writeValueAsString(changes))
73             .retrieve().toEntity(String.class));
74     }
75
76     private HalResource read(ResponseEntity<String> response) {
77         String body = response.getBody();
78         return new HalResource(json.readTree(body == null ? "{}" : body), response.getHeaders(), json);
79     }
80 }

```

brokerage-client/src/main/java/com/example/brokerage/client/Model.java

86 lines

```

1  package com.example.brokerage.client;
2
3  import java.math.BigDecimal;
4  import java.net.URI;
5  import java.time.Instant;
6  import java.util.List;
7  import java.util.Optional;
8
9  import com.fasterxml.jackson.annotation.JsonProperty;
10 import tools.jackson.databind.JsonNode;
11
12 /**
13  * The client's view of the API's representations: only the fields it uses. Enumerations
14  * are read as strings, so a status the server adds next year cannot break this client.
15  */
16 public final class Model {
17
18     private Model() {
19     }
20
21     public record Account(String id, String nickname, String type, String registration, String currency) {
22     }
23
24     public record Balances(String currency, BigDecimal cash, BigDecimal reservedForOrders,
25         BigDecimal buyingPower, BigDecimal positionsMarketValue, BigDecimal totalEquity,
26         BigDecimal unrealizedPnl) {
27     }
28
29     public record Position(String instrumentId, String symbol, BigDecimal quantity, BigDecimal averageCost,
30         BigDecimal lastPrice, BigDecimal marketValue, BigDecimal unrealizedPnl,
31         BigDecimal unrealizedPnlPercent) {
32     }
33 }

```

```

34     public record Instrument(String id, String symbol, String name, String type, boolean tradable) {
35     }
36
37     /** A quote, as served by API version 2. */
38     public record Quote(String symbol, BigDecimal last, PriceLevel bid, PriceLevel ask, BigDecimal change,
39         BigDecimal changePercent, Instant asOf) {
40
41         public record PriceLevel(BigDecimal price, BigDecimal size) {
42         }
43     }
44
45     /** What to trade: the body of a preview or a new order. */
46     public record OrderTicket(String instrumentId, String side, String type, BigDecimal quantity,
47         BigDecimal limitPrice, String timeInForce, String clientOrderId) {
48
49         public static OrderTicket market(String instrumentId, String side, BigDecimal quantity) {
50             return new OrderTicket(instrumentId, side, "MARKET", quantity, null, "DAY", null);
51         }
52
53         public static OrderTicket limit(String instrumentId, String side, BigDecimal quantity,
54             BigDecimal price) {
55             return new OrderTicket(instrumentId, side, "LIMIT", quantity, price, "DAY", null);
56         }
57     }
58
59     public record OrderPreview(String symbol, BigDecimal estimatedPrice, BigDecimal estimatedTotal,
60         BigDecimal buyingPower, boolean acceptable, List<Issue> issues) {
61
62         public record Issue(String type, String detail) {
63         }
64     }
65
66     /** An order, with the links the server offered for it at the time it was read. */
67     public record Order(String id, String symbol, String side, String type, BigDecimal quantity,
68         BigDecimal limitPrice, String status, BigDecimal filledQuantity, BigDecimal averageFillPrice,
69         Instant createdAt, @JsonProperty("_links") JsonNode links) {
70
71         public URI self() {
72             return link("self").orElseThrow();
73         }
74
75         /** Present only while the server allows the action it names. */
76         public Optional<URI> link(String rel) {
77             JsonNode href = links == null ? null : links.path(rel).path("href");
78             return href == null || href.isMissingNode() ? Optional.empty()
79                 : Optional.of(URI.create(href.asString()));
80         }
81
82         public boolean isCancellable() {
83             return link("bk:cancel").isPresent();
84         }
85     }
86 }

```

brokerage-client/src/test/java/com/example/brokerage/client/BrokerageClientTest.java

134 lines

```

1  package com.example.brokerage.client;
2
3  import static org.assertj.core.api.Assertions.assertThat;
4  import static org.assertj.core.api.Assertions.assertThatThrownBy;
5  import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
6  import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
7  import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
8  import static org.springframework.test.web.client.response.MockRestResponseCreators.withStatus;
9  import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;
10
11  import java.math.BigDecimal;
12  import java.net.URI;
13

```

```

14 import com.example.brokerage.client.Model.Order;
15 import com.example.brokerage.client.Model.OrderTicket;
16
17 import org.junit.jupiter.api.BeforeEach;
18 import org.junit.jupiter.api.Test;
19 import org.springframework.http.HttpMethod;
20 import org.springframework.http.HttpStatus;
21 import org.springframework.http.MediaType;
22 import org.springframework.test.web.client.ExpectedCount;
23 import org.springframework.test.web.client.MockRestServiceServer;
24 import org.springframework.web.client.RestClient;
25
26 /**
27  * The client follows whatever links the server sends, including unusual ones, and never
28  * builds a URL of its own.
29  */
30 class BrokerageClientTest {
31
32     static final MediaType HAL = MediaType.parseMediaType("application/hal+json");
33     static final MediaType PROBLEM = MediaType.parseMediaType("application/problem+json");
34
35     // Deliberately odd URLs: a client that constructs paths would never find them.
36     static final String ROOT = ""
37         {"_links": {"self": {"href": "https://api.test/"},
38             "bk:accounts": {"href": "https://api.test/v9/acct-list"},
39             "bk:instruments": {"href": "https://api.test/catalog{?symbol,type}", "templated": true}}}}"";
40     static final String ACCOUNTS = ""
41         {"_embedded": {"bk:accounts": [{"id": "ACC-1", "nickname": "Main", "type": "MARGIN",
42             "_links": {"self": {"href": "https://api.test/a/1"},
43                 "bk:balances": {"href": "https://api.test/money/for/ACC-1"},
44                 "bk:orders": {"href": "https://api.test/a/1/o{?status,cursor}",
45                     "templated": true}}}}}],
46         "_links": {"self": {"href": "https://api.test/v9/acct-list"}}}}"";
47
48     MockRestServiceServer server;
49     BrokerageClient brokerage;
50
51     @BeforeEach
52     void setUp() {
53         RestClient.Builder builder = RestClient.builder();
54         server = MockRestServiceServer.bindTo(builder).build();
55         brokerage = new BrokerageClient(new HypermediaClient(builder), URI.create("https://api.test/"));
56     }
57
58     @Test
59     void itFindsBalancesByFollowingLinksFromTheRoot() {
60         expectGet("https://api.test/", ROOT);
61         expectGet("https://api.test/v9/acct-list", ACCOUNTS);
62         expectGet("https://api.test/money/for/ACC-1", ""
63             {"currency": "USD", "cash": "1000.10", "buyingPower": "2000.20", "futureField": "ignored",
64             "_links": {"self": {"href": "https://api.test/money/for/ACC-1"}}}}"";
65
66         var balances = brokerage.balances("ACC-1");
67
68         assertThat(balances.cash()).isEqualByComparingTo("1000.10");
69         assertThat(balances.buyingPower()).isEqualByComparingTo("2000.20");
70         server.verify();
71     }
72
73     @Test
74     void aTransientFailureIsRetriedWithTheSameIdempotencyKey() {
75         expectGet("https://api.test/", ROOT);
76         expectGet("https://api.test/v9/acct-list", ACCOUNTS);
77         server.expect(ExpectedCount.once(), requestTo("https://api.test/a/1/o"))
78             .andExpect(method(HttpMethod.POST))
79             .andExpect(header("Idempotency-Key", "key-42"))
80             .andRespond(withStatus(HttpStatus.SERVICE_UNAVAILABLE).contentType(PROBLEM)
81                 .header("Retry-After", "0"))
82             .body("");

```

```

83         {"type": "https://docs.test/problems/unavailable", "title": "Busy",
84          "status": 503}""");
85     server.expect(ExpectedCount.once(), requestTo("https://api.test/a/1/o"))
86         .andExpect(method(HttpMethod.POST))
87         .andExpect(header("Idempotency-Key", "key-42"))
88         .andRespond(withStatus(HttpStatus.CREATED).contentType(HAL).body("""
89         {"id": "ORD-9", "status": "OPEN", "quantity": "5",
90          "_links": {"self": {"href": "https://api.test/a/1/o/9"}}}"""));
91
92     OrderTicket ticket = OrderTicket.market("EQ-X", "BUY", new BigDecimal("5"));
93     Order order = brokerage.place("ACC-1", ticket, "key-42");
94
95     assertThat(order.id()).isEqualTo("ORD-9");
96     server.verify();
97 }
98
99 @Test
100 void aBusinessRuleViolationArrivesAsATypedProblem() {
101     expectGet("https://api.test/", ROOT);
102     expectGet("https://api.test/v9/acct-list", ACCOUNTS);
103     server.expect(requestTo("https://api.test/a/1/o")).andRespond(
104         withStatus(HttpStatus.UNPROCESSABLE_CONTENT).contentType(PROBLEM).body("""
105         {"type": "https://docs.brokerage.example/problems/insufficient-buying-power",
106          "title": "The account does not have enough buying power", "status": 422,
107          "detail": "This order needs 900.00 but only 10.00 is available",
108          "available": "10.00"}"""));
109
110     OrderTicket ticket = OrderTicket.market("EQ-X", "BUY", BigDecimal.TEN);
111     assertThatThrownBy(() -> brokerage.place("ACC-1", ticket, "k"))
112         .isInstanceOf(Satisfying(ApiProblem.class, problem -> {
113             assertThat(problem.typeName()).isEqualTo("insufficient-buying-power");
114             assertThat(problem.isTransient()).isFalse();
115             assertThat(problem.body().path("available").asString()).isEqualTo("10.00");
116         }));
117 }
118
119 @Test
120 void anOrderWithoutACancelLinkIsNotCancelledAndNoRequestIsSent() {
121     Order filled = new Order("ORD-7", "AAPL", "BUY", "MARKET", BigDecimal.ONE, null, "FILLED",
122         BigDecimal.ONE, new BigDecimal("232.42"), null, null);
123
124     assertThatThrownBy(() -> brokerage.cancel(filled))
125         .isInstanceOf(IllegalStateException.class)
126         .hasMessageContaining("can no longer be cancelled");
127     server.verify();
128 }
129
130 private void expectGet(String url, String body) {
131     server.expect(ExpectedCount.once(), requestTo(url)).andExpect(method(HttpMethod.GET))
132         .andRespond(withSuccess(body, HAL));
133 }
134 }

```

E.6 The agent

brokerage-agent: the Spring AI assistant, its tools and its guardrails.

brokerage-agent/src/main/java/com/example/brokerage/agent/AgentConsole.java

80 lines

```

1 package com.example.brokerage.agent;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.util.UUID;
7
8 import org.springframework.ai.chat.client.ChatClient;

```

```

9 import org.springframework.ai.chat.memory.ChatMemory;
10 import org.springframework.ai.chat.model.ChatResponse;
11 import org.springframework.beans.factory.annotation.Value;
12 import org.springframework.boot.CommandLineRunner;
13 import org.springframework.boot.autoconfigure.condition.ConditionalOnBooleanProperty;
14 import org.springframework.stereotype.Component;
15
16 /**
17  * A console conversation. Lines starting with "/" are commands handled here, by code, and
18  * never reach the model: that is what makes "/confirm" a human decision.
19  */
20 @Component
21 @ConditionalOnBooleanProperty(name = "brokerage.console.enabled", matchIfMissing = true)
22 class AgentConsole implements CommandLineRunner {
23
24     private final ChatClient chat;
25     private final PendingActions pending;
26     private final BrokerageTools tools;
27     private final boolean modelConfigured;
28     private final String conversation = UUID.randomUUID().toString();
29
30     AgentConsole(ChatClient chat, PendingActions pending, BrokerageTools tools,
31         @Value("${spring.ai.anthropic.api-key:}") String apiKey) {
32         this.chat = chat;
33         this.pending = pending;
34         this.tools = tools;
35         this.modelConfigured = !apiKey.isBlank();
36     }
37
38     @Override
39     public void run(String... args) throws IOException {
40         System.out.println("Brokerage assistant. Ask about your accounts or ask for a trade.");
41         System.out.println("Commands: /confirm CODE, /discard CODE, /status, /quit");
42         BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
43         for (String line; (line = prompt(in)) != null && !line.equals("/quit"); ) {
44             if (line.startsWith("/confirm ")) {
45                 System.out.println(pending.confirm(line.substring(9).trim()));
46             } else if (line.startsWith("/discard ")) {
47                 boolean discarded = pending.discard(line.substring(9).trim());
48                 System.out.println(discarded ? "Discarded." : "Nothing to discard.");
49             } else if (line.equals("/status")) {
50                 // The tools are plain Java: call one directly to check the API and the token,
51                 // with no model involved.
52                 tools.listAccounts()
53                     .forEach(account -> System.out.println(account.id() + " " + account.nickname()));
54             } else if (!line.isBlank()) {
55                 System.out.println(modelConfigured ? ask(line)
56                     : "Set ANTHROPIC_API_KEY to talk to the assistant.");
57             }
58         }
59     }
60
61     private String ask(String question) {
62         ChatResponse response = chat.prompt()
63             .user(question)
64             .advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversation))
65             .call()
66             .chatResponse();
67         // Check why the model stopped before reading its text: a refusal has no answer to show.
68         String finish = response.getResult().getMetadata().getFinishReason();
69         if ("refusal".equalsIgnoreCase(finish)) {
70             return "The assistant declined that request. Try rephrasing it.";
71         }
72         return response.getResult().getOutput().getText();
73     }
74
75     private static String prompt(BufferedReader in) throws IOException {
76         System.out.print("\n> ");
77         String line = in.readLine();

```

```

78         return line == null ? null : line.trim();
79     }
80 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/BrokerageAgentApplication.java

80 lines

```

1  package com.example.brokerage.agent;
2
3  import java.net.URI;
4  import java.time.Clock;
5
6  import com.example.brokerage.client.BrokerageClient;
7  import com.example.brokerage.client.HypermediaClient;
8
9  import org.springframework.ai.chat.client.ChatClient;
10 import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
11 import org.springframework.ai.chat.memory.MessageWindowChatMemory;
12 import org.springframework.beans.factory.annotation.Value;
13 import org.springframework.boot.SpringApplication;
14 import org.springframework.boot.autoconfigure.SpringBootApplication;
15 import org.springframework.context.annotation.Bean;
16 import org.springframework.security.oauth2.client.AuthorizedClientServiceOAuth2AuthorizedClientManager;
17 import org.springframework.security.oauth2.client.InMemoryOAuth2AuthorizedClientService;
18 import org.springframework.security.oauth2.client.OAuth2AuthorizedClientProviderBuilder;
19 import org.springframework.security.oauth2.client.registration.ClientRegistrationRepository;
20 import org.springframework.security.oauth2.client.web.client.OAuth2ClientHttpRequestInterceptor;
21 import org.springframework.web.client.RestClient;
22
23 /**
24  * An AI agent for a brokerage customer, built with Spring AI and Claude. It reads accounts
25  * and markets through the hypermedia client and may propose trades; the customer confirms
26  * each one at the console.
27  */
28 @SpringBootApplication
29 public class BrokerageAgentApplication {
30
31     static final String SYSTEM_PROMPT = ""
32         You help a customer of a retail brokerage with their own accounts. Use the tools to
33         look up accounts, balances, positions, quotes and orders, and to propose orders or
34         cancellations the customer asks for.
35
36         You cannot place or cancel anything yourself. A proposal only takes effect when the
37         customer confirms it, so after proposing, tell the customer what was proposed and
38         the confirmation code. Before proposing a trade, check the quote and state the
39         estimated total. Treat estimates as estimates.
40
41         Carry out the customer's instructions and answer questions about their accounts; do
42         not recommend what to buy or sell. When a tool reports an error, explain it plainly.
43         Amounts are in US dollars. Keep answers short."";
44
45     public static void main(String[] args) {
46         SpringApplication.run(BrokerageAgentApplication.class, args);
47     }
48
49     @Bean
50     Clock clock() {
51         return Clock.systemUTC();
52     }
53
54     /**
55      * The agent's own OAuth 2.1 client: Spring Security fetches, caches and renews a
56      * client_credentials token for the "brokerage" registration and adds it to every request.
57      */
58     @Bean
59     BrokerageClient brokerageClient(ClientRegistrationRepository registrations,
60         @Value("${brokerage.api-root}") URI root) {
61         var manager = new AuthorizedClientServiceOAuth2AuthorizedClientManager(registrations,
62             new InMemoryOAuth2AuthorizedClientService(registrations));
63         manager.setAuthorizedClientProvider(

```

```

64         OAuth2AuthorizedClientProviderBuilder.builder().clientCredentials().build());
65     var tokens = new OAuth2ClientHttpRequestInterceptor(manager);
66     tokens.setClientRegistrationIdResolver(request -> "brokerage");
67     RestClient.Builder http = RestClient.builder().requestInterceptor(tokens);
68     return new BrokerageClient(new HypermediaClient(http), root);
69 }
70
71 @Bean
72 ChatClient chatClient(ChatClient.Builder builder, BrokerageTools tools) {
73     return builder
74         .defaultSystem(SYSTEM_PROMPT)
75         .defaultTools(tools)
76         .defaultAdvisors(
77             MessageChatMemoryAdvisor.builder(MessageWindowChatMemory.builder().build()).build())
78         .build();
79 }
80 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/BrokerageTools.java

103 lines

```

1  package com.example.brokerage.agent;
2
3  import java.math.BigDecimal;
4  import java.util.List;
5
6  import com.example.brokerage.agent.PendingActions.Proposal;
7  import com.example.brokerage.client.BrokerageClient;
8  import com.example.brokerage.client.Model.Account;
9  import com.example.brokerage.client.Model.Balances;
10 import com.example.brokerage.client.Model.Order;
11 import com.example.brokerage.client.Model.OrderPreview;
12 import com.example.brokerage.client.Model.OrderTicket;
13 import com.example.brokerage.client.Model.Position;
14 import com.example.brokerage.client.Model.Quote;
15
16 import org.springframework.ai.tool.annotation.Tool;
17 import org.springframework.ai.tool.annotation.ToolParam;
18 import org.springframework.stereotype.Component;
19
20 /**
21  * The agent's tools. Each is a thin wrapper over the hypermedia client, so the model sees
22  * exactly what the API allows, and every rule is still enforced by the API itself.
23  * Read tools act immediately; tools that would change anything only create proposals.
24  */
25 @Component
26 public class BrokerageTools {
27
28     private final BrokerageClient brokerage;
29     private final PendingActions pending;
30
31     public BrokerageTools(BrokerageClient brokerage, PendingActions pending) {
32         this.brokerage = brokerage;
33         this.pending = pending;
34     }
35
36     @Tool(description = "List the customer's brokerage accounts: ID, nickname, type and currency.")
37     public List<Account> listAccounts() {
38         return brokerage.accounts();
39     }
40
41     @Tool(description = "Cash, cash reserved for open orders, buying power and total equity of one account.")
42     public Balances getBalances(@ToolParam(description = "Account ID, such as ACC-1001") String accountId) {
43         return brokerage.balances(accountId);
44     }
45
46     @Tool(description = "Holdings of one account with quantity, average cost, market value and "
47         + "unrealized P&L.")
48     public List<Position> getPositions(@ToolParam(description = "Account ID") String accountId) {
49         return brokerage.positions(accountId);

```

```

50     }
51
52     @Tool(description = "The latest quote for a stock or ETF: last price, bid, ask and change on the day.")
53     public Quote getQuote(@ToolParam(description = "Ticker symbol, such as AAPL") String symbol) {
54         return brokerage.quote(symbol);
55     }
56
57     @Tool(description = "Orders in one account that are still working (open or partially filled).")
58     public List<Order> getOpenOrders(@ToolParam(description = "Account ID") String accountId) {
59         return brokerage.openOrders(accountId);
60     }
61
62     @Tool(description = ""
63         Propose an order for the customer to confirm. The brokerage checks it first and the
64         estimate is returned. Nothing is placed until the customer confirms the proposal.""")
65     public Object proposeOrder(@ToolParam(description = "Account ID") String accountId,
66         @ToolParam(description = "Ticker symbol") String symbol,
67         @ToolParam(description = "BUY or SELL") String side,
68         @ToolParam(description = "Number of shares, as a decimal string such as \"10\"") String quantity,
69         @ToolParam(description = "Limit price as a decimal string; omit for a market order",
70             required = false) String limitPrice) {
71         String instrumentId = brokerage.instrument(symbol).id();
72         BigDecimal shares = new BigDecimal(quantity);
73         OrderTicket ticket = limitPrice == null || limitPrice.isBlank()
74             ? OrderTicket.market(instrumentId, side, shares)
75             : OrderTicket.limit(instrumentId, side, shares, new BigDecimal(limitPrice));
76         OrderPreview preview = brokerage.preview(accountId, ticket);
77         if (!preview.acceptable()) {
78             return preview; // the issues explain why; nothing to confirm
79         }
80         String price = ticket.limitPrice() == null ? "at market" : "limit " + ticket.limitPrice();
81         String summary = "%s %s %s in %s, %s, estimated total %s USD".formatted(side, quantity, symbol,
82             accountId, price, preview.estimatedTotal());
83         return pending.propose(summary, idempotencyKey -> {
84             Order order = brokerage.place(accountId, ticket, idempotencyKey);
85             return "Placed order " + order.id() + ": " + order.status();
86         });
87     }
88
89     @Tool(description = "Propose cancelling a working order. "
90         + "Nothing is cancelled until the customer confirms.")
91     public Proposal proposeCancellation(@ToolParam(description = "Account ID") String accountId,
92         @ToolParam(description = "Order ID, such as ORD-100002") String orderId) {
93         Order order = brokerage.openOrders(accountId).stream()
94             .filter(open -> open.id().equals(orderId))
95             .findFirst()
96             .orElseThrow(() -> new IllegalArgumentException(
97                 "No working order " + orderId + " in " + accountId));
98         String summary = "Cancel " + order.side() + " " + order.quantity() + " " + order.symbol()
99             + " (" + orderId + ")";
100         return pending.propose(summary,
101             ignored -> "Cancellation requested: " + brokerage.cancel(order).status());
102     }
103 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/PendingActions.java

65 lines

```

1 package com.example.brokerage.agent;
2
3 import java.time.Clock;
4 import java.time.Duration;
5 import java.time.Instant;
6 import java.util.Locale;
7 import java.util.Map;
8 import java.util.UUID;
9 import java.util.concurrent.ConcurrentHashMap;
10 import java.util.function.Function;
11
12 import org.springframework.stereotype.Component;

```

```

13
14 /**
15  * The human-in-the-loop gate. The model can only propose a trade; carrying it out takes a
16  * person typing "/confirm CODE" at the console, a path that no tool can reach.
17  *
18  * <p>The confirmation code doubles as the order's Idempotency-Key, so confirming the same
19  * proposal twice, or retrying after a timeout, can never place a second order.
20  */
21 @Component
22 public class PendingActions {
23
24     /** What the model is told about a proposal: enough to ask the customer, nothing to act on. */
25     public record Proposal(String code, String summary, Instant expiresAt, String instructions) {
26     }
27
28     private record Pending(Proposal proposal, Function<String, String> action) {
29     }
30
31     static final Duration LIFETIME = Duration.ofMinutes(5);
32
33     private final Map<String, Pending> pending = new ConcurrentHashMap<>();
34     private final Clock clock;
35
36     public PendingActions(Clock clock) {
37         this.clock = clock;
38     }
39
40     /** Records an action to run later. The action receives the code to use as its Idempotency-Key. */
41     public Proposal propose(String summary, Function<String, String> action) {
42         String code = UUID.randomUUID().toString().substring(0, 8).toUpperCase(Locale.ROOT);
43         Proposal proposal = new Proposal(code, summary, clock.instant().plus(LIFETIME),
44             "Nothing has been done yet. Ask the customer to type /confirm " + code
45             + " to go ahead, or /discard " + code + " to drop it. It expires in 5 minutes.");
46         pending.put(code, new Pending(proposal, action));
47         return proposal;
48     }
49
50     /** Called only from the console, when a person types /confirm CODE. */
51     public String confirm(String code) {
52         Pending action = pending.remove(code.toUpperCase(Locale.ROOT));
53         if (action == null) {
54             return "There is no pending action " + code + ".";
55         }
56         if (clock.instant().isAfter(action.proposal().expiresAt())) {
57             return "Proposal " + code + " expired; ask the assistant again.";
58         }
59         return action.action().apply(action.proposal().code());
60     }
61
62     public boolean discard(String code) {
63         return pending.remove(code.toUpperCase(Locale.ROOT)) != null;
64     }
65 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/RefusalFallbacks.java

58 lines

```

1 package com.example.brokerage.agent;
2
3 import java.io.IOException;
4
5 import okhttp3.Interceptor;
6 import okhttp3.MediaType;
7 import okhttp3.Request;
8 import okhttp3.RequestBody;
9 import okhttp3.Response;
10 import okio.Buffer;
11 import tools.jackson.databind.json.JsonMapper;
12 import tools.jackson.databind.node.ObjectNode;
13

```

```

14 import org.springframework.ai.anthropic.http.okhttp.AnthropicHttpClientBuilderCustomizer;
15 import org.springframework.ai.anthropic.http.okhttp.SpringAiAnthropicHttpClient;
16 import org.springframework.stereotype.Component;
17
18 /**
19  * Opts every Messages API request into server-side refusal fallbacks: if a safety
20  * classifier declines a request, the API re-runs it on the fallback model Anthropic
21  * recommends for that category instead of returning the refusal. Spring AI has no option
22  * for this request field, so it is added at the HTTP layer.
23  */
24 @Component
25 class RefusalFallbacks implements AnthropicHttpClientBuilderCustomizer, Interceptor {
26
27     static final String BETA = "server-side-fallback-2026-07-01";
28     private static final ObjectMapper JSON = ObjectMapper.builder().build();
29
30     @Override
31     public void customize(SpringAiAnthropicHttpClient.Builder builder) {
32         builder.interceptor(this);
33     }
34
35     @Override
36     public Response intercept(Chain chain) throws IOException {
37         Request request = chain.request();
38         if (!"POST".equals(request.method()) || !request.url().encodedPath().endsWith("/v1/messages")
39             || request.body() == null) {
40             return chain.proceed(request);
41         }
42         Buffer body = new Buffer();
43         request.body().writeTo(body);
44         MediaType type = request.body().contentType();
45         String existing = request.header("anthropic-beta");
46         return chain.proceed(request.newBuilder()
47             .header("anthropic-beta", existing == null ? BETA : existing + "," + BETA)
48             .post(RequestBody.create(withFallbacks(body.readUtf8()), type))
49             .build());
50     }
51
52     /** The request body with "fallbacks": "default" added. */
53     static String withFallbacks(String json) {
54         ObjectNode request = (ObjectNode) JSON.readTree(json);
55         request.put("fallbacks", "default");
56         return JSON.writeValueAsString(request);
57     }
58 }

```

brokerage-agent/src/main/resources/application.yaml

34 lines

```

1 spring:
2   application:
3     name: brokerage-agent
4   main:
5     web-application-type: none
6     banner-mode: off
7   ai:
8     anthropic:
9       api-key: ${ANTHROPIC_API_KEY:}
10      chat:
11        options:
12          model: claude-opus-5
13          # Leave room for tool calls and thinking; Spring AI's default is 4096.
14          max-tokens: 16000
15      security:
16        oauth2:
17          client:
18            registration:
19              brokerage:
20                provider: brokerage-auth
21                client-id: brokerage-agent

```

```

22         client-secret: agent-secret
23         authorization-grant-type: client_credentials
24         scope: accounts:read, market-data:read, orders:read, orders:write
25     provider:
26         brokerage-auth:
27             token-uri: http://localhost:9000/oauth2/token
28
29 brokerage:
30     api-root: http://localhost:8080/
31
32 logging:
33     level:
34         root: warn

```

brokerage-agent/src/test/java/com/example/brokerage/agent/BrokerageToolsTest.java

130 lines

```

1 package com.example.brokerage.agent;
2
3 import static org.assertj.core.api.Assertions.assertThat;
4 import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
5 import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
6 import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
7 import static org.springframework.test.web.client.response.MockRestResponseCreators.withStatus;
8 import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;
9
10 import java.net.URI;
11 import java.time.Clock;
12 import java.util.Arrays;
13
14 import com.example.brokerage.agent.PendingActions.Proposal;
15 import com.example.brokerage.client.BrokerageClient;
16 import com.example.brokerage.client.HypermediaClient;
17
18 import org.junit.jupiter.api.BeforeEach;
19 import org.junit.jupiter.api.Test;
20 import org.springframework.ai.support.ToolCallbacks;
21 import org.springframework.ai.tool.ToolCallback;
22 import org.springframework.http.HttpMethod;
23 import org.springframework.http.HttpStatus;
24 import org.springframework.http.MediaType;
25 import org.springframework.test.web.client.ExpectedCount;
26 import org.springframework.test.web.client.MockRestServiceServer;
27 import org.springframework.web.client.RestClient;
28
29 /**
30  * The tools as the model will see them, and the guarantee that matters most: a proposal
31  * places nothing, and only a human confirmation does.
32  */
33 class BrokerageToolsTest {
34
35     static final MediaType HAL = MediaType.parseMediaType("application/hal+json");
36     static final String ROOT = ""
37         + {"_links": {"self": {"href": "http://api/"}, "bk:accounts": {"href": "http://api/accounts"},
38           "bk:instruments": {"href": "http://api/instruments{?symbol,type}", "templated": true}}}}"";
39     static final String ACCOUNTS = ""
40         + {"_embedded": {"bk:accounts": [{"id": "ACC-1001", "nickname": "Main", "type": "MARGIN",
41           "_links": {"self": {"href": "http://api/accounts/ACC-1001"},
42             "bk:orders": {"href": "http://api/accounts/ACC-1001/orders{?status,symbol,cursor,limit}",
43               "templated": true},
44             "bk:order-previews": {"href": "http://api/accounts/ACC-1001/order-previews"}}}}}}"";
45     static final String AAPL = ""
46         + {"_embedded": {"bk:instruments": [{"id": "EQ-AAPL", "symbol": "AAPL", "name": "Apple Inc.",
47           "type": "EQUITY", "tradable": true,
48           "_links": {"self": {"href": "http://api/instruments/EQ-AAPL"}}}}}}"";
49
50     MockRestServiceServer api;
51     PendingActions pending;
52     BrokerageTools tools;
53

```

```

54 @BeforeEach
55 void setUp() {
56     RestClient.Builder builder = RestClient.builder();
57     api = MockRestServiceServer.bindTo(builder).build();
58     pending = new PendingActions(Clock.systemUTC());
59     var brokerage = new BrokerageClient(new HypermediaClient(builder), URI.create("http://api/"));
60     tools = new BrokerageTools(brokerage, pending);
61 }
62
63 @Test
64 void theModelSeesSevenToolsWithDescribedParameters() {
65     ToolCallback[] callbacks = ToolCallbacks.from(tools);
66     assertThat(Arrays.stream(callbacks).map(callback -> callback.getToolDefinition().name()))
67         .containsExactlyInAnyOrder("listAccounts", "getBalances", "getPositions", "getQuote",
68             "getOpenOrders", "proposeOrder", "proposeCancellation");
69     String proposeOrder = Arrays.stream(callbacks)
70         .filter(callback -> callback.getToolDefinition().name().equals("proposeOrder"))
71         .findFirst().orElseThrow().getToolDefinition().inputSchema();
72     assertThat(proposeOrder).contains("\\required\\").contains("\\quantity\\").contains("decimal string");
73 }
74
75 @Test
76 void proposingPlacesNothingAndConfirmingPlacesExactlyOnce() {
77     expect("http://api/", ROOT); // symbol -> instrument
78     expect("http://api/instruments?symbol=AAPL", AAPL);
79     expect("http://api/", ROOT); // account -> its previews link
80     expect("http://api/accounts", ACCOUNTS);
81     api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/order-previews"))
82         .andExpect(method(HttpMethod.POST))
83         .andRespond(withSuccess("""
84             {"symbol": "AAPL", "estimatedPrice": "232.42", "estimatedTotal": "2324.20",
85             "buyingPower": "55241.10", "acceptable": true, "issues": []}""", HAL));
86
87     Object result = tools.proposeOrder("ACC-1001", "AAPL", "BUY", "10", null);
88
89     assertThat(result).assertInstanceOf(Proposal.class);
90     Proposal proposal = (Proposal) result;
91     assertThat(proposal.summary()).contains("BUY 10 AAPL").contains("2324.20");
92     api.verify(); // a preview, and no order
93
94     api.reset();
95     expect("http://api/", ROOT);
96     expect("http://api/accounts", ACCOUNTS);
97     api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/orders"))
98         .andExpect(method(HttpMethod.POST))
99         .andExpect(header("Idempotency-Key", proposal.code()))
100         .andRespond(withStatus(HttpStatus.CREATED).contentType(HAL).body("""
101             {"id": "ORD-100009", "status": "OPEN",
102             "_links": {"self": {"href": "http://api/accounts/ACC-1001/orders/ORD-100009"}}}"""));
103
104     assertThat(pending.confirm(proposal.code())).isEqualTo("Placed order ORD-100009: OPEN");
105     assertThat(pending.confirm(proposal.code())).startsWith("There is no pending action");
106     api.verify();
107 }
108
109 @Test
110 void anUnacceptablePreviewIsReturnedWithoutAProposal() {
111     expect("http://api/", ROOT); // symbol -> instrument
112     expect("http://api/instruments?symbol=AAPL", AAPL);
113     expect("http://api/", ROOT); // account -> its previews link
114     expect("http://api/accounts", ACCOUNTS);
115     api.expect(requestTo("http://api/accounts/ACC-1001/order-previews")).andRespond(withSuccess("""
116         {"symbol": "AAPL", "acceptable": false, "issues": [{"type":
117             "https://docs.brokerage.example/problems/insufficient-buying-power",
118             "detail": "This order needs 2324200.00 but only 55241.10 is available"}]}""", HAL));
119
120     Object result = tools.proposeOrder("ACC-1001", "AAPL", "BUY", "10000", null);
121
122     assertThat(result).isNotInstanceOf(Proposal.class);

```

```

123         assertThat(result.toString()).contains("insufficient-buying-power");
124     }
125
126     private void expect(String url, String body) {
127         api.expect(ExpectedCount.once(), requestTo(url)).andExpect(method(HttpMethod.GET))
128             .andRespond(withSuccess(body, HAL));
129     }
130 }

```

brokerage-agent/src/test/java/com/example/brokerage/agent/GuardrailsTest.java

81 lines

```

1  package com.example.brokerage.agent;
2
3  import static org.assertj.core.api.Assertions.assertThat;
4
5  import java.time.Clock;
6  import java.time.Instant;
7  import java.time.ZoneOffset;
8  import java.util.concurrent.atomic.AtomicInteger;
9
10 import org.junit.jupiter.api.Test;
11
12 /**
13  * Proposals expire, can be discarded, and run at most once; every Messages API request is
14  * opted into server-side refusal fallbacks.
15  */
16 class GuardrailsTest {
17
18     @Test
19     void anExpiredProposalIsNotCarriedOut() {
20         MutableClock clock = new MutableClock();
21         PendingActions pending = new PendingActions(clock);
22         AtomicInteger runs = new AtomicInteger();
23         var proposal = pending.propose("Buy 1 AAPL", key -> "ran " + runs.incrementAndGet());
24
25         clock.advanceSeconds(301);
26
27         assertThat(pending.confirm(proposal.code()).contains("expired"));
28         assertThat(runs).hasValue(0);
29     }
30
31     @Test
32     void aDiscardedProposalCannotBeConfirmed() {
33         PendingActions pending = new PendingActions(Clock.systemUTC());
34         var proposal = pending.propose("Sell 1 MSFT", key -> "ran");
35
36         assertThat(pending.discard(proposal.code())).isTrue();
37         assertThat(pending.confirm(proposal.code()).startsWith("There is no pending action"));
38     }
39
40     @Test
41     void theConfirmationCodeIsTheIdempotencyKey() {
42         PendingActions pending = new PendingActions(Clock.systemUTC());
43         var proposal = pending.propose("Buy 1 AAPL", key -> key);
44
45         assertThat(pending.confirm(proposal.code().toLowerCase()).isEqualTo(proposal.code()));
46     }
47
48     @Test
49     void everyMessagesRequestAsksForServerSideFallbacks() {
50         String request = "{\"model\":\"claude-opus-5\",\"max_tokens\":16000,\"messages\":[]}";
51
52         assertThat(RefusalFallbacks.withFallbacks(request))
53             .contains("{\"fallbacks\":\"default\"}")
54             .contains("{\"model\":\"claude-opus-5\"}")
55             .contains("{\"max_tokens\":16000}");
56     }
57
58     /** A clock that tests can move forward. */

```

```

59     static final class MutableClock extends Clock {
60         private Instant now = Instant.parse("2026-09-28T15:00:00Z");
61
62         void advanceSeconds(long seconds) {
63             now = now.plusSeconds(seconds);
64         }
65
66         @Override
67         public Instant instant() {
68             return now;
69         }
70
71         @Override
72         public ZoneOffset getZone() {
73             return ZoneOffset.UTC;
74         }
75
76         @Override
77         public Clock withZone(java.time.ZoneId zone) {
78             return this;
79         }
80     }
81 }

```

E.7 The OpenAPI contract

openapi/brokerage-api.yaml: the API's OpenAPI 3.2 description.

openapi/brokerage-api.yaml

1086 lines

```

1  openapi: 3.2.1
2  $self: https://api.brokerage.example/openapi.yaml
3
4  info:
5    title: Brokerage API
6    version: "2.0"
7    summary: Accounts, positions, market data and trading for a retail brokerage, as hypermedia.
8    description: |
9      A resource-oriented REST API that follows the standards in *REST API Standards with
10     HATEOAS* (Smiroh Publishers).
11
12     * **Hypermedia.** Every representation is HAL (`application/hal+json`) with a `_links`
13     object. Ask for `application/prs.hal-forms+json` to also receive `_templates`: the
14     state transitions available right now. Clients should bookmark only the API root and
15     follow links from there. Custom link relations use the CURIE `bk:`, which expands to
16     `https://docs.brokerage.example/rels/{rel}`.
17     * **Money.** Prices, amounts and quantities are decimal strings, never JSON numbers.
18     * **Errors.** Every error is an RFC 9457 problem document (`application/problem+json`)
19     whose `type` URI identifies the error. Branch on `type`, never on `title` or `detail`.
20     * **Versions.** Send `API-Version: 2` to opt in to version 2. Version 1 is the default and
21     is deprecated: its responses carry `Deprecation` (RFC 9745) and `Sunset` (RFC 8594).
22     * **Quotas.** Every response carries `RateLimit-Policy` and `RateLimit`. Exceeding a quota
23     returns 429 with `Retry-After`.
24     * **Tracing.** Every response echoes `X-Request-Id`; quote it in support requests.
25  contact:
26    name: Brokerage API support
27    url: https://docs.brokerage.example/support
28  license:
29    name: Apache 2.0
30    identifier: Apache-2.0
31
32  servers:
33    - url: http://localhost:8080
34      description: Local development (see brokerage-apis/README.md)
35    - url: https://api.brokerage.example
36      description: Production (illustrative)
37

```

```

38 tags:
39   - name: discovery
40     summary: Discovery
41     description: The API root and the protected resource metadata, where every client starts.
42     kind: nav
43   - name: accounts
44     summary: Accounts
45     description: A customer's brokerage accounts.
46     kind: nav
47   - name: holdings
48     summary: Balances and positions
49     parent: accounts
50     kind: nav
51   - name: transactions
52     summary: Transaction history
53     parent: accounts
54     kind: nav
55   - name: trading
56     summary: Trading
57     description: Orders, previews and live order events.
58     kind: nav
59   - name: orders
60     summary: Orders
61     parent: trading
62     kind: nav
63   - name: order-events
64     summary: Order events
65     parent: trading
66     kind: nav
67   - name: market-data
68     summary: Market data
69     description: Instruments, quotes and option chains.
70     kind: nav
71   - name: deprecated
72     summary: Deprecated
73     description: Operations or representations scheduled for removal.
74     kind: badge
75
76 paths:
77   /:
78     get:
79       operationId: getApiRoot
80       summary: The API root
81       description: The one URL a client needs to know. Everything else is discovered from its links.
82       tags: [discovery]
83       security: []
84       responses:
85         "200":
86           description: Links to the top-level resources and to this document.
87           headers:
88             X-Request-Id: { $ref: "#/components/headers/X-Request-Id" }
89           content:
90             application/hal+json:
91               schema: { $ref: "#/components/schemas/ApiRoot" }
92
93   /.well-known/oauth-protected-resource:
94     get:
95       operationId: getProtectedResourceMetadata
96       summary: OAuth 2.0 protected resource metadata (RFC 9728)
97       description: Which authorization server issues tokens for this API, and which scopes it uses.
98       tags: [discovery]
99       security: []
100      responses:
101        "200":
102          description: The metadata document.
103          content:
104            application/json:
105              schema: { $ref: "#/components/schemas/ProtectedResourceMetadata" }
106

```

```

107 /accounts:
108   get:
109     operationId: listAccounts
110     summary: List accounts
111     tags: [accounts]
112     security: [{ oauth2: [accounts:read] }]
113     responses:
114       "200":
115         description: The caller's accounts. Links depend on the token's scopes.
116         headers:
117           RateLimit: { $ref: "#/components/headers/RateLimit" }
118           RateLimit-Policy: { $ref: "#/components/headers/RateLimit-Policy" }
119         content:
120           application/hal+json:
121             schema: { $ref: "#/components/schemas/AccountCollection" }
122       "401": { $ref: "#/components/responses/Unauthorized" }
123       "403": { $ref: "#/components/responses/Forbidden" }
124       "429": { $ref: "#/components/responses/TooManyRequests" }
125
126 /accounts/{accountId}:
127   parameters:
128     - $ref: "#/components/parameters/accountId"
129   get:
130     operationId: getAccount
131     summary: Get an account
132     tags: [accounts]
133     security: [{ oauth2: [accounts:read] }]
134     responses:
135       "200":
136         description: >-
137           The account. With orders:read the links include orders, previews and events; with
138           orders:write, HAL-FORMS clients also receive a placeOrder template.
139         content:
140           application/hal+json:
141             schema: { $ref: "#/components/schemas/Account" }
142           application/prs.hal-forms+json:
143             schema: { $ref: "#/components/schemas/Account" }
144       "401": { $ref: "#/components/responses/Unauthorized" }
145       "403": { $ref: "#/components/responses/Forbidden" }
146       "404": { $ref: "#/components/responses/NotFound" }
147
148 /accounts/{accountId}/balances:
149   parameters:
150     - $ref: "#/components/parameters/accountId"
151   get:
152     operationId: getBalances
153     summary: Get balances and buying power
154     tags: [holdings]
155     security: [{ oauth2: [accounts:read] }]
156     responses:
157       "200":
158         description: Cash, reservations for open orders, buying power and market value.
159         content:
160           application/hal+json:
161             schema: { $ref: "#/components/schemas/Balances" }
162       "401": { $ref: "#/components/responses/Unauthorized" }
163       "404": { $ref: "#/components/responses/NotFound" }
164
165 /accounts/{accountId}/positions:
166   parameters:
167     - $ref: "#/components/parameters/accountId"
168   get:
169     operationId: listPositions
170     summary: List positions
171     tags: [holdings]
172     security: [{ oauth2: [accounts:read] }]
173     responses:
174       "200":
175         description: Every holding, valued at the current market price.

```

```

176         content:
177             application/hal+json:
178                 schema: { $ref: "#/components/schemas/PositionCollection" }
179         "401": { $ref: "#/components/responses/Unauthorized" }
180         "404": { $ref: "#/components/responses/NotFound" }
181
182 /accounts/{accountId}/positions/{instrumentId}:
183 parameters:
184     - $ref: "#/components/parameters/accountId"
185     - $ref: "#/components/parameters/instrumentId"
186 get:
187     operationId: getPosition
188     summary: Get one position
189     tags: [holdings]
190     security: [{ oauth2: [accounts:read] }]
191     responses:
192         "200":
193             description: The position in one instrument.
194             content:
195                 application/hal+json:
196                     schema: { $ref: "#/components/schemas/Position" }
197         "404": { $ref: "#/components/responses/NotFound" }
198
199 /accounts/{accountId}/transactions:
200 parameters:
201     - $ref: "#/components/parameters/accountId"
202 get:
203     operationId: listTransactions
204     summary: List transactions
205     description: Newest first. Follow the `next` link for older entries.
206     tags: [transactions]
207     security: [{ oauth2: [accounts:read] }]
208     parameters:
209         - name: type
210           in: query
211           schema: { $ref: "#/components/schemas/TransactionType" }
212         - name: from
213           in: query
214           description: Earliest date to include (UTC), inclusive.
215           schema: { type: string, format: date }
216         - name: to
217           in: query
218           description: Latest date to include (UTC), inclusive.
219           schema: { type: string, format: date }
220         - $ref: "#/components/parameters/cursor"
221         - $ref: "#/components/parameters/limit"
222     responses:
223         "200":
224             description: One page of transactions.
225             content:
226                 application/hal+json:
227                     schema: { $ref: "#/components/schemas/TransactionPage" }
228         "400": { $ref: "#/components/responses/BadRequest" }
229         "404": { $ref: "#/components/responses/NotFound" }
230
231 /accounts/{accountId}/orders:
232 parameters:
233     - $ref: "#/components/parameters/accountId"
234 get:
235     operationId: listOrders
236     summary: List orders
237     description: Newest first, optionally filtered. `status` may be repeated.
238     tags: [orders]
239     security: [{ oauth2: [orders:read] }]
240     parameters:
241         - name: status
242           in: query
243           style: form
244           explode: true

```

```

245     schema:
246       type: array
247       items: { $ref: "#/components/schemas/OrderStatus" }
248   - name: symbol
249     in: query
250     schema: { type: string, examples: [AAPL] }
251   - $ref: "#/components/parameters/cursor"
252   - $ref: "#/components/parameters/limit"
253 responses:
254   "200":
255     description: One page of orders.
256     content:
257       application/hal+json:
258         schema: { $ref: "#/components/schemas/OrderPage" }
259   "400": { $ref: "#/components/responses/BadRequest" }
260   "403": { $ref: "#/components/responses/Forbidden" }
261   "404": { $ref: "#/components/responses/NotFound" }
262 post:
263   operationId: placeOrder
264   summary: Place an order
265   description: >-
266     Every new order needs a unique Idempotency-Key. Retrying with the same key and body
267     returns the original order (Idempotent-Replayed: true); the same key with a different
268     body is refused.
269   tags: [orders]
270   security: [{ oauth2: [orders:write] }]
271   parameters:
272     - $ref: "#/components/parameters/Idempotency-Key"
273   requestBody:
274     required: true
275     content:
276       application/json:
277         schema: { $ref: "#/components/schemas/OrderRequest" }
278         examples:
279           limitBuy:
280             summary: Buy 10 AAPL at 230.00 or better
281             dataValue:
282               instrumentId: EQ-AAPL
283               side: BUY
284               type: LIMIT
285               timeInForce: DAY
286               quantity: "10"
287               limitPrice: "230.00"
288               clientOrderId: rebalance-2026-09
289   responses:
290   "201":
291     description: The order was accepted and is now working.
292     headers:
293       Location: { $ref: "#/components/headers/Location" }
294       ETag: { $ref: "#/components/headers/ETag" }
295       Idempotent-Replayed: { $ref: "#/components/headers/Idempotent-Replayed" }
296     content:
297       application/hal+json:
298         schema: { $ref: "#/components/schemas/Order" }
299   links:
300     getOrder:
301       operationId: getOrder
302       parameters:
303         accountId: $request.path.accountId
304         orderId: $response.body#/id
305     cancelOrder:
306       operationId: cancelOrder
307       parameters:
308         accountId: $request.path.accountId
309         orderId: $response.body#/id
310   "400": { $ref: "#/components/responses/BadRequest" }
311   "403": { $ref: "#/components/responses/Forbidden" }
312   "404": { $ref: "#/components/responses/NotFound" }
313   "422": { $ref: "#/components/responses/UnprocessableContent" }

```

```

314         "429": { $ref: "#/components/responses/TooManyRequests" }
315
316 /accounts/{accountId}/orders/{orderId}:
317   parameters:
318     - $ref: "#/components/parameters/accountId"
319     - $ref: "#/components/parameters/orderId"
320   get:
321     operationId: getOrder
322     summary: Get an order
323     tags: [orders]
324     security: [{ oauth2: [orders:read] }]
325     parameters:
326       - $ref: "#/components/parameters/If-None-Match"
327     responses:
328       "200":
329         description: >-
330           The order. A working order links to `edit` and `bk:cancel`; a filled or cancelled
331           order does not.
332         headers:
333           ETag: { $ref: "#/components/headers/ETag" }
334         content:
335           application/hal+json:
336             schema: { $ref: "#/components/schemas/Order" }
337           application/prs.hal-forms+json:
338             schema: { $ref: "#/components/schemas/Order" }
339       "304":
340         description: The order has not changed since the ETag in If-None-Match.
341       "404": { $ref: "#/components/responses/NotFound" }
342   patch:
343     operationId: replaceOrder
344     summary: Change a working order
345     description: >-
346       A JSON merge patch (RFC 7396): only the fields sent are changed. If-Match with the
347       order's current ETag is required, so concurrent edits cannot overwrite one another.
348     tags: [orders]
349     security: [{ oauth2: [orders:write] }]
350     parameters:
351       - $ref: "#/components/parameters/If-Match"
352     requestBody:
353       required: true
354       content:
355         application/merge-patch+json:
356           schema: { $ref: "#/components/schemas/OrderAmendment" }
357     responses:
358       "200":
359         description: The changed order.
360         headers:
361           ETag: { $ref: "#/components/headers/ETag" }
362         content:
363           application/hal+json:
364             schema: { $ref: "#/components/schemas/Order" }
365       "404": { $ref: "#/components/responses/NotFound" }
366       "409": { $ref: "#/components/responses/Conflict" }
367       "412": { $ref: "#/components/responses/PreconditionFailed" }
368       "422": { $ref: "#/components/responses/UnprocessableContent" }
369       "428": { $ref: "#/components/responses/PreconditionRequired" }
370
371 /accounts/{accountId}/orders/{orderId}/cancellation:
372   parameters:
373     - $ref: "#/components/parameters/accountId"
374     - $ref: "#/components/parameters/orderId"
375   post:
376     operationId: cancelOrder
377     summary: Request cancellation
378     description: >-
379       Cancelling is asynchronous: the order moves to PENDING_CANCEL at once and to CANCELLED
380       shortly after, and it may still fill in between. Orders are never deleted.
381     tags: [orders]
382     security: [{ oauth2: [orders:write] }]

```

```

383     responses:
384       "202":
385         description: Cancellation requested. Location points at the order.
386         headers:
387           Location: { $ref: "#/components/headers/Location" }
388         content:
389           application/hal+json:
390             schema: { $ref: "#/components/schemas/Order" }
391       "404": { $ref: "#/components/responses/NotFound" }
392       "409": { $ref: "#/components/responses/Conflict" }
393
394 /accounts/{accountId}/order-previews:
395   parameters:
396     - $ref: "#/components/parameters/accountId"
397   post:
398     operationId: previewOrder
399     summary: Preview an order
400     description: >=
401       Estimates cost and checks every rule without placing anything. A preview that would be
402       refused is still a 200, with `acceptable: false` and the reasons.
403     tags: [orders]
404     security: [{ oauth2: [orders:read] }]
405     requestBody:
406       required: true
407       content:
408         application/json:
409           schema: { $ref: "#/components/schemas/OrderRequest" }
410     responses:
411       "200":
412         description: The estimate.
413         content:
414           application/hal+json:
415             schema: { $ref: "#/components/schemas/OrderPreview" }
416       "400": { $ref: "#/components/responses/BadRequest" }
417       "404": { $ref: "#/components/responses/NotFound" }
418
419 /accounts/{accountId}/order-events:
420   parameters:
421     - $ref: "#/components/parameters/accountId"
422   get:
423     operationId: streamOrderEvents
424     summary: Stream order events
425     description: >=
426       Server-Sent Events for every order in the account. Reconnect with Last-Event-ID to
427       receive only the events you missed.
428     tags: [order-events]
429     security: [{ oauth2: [orders:read] }]
430     parameters:
431       - name: Last-Event-ID
432         in: header
433         schema: { type: integer, minimum: 0 }
434     responses:
435       "200":
436         description: An unending stream of events.
437         content:
438           text/event-stream:
439             itemSchema:
440               type: object
441               required: [id, event, data]
442               properties:
443                 id: { type: string }
444                 event:
445                   type: string
446                   enum: [order.created, order.replaced, order.partially-filled, order.filled,
447                     order.cancel-requested, order.cancelled]
448                 data:
449                   type: string
450                   contentType: application/json
451                   contentSchema: { $ref: "#/components/schemas/OrderEventData" }

```

```

452         "404": { $ref: "#/components/responses/NotFound" }
453
454 /instruments:
455   get:
456     operationId: searchInstruments
457     summary: Search instruments
458     tags: [market-data]
459     security: [{ oauth2: [market-data:read] }]
460     parameters:
461       - name: symbol
462         in: query
463         schema: { type: string, examples: [AAPL] }
464       - name: type
465         in: query
466         schema: { $ref: "#/components/schemas/InstrumentType" }
467     responses:
468       "200":
469         description: Matching instruments.
470         content:
471           application/hal+json:
472             schema: { $ref: "#/components/schemas/InstrumentCollection" }
473
474 /instruments/{instrumentId}:
475   parameters:
476     - $ref: "#/components/parameters/instrumentId"
477   get:
478     operationId: getInstrument
479     summary: Get an instrument
480     tags: [market-data]
481     security: [{ oauth2: [market-data:read] }]
482     responses:
483       "200":
484         description: The instrument, with links to its quote and option chain or underlying.
485         content:
486           application/hal+json:
487             schema: { $ref: "#/components/schemas/Instrument" }
488       "404": { $ref: "#/components/responses/NotFound" }
489
490 /instruments/{instrumentId}/quote:
491   parameters:
492     - $ref: "#/components/parameters/instrumentId"
493   get:
494     operationId: getQuote
495     summary: Get a quote
496     description: >–
497       The shape depends on the API version: version 1 (the deprecated default) is flat;
498       version 2 groups price and size into bid and ask levels and adds the day's change.
499     tags: [market-data]
500     security: [{ oauth2: [market-data:read] }]
501     parameters:
502       - $ref: "#/components/parameters/API-Version"
503       - $ref: "#/components/parameters/If-None-Match"
504     responses:
505       "200":
506         description: The latest quote. Cacheable privately for one second.
507         headers:
508           Cache-Control:
509             schema: { type: string, examples: ["max-age=1, private"] }
510           ETag: { $ref: "#/components/headers/ETag" }
511           Deprecation: { $ref: "#/components/headers/Deprecation" }
512           Sunset: { $ref: "#/components/headers/Sunset" }
513         content:
514           application/hal+json:
515             schema:
516               oneOf:
517                 - $ref: "#/components/schemas/Quote"
518                 - $ref: "#/components/schemas/QuoteV1"
519       "304":
520         description: The quote has not changed.

```

```

521     "400": { $ref: "#/components/responses/BadRequest" }
522     "404": { $ref: "#/components/responses/NotFound" }
523
524 /instruments/{instrumentId}/option-chain:
525   parameters:
526     - $ref: "#/components/parameters/instrumentId"
527   get:
528     operationId: getOptionChain
529     summary: Get an option chain
530     description: >-
531       Contracts for one expiration (the nearest by default). Every available expiration is
532       linked with the relation `bk:expiration`, named by its date.
533     tags: [market-data]
534     security: [{ oauth2: [market-data:read] }]
535     parameters:
536       - name: expiration
537         in: query
538         schema: { type: string, format: date }
539     responses:
540       "200":
541         description: The option contracts.
542         content:
543           application/hal+json:
544             schema: { $ref: "#/components/schemas/InstrumentCollection" }
545       "404": { $ref: "#/components/responses/NotFound" }
546
547 components:
548   securitySchemes:
549     oauth2:
550       type: oauth2
551       description: >-
552         OAuth 2.1. Web apps sign customers in through their backend, a confidential client,
553         with the authorization code grant and PKCE; a customer's own scripts and agents use
554         client credentials registered to that customer. Tokens carry `customer_id` and, for
555         delegated clients, `trading_limit`.
556       oauth2MetadataUrl: http://localhost:9000/.well-known/oauth-authorization-server
557       flows:
558         authorizationCode:
559           authorizationUrl: http://localhost:9000/oauth2/authorize
560           tokenUrl: http://localhost:9000/oauth2/token
561           refreshUrl: http://localhost:9000/oauth2/token
562         scopes:
563           market-data:read: Instruments, quotes and option chains
564           accounts:read: Accounts, balances, positions and transactions
565           orders:read: Orders, previews and order events
566           orders:write: Place, change and cancel orders
567         clientCredentials:
568           tokenUrl: http://localhost:9000/oauth2/token
569         scopes:
570           market-data:read: Instruments, quotes and option chains
571           accounts:read: Accounts, balances, positions and transactions
572           orders:read: Orders, previews and order events
573           orders:write: Place, change and cancel orders
574
575   parameters:
576     accountId:
577       name: accountId
578       in: path
579       required: true
580       schema: { type: string, examples: [ACC-1001] }
581     orderId:
582       name: orderId
583       in: path
584       required: true
585       schema: { type: string, examples: [ORD-100002] }
586     instrumentId:
587       name: instrumentId
588       in: path
589       required: true

```

```

590     schema: { type: string, examples: [EQ-AAPL] }
591 cursor:
592   name: cursor
593   in: query
594   description: Opaque. Take it from a `next` link; never build one.
595   schema: { type: string }
596 limit:
597   name: limit
598   in: query
599   schema: { type: integer, minimum: 1, maximum: 100, default: 25 }
600 Idempotency-Key:
601   name: Idempotency-Key
602   in: header
603   required: true
604   description: A unique value per new order (a UUID works well). Retries reuse it.
605   schema: { type: string, minLength: 1, maxLength: 255 }
606 If-Match:
607   name: If-Match
608   in: header
609   required: true
610   description: The order's current ETag.
611   schema: { type: string }
612 If-None-Match:
613   name: If-None-Match
614   in: header
615   description: An ETag from an earlier response; an unchanged resource returns 304.
616   schema: { type: string }
617 API-Version:
618   name: API-Version
619   in: header
620   description: The API version to use. Defaults to 1 (deprecated).
621   schema: { type: string, enum: ["1", "2"], default: "1" }
622
623 headers:
624   X-Request-Id:
625     description: The request's ID, echoed from the request or generated.
626     schema: { type: string }
627   Location:
628     description: The URL of the resource created or affected.
629     schema: { type: string, format: uri }
630   ETag:
631     description: The representation's entity tag.
632     schema: { type: string }
633   Idempotent-Replayed:
634     description: true when the response replays an earlier request with the same Idempotency-Key.
635     schema: { type: string, enum: ["true", "false"] }
636   RateLimit:
637     description: 'Remaining quota and seconds until reset, e.g. "default";r=117;t=42.'
638     schema: { type: string }
639   RateLimit-Policy:
640     description: 'The quota policy, e.g. "default";q=120;w=60.'
641     schema: { type: string }
642   Retry-After:
643     description: Seconds to wait before retrying.
644     schema: { type: integer }
645   Deprecation:
646     description: When this version was deprecated (RFC 9745), e.g. @1788220800.
647     schema: { type: string }
648   Sunset:
649     description: When this version stops working (RFC 8594).
650     schema: { type: string }
651
652 responses:
653   BadRequest:
654     description: The request is malformed or fails validation.
655     content:
656       application/problem+json:
657         schema: { $ref: "#/components/schemas/Problem" }
658         examples:

```

```

659         invalidFields:
660             dataValue:
661                 type: https://docs.brokerage.example/problems/invalid-request
662                 title: The request is not valid
663                 status: 400
664                 detail: One or more fields are invalid
665                 instance: /accounts/ACC-1001/orders
666                 requestId: 7f3c2a90-4a1e-4c1b-9d3e-2b8f1c6e0a11
667                 errors:
668                     - { field: quantity, message: must be greater than 0 }
669 Unauthorized:
670     description: No valid bearer token. See WWW-Authenticate.
671     headers:
672         WWW-Authenticate:
673             schema: { type: string }
674     content:
675         application/problem+json:
676             schema: { $ref: "#/components/schemas/Problem" }
677 Forbidden:
678     description: The token lacks the scope, or the order exceeds the client's trading limit.
679     content:
680         application/problem+json:
681             schema: { $ref: "#/components/schemas/Problem" }
682 NotFound:
683     description: The resource does not exist, or belongs to another customer.
684     content:
685         application/problem+json:
686             schema: { $ref: "#/components/schemas/Problem" }
687 Conflict:
688     description: The order's state does not allow this change.
689     content:
690         application/problem+json:
691             schema: { $ref: "#/components/schemas/Problem" }
692 PreconditionFailed:
693     description: The ETag in If-Match is stale.
694     content:
695         application/problem+json:
696             schema: { $ref: "#/components/schemas/Problem" }
697 PreconditionRequired:
698     description: If-Match is missing.
699     content:
700         application/problem+json:
701             schema: { $ref: "#/components/schemas/Problem" }
702 UnprocessableContent:
703     description: The request is well formed but breaks a business rule.
704     content:
705         application/problem+json:
706             schema: { $ref: "#/components/schemas/Problem" }
707     examples:
708         buyingPower:
709             dataValue:
710                 type: https://docs.brokerage.example/problems/insufficient-buying-power
711                 title: The account does not have enough buying power
712                 status: 422
713                 detail: This order needs 250000.00 but only 52913.10 is available
714                 instance: /accounts/ACC-1001/orders
715                 requestId: b1b72c27-9103-4236-9cc0-c2b21d4d86bf
716                 required: "250000.00"
717                 available: "52913.10"
718 TooManyRequests:
719     description: The client's quota is used up.
720     headers:
721         Retry-After: { $ref: "#/components/headers/Retry-After" }
722     content:
723         application/problem+json:
724             schema: { $ref: "#/components/schemas/Problem" }
725
726 schemas:
727     Decimal:

```

```

728     description: An exact decimal number, as a string.
729     type: string
730     pattern: '^-[0-9]+(\\.[0-9]+)?$'
731     examples: ["187.50"]
732 Link:
733     type: object
734     required: [href]
735     properties:
736         href: { type: string }
737         templated: { type: boolean }
738         type: { type: string }
739         name: { type: string }
740         title: { type: string }
741 Links:
742     description: HAL links, keyed by relation. A relation may hold one link or several.
743     type: object
744     properties:
745         self: { $ref: "#/components/schemas/Link" }
746         curies:
747             type: array
748             items: { $ref: "#/components/schemas/Link" }
749     additionalProperties:
750         anyOf:
751             - $ref: "#/components/schemas/Link"
752             - type: array
753               items: { $ref: "#/components/schemas/Link" }
754 Templates:
755     description: HAL-FORMS templates, present only for application/prs.hal-forms+json.
756     type: object
757     additionalProperties:
758         type: object
759         required: [method]
760         properties:
761             method: { type: string }
762             contentType: { type: string }
763             target: { type: string }
764             properties: { type: array, items: { type: object } }
765 Problem:
766     description: An RFC 9457 problem document. Extension members depend on the type.
767     type: object
768     required: [type, title, status]
769     properties:
770         type: { type: string, format: uri }
771         title: { type: string }
772         status: { type: integer }
773         detail: { type: string }
774         instance: { type: string, format: uri-reference }
775         requestId: { type: string }
776         errors:
777             type: array
778             items:
779                 type: object
780                 required: [field, message]
781                 properties:
782                     field: { type: string }
783                     message: { type: string }
784 ApiRoot:
785     type: object
786     required: [name, _links]
787     properties:
788         name: { type: string }
789         description: { type: string }
790         _links: { $ref: "#/components/schemas/Links" }
791 ProtectedResourceMetadata:
792     type: object
793     required: [resource, authorization_servers]
794     properties:
795         resource: { type: string, format: uri }
796         resource_name: { type: string }

```

```

797     authorization_servers: { type: array, items: { type: string, format: uri } }
798     scopes_supported: { type: array, items: { type: string } }
799     bearer_methods_supported: { type: array, items: { type: string } }
800     tls_client_certificate_bound_access_tokens: { type: boolean }
801
802 # ----- accounts
803 Account:
804     type: object
805     required: [id, number, type, registration, currency, _links]
806     properties:
807         id: { type: string }
808         number:
809             type: string
810             description: Masked; only the last four digits are sent
811             examples: ["****4821"]
812         nickname: { type: string }
813         type: { type: string, enum: [CASH, MARGIN] }
814         registration: { type: string, enum: [INDIVIDUAL, JOINT, ROTH_IRA, TRADITIONAL_IRA] }
815         currency: { type: string, pattern: "^[A-Z]{3}$" }
816         openedOn: { type: string, format: date }
817         _links: { $ref: "#/components/schemas/Links" }
818         _templates: { $ref: "#/components/schemas/Templates" }
819 AccountCollection:
820     type: object
821     required: [_links]
822     properties:
823         _embedded:
824             type: object
825             properties:
826                 bk:accounts: { type: array, items: { $ref: "#/components/schemas/Account" } }
827         _links: { $ref: "#/components/schemas/Links" }
828 Balances:
829     type: object
830     required: [accountId, currency, cash, reservedForOrders, buyingPower, positionsMarketValue,
831         totalEquity, unrealizedPnl, asOf, _links]
832     properties:
833         accountId: { type: string }
834         currency: { type: string }
835         cash: { $ref: "#/components/schemas/Decimal" }
836         reservedForOrders: { $ref: "#/components/schemas/Decimal" }
837         buyingPower: { $ref: "#/components/schemas/Decimal" }
838         positionsMarketValue: { $ref: "#/components/schemas/Decimal" }
839         totalEquity: { $ref: "#/components/schemas/Decimal" }
840         unrealizedPnl: { $ref: "#/components/schemas/Decimal" }
841         asOf: { type: string, format: date-time }
842         _links: { $ref: "#/components/schemas/Links" }
843 Position:
844     type: object
845     required: [instrumentId, symbol, instrumentType, quantity, averageCost, costBasis, lastPrice,
846         marketValue, unrealizedPnl, unrealizedPnlPercent, _links]
847     properties:
848         instrumentId: { type: string }
849         symbol: { type: string }
850         instrumentType: { $ref: "#/components/schemas/InstrumentType" }
851         quantity: { $ref: "#/components/schemas/Decimal" }
852         averageCost: { $ref: "#/components/schemas/Decimal" }
853         costBasis: { $ref: "#/components/schemas/Decimal" }
854         lastPrice: { $ref: "#/components/schemas/Decimal" }
855         marketValue: { $ref: "#/components/schemas/Decimal" }
856         unrealizedPnl: { $ref: "#/components/schemas/Decimal" }
857         unrealizedPnlPercent: { $ref: "#/components/schemas/Decimal" }
858         _links: { $ref: "#/components/schemas/Links" }
859 PositionCollection:
860     type: object
861     required: [_links]
862     properties:
863         _embedded:
864             type: object
865             properties:

```

```

866         bk:positions: { type: array, items: { $ref: "#/components/schemas/Position" } }
867     _links: { $ref: "#/components/schemas/Links" }
868 TransactionType:
869     type: string
870     enum: [DEPOSIT, WITHDRAWAL, TRADE, DIVIDEND, FEE]
871 Transaction:
872     type: object
873     required: [id, type, description, amount, occurredAt, settlesOn]
874     properties:
875         id: { type: string }
876         type: { $ref: "#/components/schemas/TransactionType" }
877         description: { type: string }
878         amount: { $ref: "#/components/schemas/Decimal", description: Positive into the account, negative out }
879         instrumentId: { type: string }
880         symbol: { type: string }
881         quantity: { $ref: "#/components/schemas/Decimal" }
882         price: { $ref: "#/components/schemas/Decimal" }
883         orderId: { type: string }
884         occurredAt: { type: string, format: date-time }
885         settlesOn: { type: string, format: date }
886 TransactionPage:
887     type: object
888     required: [_links]
889     properties:
890         _embedded:
891             type: object
892             properties:
893                 bk:transactions: { type: array, items: { $ref: "#/components/schemas/Transaction" } }
894         _links: { $ref: "#/components/schemas/Links" }
895
896 # ----- market data
897 InstrumentType:
898     type: string
899     enum: [EQUITY, ETF, OPTION]
900 Instrument:
901     type: object
902     required: [id, symbol, name, type, currency, tradable, fractionable, multiplier, _links]
903     properties:
904         id: { type: string }
905         symbol: { type: string }
906         name: { type: string }
907         type: { $ref: "#/components/schemas/InstrumentType" }
908         exchange: { type: string, description: ISO 10383 market identifier code }
909         currency: { type: string }
910         tradable: { type: boolean }
911         fractionable: { type: boolean }
912         multiplier: { $ref: "#/components/schemas/Decimal" }
913         option:
914             type: object
915             required: [underlyingId, right, strike, expiration]
916             properties:
917                 underlyingId: { type: string }
918                 underlyingSymbol: { type: string }
919                 right: { type: string, enum: [CALL, PUT] }
920                 strike: { $ref: "#/components/schemas/Decimal" }
921                 expiration: { type: string, format: date }
922                 style: { type: string, enum: [AMERICAN, EUROPEAN] }
923         _links: { $ref: "#/components/schemas/Links" }
924 InstrumentCollection:
925     type: object
926     required: [_links]
927     properties:
928         _embedded:
929             type: object
930             properties:
931                 bk:instruments: { type: array, items: { $ref: "#/components/schemas/Instrument" } }
932         _links: { $ref: "#/components/schemas/Links" }
933 QuoteV1:
934     description: Version 1 (deprecated). Flat fields.

```

```

935     type: object
936     required: [instrumentId, symbol, last, bid, ask, bidSize, askSize, volume, asOf]
937     properties:
938         instrumentId: { type: string }
939         symbol: { type: string }
940         last: { $ref: "#/components/schemas/Decimal" }
941         bid: { $ref: "#/components/schemas/Decimal" }
942         ask: { $ref: "#/components/schemas/Decimal" }
943         bidSize: { $ref: "#/components/schemas/Decimal" }
944         askSize: { $ref: "#/components/schemas/Decimal" }
945         volume: { type: integer }
946         asOf: { type: string, format: date-time }
947         _links: { $ref: "#/components/schemas/Links" }
948 Quote:
949     description: Version 2.
950     type: object
951     required: [instrumentId, symbol, last, bid, ask, change, changePercent, volume, asOf]
952     properties:
953         instrumentId: { type: string }
954         symbol: { type: string }
955         last: { $ref: "#/components/schemas/Decimal" }
956         bid: { $ref: "#/components/schemas/PriceLevel" }
957         ask: { $ref: "#/components/schemas/PriceLevel" }
958         change: { $ref: "#/components/schemas/Decimal" }
959         changePercent: { $ref: "#/components/schemas/Decimal" }
960         volume: { type: integer }
961         asOf: { type: string, format: date-time }
962         _links: { $ref: "#/components/schemas/Links" }
963 PriceLevel:
964     type: object
965     required: [price, size]
966     properties:
967         price: { $ref: "#/components/schemas/Decimal" }
968         size: { $ref: "#/components/schemas/Decimal" }
969
970 # ----- orders
971 OrderStatus:
972     type: string
973     enum: [OPEN, PARTIALLY_FILLED, FILLED, PENDING_CANCEL, CANCELLED, REJECTED]
974 OrderSide:
975     type: string
976     enum: [BUY, SELL]
977 OrderType:
978     type: string
979     enum: [MARKET, LIMIT, STOP, STOP_LIMIT]
980 TimeInForce:
981     type: string
982     enum: [DAY, GTC]
983     default: DAY
984 OrderRequest:
985     type: object
986     required: [instrumentId, side, type, quantity]
987     properties:
988         instrumentId: { type: string, minLength: 1 }
989         side: { $ref: "#/components/schemas/OrderSide" }
990         type: { $ref: "#/components/schemas/OrderType" }
991         timeInForce: { $ref: "#/components/schemas/TimeInForce" }
992         quantity: { $ref: "#/components/schemas/Decimal", description: Shares, or contracts for options }
993         limitPrice: { $ref: "#/components/schemas/Decimal", description: Required for LIMIT and STOP_LIMIT }
994         stopPrice: { $ref: "#/components/schemas/Decimal", description: Required for STOP and STOP_LIMIT }
995         clientOrderId: { type: string, maxLength: 64, pattern: "^[A-Za-z0-9._-]*$" }
996 OrderAmendment:
997     type: object
998     minProperties: 1
999     properties:
1000         quantity: { $ref: "#/components/schemas/Decimal" }
1001         limitPrice: { $ref: "#/components/schemas/Decimal" }
1002         stopPrice: { $ref: "#/components/schemas/Decimal" }
1003         timeInForce: { $ref: "#/components/schemas/TimeInForce" }

```

```

1004 Fill:
1005     type: object
1006     required: [id, quantity, price, executedAt]
1007     properties:
1008         id: { type: string }
1009         quantity: { $ref: "#/components/schemas/Decimal" }
1010         price: { $ref: "#/components/schemas/Decimal" }
1011         executedAt: { type: string, format: date-time }
1012 Order:
1013     type: object
1014     required: [id, accountId, instrumentId, symbol, instrumentType, side, type, timeInForce, quantity,
1015 status, filledQuantity, remainingQuantity, fills, createdAt, updatedAt, _links]
1016     properties:
1017         id: { type: string }
1018         accountId: { type: string }
1019         clientOrderId: { type: string }
1020         instrumentId: { type: string }
1021         symbol: { type: string }
1022         instrumentType: { $ref: "#/components/schemas/InstrumentType" }
1023         side: { $ref: "#/components/schemas/OrderSide" }
1024         type: { $ref: "#/components/schemas/OrderType" }
1025         timeInForce: { $ref: "#/components/schemas/TimeInForce" }
1026         quantity: { $ref: "#/components/schemas/Decimal" }
1027         limitPrice: { $ref: "#/components/schemas/Decimal" }
1028         stopPrice: { $ref: "#/components/schemas/Decimal" }
1029         status: { $ref: "#/components/schemas/OrderStatus" }
1030         filledQuantity: { $ref: "#/components/schemas/Decimal" }
1031         remainingQuantity: { $ref: "#/components/schemas/Decimal" }
1032         averageFillPrice: { $ref: "#/components/schemas/Decimal" }
1033         fills: { type: array, items: { $ref: "#/components/schemas/Fill" } }
1034         createdAt: { type: string, format: date-time }
1035         updatedAt: { type: string, format: date-time }
1036         _links: { $ref: "#/components/schemas/Links" }
1037         _templates: { $ref: "#/components/schemas/Templates" }
1038 OrderPage:
1039     type: object
1040     required: [_links]
1041     properties:
1042         _embedded:
1043             type: object
1044             properties:
1045                 bk:orders: { type: array, items: { $ref: "#/components/schemas/Order" } }
1046         _links: { $ref: "#/components/schemas/Links" }
1047 OrderPreview:
1048     type: object
1049     required: [accountId, instrumentId, side, type, quantity, acceptable, issues]
1050     properties:
1051         accountId: { type: string }
1052         instrumentId: { type: string }
1053         symbol: { type: string }
1054         side: { $ref: "#/components/schemas/OrderSide" }
1055         type: { $ref: "#/components/schemas/OrderType" }
1056         quantity: { $ref: "#/components/schemas/Decimal" }
1057         estimatedPrice: { $ref: "#/components/schemas/Decimal" }
1058         estimatedNotional: { $ref: "#/components/schemas/Decimal" }
1059         estimatedCommission: { $ref: "#/components/schemas/Decimal" }
1060         estimatedTotal: { $ref: "#/components/schemas/Decimal" }
1061         buyingPower: { $ref: "#/components/schemas/Decimal" }
1062         acceptable: { type: boolean }
1063         issues:
1064             type: array
1065             items:
1066                 type: object
1067                 required: [type, detail]
1068                 properties:
1069                     type: { type: string, format: uri }
1070                     detail: { type: string }
1071         _links: { $ref: "#/components/schemas/Links" }
1072 OrderEventData:

```

```
1073     type: object
1074     required: [orderId, status, filledQuantity, remainingQuantity, occurredAt, _links]
1075     properties:
1076       orderId: { type: string }
1077       status: { $ref: "#/components/schemas/OrderStatus" }
1078       filledQuantity: { $ref: "#/components/schemas/Decimal" }
1079       remainingQuantity: { $ref: "#/components/schemas/Decimal" }
1080       averageFillPrice: { $ref: "#/components/schemas/Decimal" }
1081       occurredAt: { type: string, format: date-time }
1082     _links:
1083       type: object
1084       required: [order]
1085       properties:
1086         order: { $ref: "#/components/schemas/Link" }
```